
Σύγκριση και αξιολόγηση Προηγμένων Αλγορίθμων Βαθιάς Ενισχυτικής Μάθησης

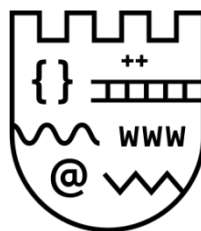
Comparison and Evaluation of Advanced Deep Reinforcement
Learning Algorithms

Γκιουζέλης Χρυσάφης

AEM: 4223

Επιβλέπων Καθηγητής:

Ιωάννης Βλαχάβας



[Ιούλιος] – [2025]

Με πλήρη επίγνωση των συνεπειών του νόμου περί πνευματικών δικαιωμάτων, δηλώνω ρητά ότι η παρούσα πτυχιακή εργασία, καθώς και τα ηλεκτρονικά αρχεία και πηγαίοι κώδικες που αναπτύχθηκαν ή τροποποιήθηκαν στο πλαίσιο αυτής της εργασίας, αποτελεί αποκλειστικά προϊόν προσωπικής μου εργασίας, δεν προσβάλλει κάθε μορφής δικαιώματα διανοητικής ιδιοκτησίας, προσωπικότητας και προσωπικών δεδομένων τρίτων, δεν περιέχει έργα/εισφορές τρίτων για τα οποία απαιτείται άδεια των δημιουργών/δικαιούχων και δεν είναι προϊόν μερικής ή ολικής αντιγραφής, οι πηγές δε που χρησιμοποιήθηκαν περιορίζονται στις βιβλιογραφικές αναφορές και μόνον και πληρούν τους κανόνες της επιστημονικής παράθεσης. Τα σημεία όπου έχω χρησιμοποιήσει ιδέες, κείμενο, αρχεία ή/και πηγές άλλων συγγραφέων, αναφέρονται ευδιάκριτα στο κείμενο με την κατάλληλη παραπομπή και η σχετική αναφορά περιλαμβάνεται στο τμήμα των βιβλιογραφικών αναφορών με πλήρη περιγραφή. Αναλαμβάνω πλήρως, ατομικά και προσωπικά, όλες τις νομικές και διοικητικές συνέπειες που δύναται να προκύψουν στην περίπτωση κατά την οποία αποδειχθεί, διαχρονικά, ότι η εργασία αυτή ή τμήμα της δεν μου ανήκει διότι είναι προϊόν λογοκλοπής.

Περίληψη

Η ενισχυτική μάθηση αποτελεί ένα ραγδαίως εξελισσόμενο πεδίο της τεχνητής νοημοσύνης που έχει πολλαπλές εφαρμογές σε τομείς όπως αυτόνομα συστήματα, ρομποτική και παιχνίδια στρατηγικής. Η παρούσα πτυχιακή εργασία εστιάζει στην ανάλυση, σύγκριση και αξιολόγηση ορισμένων προηγμένων αλγορίθμων ενισχυτικής μάθησης. Εμπεριέχονται αλγόριθμοι και από τις τρεις βασικές κατηγορίες: τους αλγορίθμους βασισμένους στην συνάρτηση αξίας (Value-Based), τους αλγορίθμους Policy Gradient και αυτούς που δημιουργούν ένα μοντέλο του περιβάλλοντος (Model-Based). Οι αλγόριθμοι, μετά από σχετική παρουσίαση του θεωρητικού υποβάθρου τους, εφαρμόζονται, μέσω της βιβλιοθήκης DI-engine, σε περιβάλλοντα του OpenAI Gym προκειμένου να αξιολογηθεί η απόδοση τους βάσει κριτηρίων όπως ο χρόνος εκπαίδευσης και ο αριθμός των ενεργειών που εκτελεί ο πράκτορας στο περιβάλλον.

Abstract

Reinforcement learning is a rapidly evolving field of artificial intelligence that has multiple applications in areas such as autonomous systems, robotics and strategy games. This thesis focuses on the analysis, comparison and evaluation of a select number of advanced reinforcement learning algorithms. Algorithms from all three main categories are included: Value-Based algorithms, Policy Gradient algorithms and Model-Based algorithms. The algorithms, after a presentation of their theoretical background, are applied using DL-engine framework, to OpenAI Gym environments in order to evaluate their performance based on criteria such as training time and the number of environment steps performed by the agent.

Ευχαριστίες

Θα ήθελα να εκφράσω τις ευχαριστίες μου στον επιβλέποντα καθηγητή μου, κ. Ιωάννη Βλαχάβα που μου έδωσε την ευκαιρία να εκπονήσω την πτυχιακή μου εργασία κάτω από την καθοδήγηση του, στο Εργαστήριο Ευφυών Συστημάτων. Επίσης, θα ήθελα να ευχαριστήσω τον συνεπιβλέποντα της εργασίας μου, κ. Βασίλη Κοχλιαρίδη, υποψήφιο διδάκτορα, για την πολύτιμη βοήθεια του καθ' όλη την διάρκεια της εργασίας και την διαθεσιμότητά του όποτε παρουσιαζόταν ανάγκη. Τέλος, θα ήθελα να ευχαριστήσω την οικογένειά και τους φίλους μου για τη στήριξη και συμπαράσταση σε όλη τη διάρκεια των σπουδών μου στο Τμήμα Πληροφορικής.

Περιεχόμενα

1.	Εισαγωγή.....	1
2.	Μηχανική Μάθηση.....	3
2.1	Ορισμός της Μηχανικής Μάθησης.....	3
2.2	Μάθηση με Επίβλεψη.....	4
2.2.1	Γραμμική Παλινδρόμηση.....	4
2.2.2	Λογιστική Παλινδρόμηση.....	5
2.2.3	Support Vector Machines.....	6
2.2.4	Τεχνητά Νευρωνικά Δίκτυα.....	8
2.2.5	Συνελικτικά Νευρωνικά Δίκτυα.....	10
2.2.6	Αξιολόγηση Μοντέλων.....	10
2.3	Μάθηση χωρίς Επίβλεψη.....	12
2.3.1	Κανόνες Συσχέτισης.....	12
2.3.2	Ομάδες.....	13
2.4	Ενισχυτική Μάθηση.....	13
2.4.1	Μαρκοβιανές Διαδικασίες Απόφασης.....	14
2.4.2	Δυναμικός Προγραμματισμός.....	14
2.4.3	Μέθοδοι Χρονικών Διαφορών.....	14
2.4.4	Βαθιά Ενισχυτική Μάθηση.....	15
2.4.5	Deep Q-Network.....	15
2.4.6	Μέθοδοι Policy Gradient.....	16
2.4.7	Μέθοδοι Model-Based.....	17
3.	Προηγμένοι Αλγόριθμοι Βαθιάς Ενισχυτικής Μάθησης.....	19
3.1	Value-Based.....	19
3.1.1	R2D2.....	19
3.1.2	Agent57.....	19
3.2	Policy Gradient.....	20
3.2.1	Soft Actor-Critic (SAC).....	21
3.2.2	Phasic Policy Gradient (PPG).....	21
3.2.3	Robust Policy Optimization (RPO).....	22
3.2.4	IMPALA.....	23
3.2.5	DrQ-v2.....	24
3.3	Model-Based.....	25
3.3.1	MuZero.....	25

3.3.2	EfficientZero V2	26
3.3.3	DreamerV3	27
3.3.4	Plan2Explore	28
3.4	Exploration Module.....	29
3.4.1	Intrinsic Curiosity Module (ICM)	29
3.4.2	Random Network Distillation (RND).....	30
3.4.3	Efficient Random Encoders (RE3)	30
4.	Σύγκριση και αξιολόγηση Προηγμένων Αλγορίθμων Βαθιάς Ενισχυτικής Μάθησης.....	33
4.1	Βιβλιοθήκες	33
4.1.1	DI-Engine.....	33
4.1.2	PyTorch	35
4.1.3	NumPy	35
4.2	Περιβάλλοντα εκπαίδευσης.....	36
4.3	Αλυσίδα διεργασιών (Pipeline)	39
4.4	Πειράματα και Αποτελέσματα	44
4.4.1	Αλγόριθμοι και Υπερ-παράμετροι.....	44
4.4.2	Αποτελέσματα πειραμάτων.....	48
4.4.3	Συμπεράσματα	49
5.	Συμπεράσματα και Μελλοντική Έρευνα	51
6.	Αναφορές	53

Κεφάλαιο 1

Εισαγωγή

Η ενισχυτική μάθηση (Reinforcement Learning) αποτελεί μία εκ των τριών κύριων κατηγοριών της μηχανικής μάθησης (Machine Learning) που γνωρίζει ραγδαία ανάπτυξη τα τελευταία χρόνια. Η διαφοροποίηση της από τις άλλες δύο έγκειται στον τρόπο που εκπαιδεύεται ο πράκτορας, καθώς δεν μαθαίνει απλώς από ένα στατικό σύνολο δεδομένων, αλλά μέσα από συνεχή εξερεύνηση και αλληλεπίδραση με το περιβάλλον εκπαίδευσης. Το περιβάλλον εκπαίδευσης μπορεί να είναι οποιοδήποτε πρόβλημα μπορεί να μοντελοποιηθεί, έτσι ώστε να τροφοδοτεί τον πράκτορα με ανταμοιβές, όποτε αυτός επιλέγει την κατάλληλη ενέργεια προς επίλυση του προβλήματος. Συνεπώς στόχος του πράκτορα είναι η μεγιστοποίηση των αθροιστικών ανταμοιβών μέσα από την επιλογή ενεργειών, χρησιμοποιώντας κάποια πολιτική, δηλαδή κάποια στρατηγική, καθώς μεταβαίνει στις διάφορες καταστάσεις του περιβάλλοντος. Η ενισχυτική μάθηση είναι κατάλληλη για προβλήματα λήψης αποφάσεων που συναντώνται στην ρομποτική, αυτόνομο έλεγχο ή παιχνίδια στρατηγικής [2].

Με την ανάπτυξη της υπολογιστικής ισχύος και τις τεχνικές διαχείρισης μεγάλων δεδομένων, έχει δημιουργηθεί μεγάλο πλήθος προηγμένων αλγορίθμων ενισχυτικής μάθησης. Κάθε αλγόριθμος προσεγγίζει την εξερεύνηση και «εκμετάλλευση» των ανταμοιβών του περιβάλλοντος με διαφορετικό τρόπο και κατά συνέπεια η απόδοση του είναι μεγαλύτερη σε κάποια προβλήματα από ότι σε κάποια άλλα [2]. Η παρούσα πτυχιακή εργασία εστιάζει στην εφαρμογή ορισμένων προηγμένων αλγορίθμων ενισχυτικής μάθησης, στην σύγκριση μεταξύ τους και τέλος την αξιολόγηση και μελέτη των αποτελεσμάτων.

Αρχικά στο κεφάλαιο 2 γίνεται μία εκτενής περιγραφή της μηχανικής μάθησης και των τριών βασικών κατηγοριών της. Αναφέρονται τεχνικές μάθησης με επίβλεψη, μάθησης χωρίς επίβλεψη και ενισχυτικής μάθησης που αποτελεί και το ζήτημα της εργασίας. Αποτελεί ένα θεωρητικό υπόβαθρο, πάνω στο οποίο δομούνται απαραίτητες μέθοδοι, όπως τα νευρωνικά δίκτυα, που χρησιμοποιούν οι αλγόριθμοι της μηχανικής μάθησης και ειδικότερα οι αλγόριθμοι της ενισχυτικής μάθησης.

Στόχος του κεφαλαίου 3 είναι η ανάλυση ορισμένων από τους προηγμένους αλγόριθμους ενισχυτικής μάθησης που θα αξιολογηθούν. Ταξινομούνται σε κατηγορίες βάσει της λειτουργίας τους και της προσέγγισης που ακολουθούν. Συγκεκριμένα περιγράφονται αλγόριθμοι Value-Based, Policy Gradient και Model-Based, ενώ αναφέρονται στην ενότητα 3.4 και μέθοδοι αποδοτικότερης εξερεύνησης του περιβάλλοντος από τον πράκτορα.

Το κεφάλαιο 4 αφορά τις λεπτομέρειες υλοποίησης της εργασίας, εμβαθύνοντας στο framework που παρέχει τους αλγορίθμους και κάνει δυνατή την σύγκριση και αξιολόγηση τους κάτω από τις ίδιες συνθήκες. Επεξηγείται πλήρως η αλυσίδα διεργασιών (pipeline) της εκπαίδευσης ενός αλγορίθμου, με όλα τα συστατικά που την απαρτίζουν και συνεργάζονται προκειμένου να παραχθεί η εκπαιδευμένη πολιτική. Στην συνέχεια αναφέρονται τα πειράματα που εξήχθησαν, παρουσιάζονται τα αποτελέσματα σε γραφήματα και τα αντίστοιχα συμπεράσματα.

Η εργασία ολοκληρώνεται με το κεφάλαιο 5, στο οποίο αναγράφονται γενικότερα συμπεράσματα από την συνολική ανάλυση και πειράματα που προηγήθηκαν. Παράλληλα διατυπώνονται προτάσεις για μελλοντική έρευνα, με σκοπό την περαιτέρω εμβάθυνση σε επιμέρους θέματα, όπως εκπαίδευση περισσότερων και ισχυρότερων αλγορίθμων σε δυσκολότερα περιβάλλοντα, εύρεση βέλτιστων υπερ-παραμέτρων και εφαρμογή προσαρμοσμένων επεκτάσεων.

Κεφάλαιο 2

Μηχανική Μάθηση

Η μηχανική μάθηση είναι μια ερευνητική περιοχή της τεχνητής νοημοσύνης, η οποία γνωρίζει τεράστια ανάπτυξη τα τελευταία χρόνια. Αίτιο αυτής της ανάπτυξης είναι η εκθετική αύξηση των δεδομένων καθώς και η εξέλιξη των τεχνολογιών επεξεργασίας τους [2]. Το κεφάλαιο αυτό παρουσιάζει την έννοια της μηχανικής μάθησης και αποσαφηνίζει τις τρεις βασικές κατηγορίες στις οποίες διαιρείται. Η διαίρεση αυτή γίνεται βάσει του τύπου επίβλεψης που δέχονται κατά την εκπαίδευση: Μάθηση με Επίβλεψη (Supervised Learning), Μάθηση χωρίς Επίβλεψη (Unsupervised Learning) και Ενισχυτική Μάθηση (Reinforcement Learning). Η τελευταία κατηγορία θα αναλυθεί λεπτομερώς, καθώς οι αλγόριθμοι που θα αξιολογηθούν ανήκουν σε αυτήν.

2.1 Ορισμός της Μηχανικής Μάθησης

Η μηχανική μάθηση αφορά υπολογιστικές μεθόδους που εφαρμόζονται πάνω σε δεδομένα. Με βάση τα δεδομένα αυτά, δημιουργούνται μοντέλα ή πρότυπα που χρησιμοποιούνται στην επίλυση ενός προβλήματος. Η χρήση της ενδείκνυται ιδιαίτερα σε πολύπλοκα προβλήματα όπου ο ρητός προγραμματισμός θα ήταν απαιτητικός και μη αποδοτικός. Η απόδοση των μοντέλων μηχανικής μάθησης έγκειται στην ποιότητα και τον όγκο των δεδομένων εκπαίδευσης. Στην περίπτωση που τα δεδομένα δεν είναι κατάλληλα η διαδικασία της εκπαίδευσης, δεν θα επιφέρει τις επιθυμητές αποδόσεις [1].

Τα τελευταία χρόνια, λόγω του όγκου των δεδομένων και την εξέλιξη των υπολογιστικών συστημάτων, η μηχανική μάθηση έχει γνωρίσει τεράστια άνθιση. Αξιοποιείται σε πληθώρα σύγχρονων εφαρμογών, μερικές εξ αυτών είναι:

- Ιατρική, με αναγνώριση εικόνων για την παραγωγή διαγνώσεων.
- Πρόβλεψη χρηματοοικονομικών τιμών, ζήτησης προϊόντων.
- Ανίχνευση απάτης σε τραπεζικές συναλλαγές.
- Υπολογιστική όραση, όπως στην παραγωγή αυτοκινούμενων οχημάτων.
- Βιντεοπαιχνίδια, με την εκπαίδευση πρακτόρων που ανταγωνίζονται τον άνθρωπο.

Οι παραπάνω εφαρμογές υλοποιούνται με αλγορίθμους μηχανικής μάθησης, όπου η κάθε κατηγορία της εξειδικεύεται σε διαφορετικές εφαρμογές [1]. Όπως αναφέρθηκε στην εισαγωγή του κεφαλαίου, οι κατηγορίες που απαρτίζουν την μηχανική μάθηση είναι η μάθηση με επίβλεψη, η μάθηση χωρίς επίβλεψη και η ενισχυτική μάθηση.

Η μάθηση με επίβλεψη πραγματεύεται την προσέγγιση μιας άγνωστης συνάρτησης, η οποία καλείται συνάρτηση στόχος, αξιοποιώντας το σύνολο εκπαίδευσης. Το σύνολο εκπαίδευσης περιλαμβάνει τόσο δεδομένα εισόδου, όσο και εξόδου. Στην μάθηση χωρίς επίβλεψη το σύστημα έχοντας μόνο δεδομένα εισόδου στην διάθεση του, πρέπει να αναγνωρίσει συσχετίσεις στα δεδομένα και να δημιουργήσει πρότυπα. Η τελευταία κατηγορία, η ενισχυτική μάθηση αφορά την εκπαίδευση της συμπεριφοράς ενός πράκτορα. Ο πράκτορας αλληλεπιδρά με ένα περιβάλλον, το οποίο αναπαρίσταται με καταστάσεις, και καλείται να επιλέξει την κατάλληλη ενέργεια κάθε φορά προκειμένου να φτάσει στην τερματική κατάσταση. Για κάθε ενέργεια ο πράκτορας λαμβάνει μία ανταμοιβή (reward) και στόχος του είναι η μεγιστοποίηση της συνολικής ανταμοιβής του μέσω της

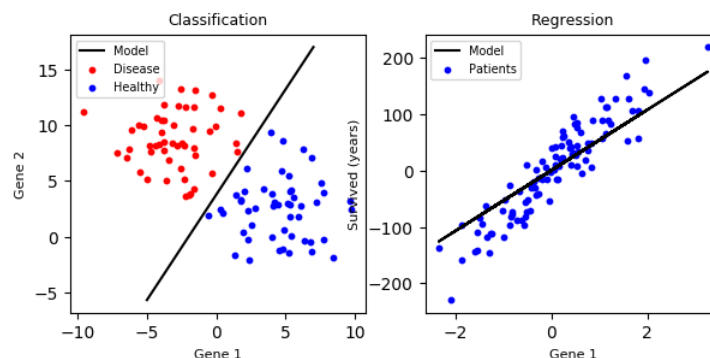
αλληλεπίδρασης με το περιβάλλον. Η σειρά ενεργειών καθορίζει την πολιτική (policy) του πράκτορα [1].

2.2 Μάθηση με Επίβλεψη

Οι αλγόριθμοι επιβλεπόμενης μάθησης εκπαιδεύονται μέσω ενός συνόλου δεδομένων με σκοπό την προσέγγιση μιας συνάρτησης στόχου (target function). Το πεδίο ορισμού της συνάρτησης αυτής ονομάζεται χώρος περιπτώσεων. Τα δεδομένα εκπαίδευσης αποτελούν υποσύνολο του χώρου περιπτώσεων, για τα οποία η έξοδος είναι γνωστή [2].

Η απλούστερη λύση στην προσέγγιση της συνάρτησης είναι η μάθηση με απομνημόνευση. Στην διαδικασία αυτή το σύνολο δεδομένων εκπαίδευσης αποθηκεύεται στην μνήμη. Αν έρθει κάποιο νέο δεδομένο δίνεται η τιμή της μεταβλητής εξόδου, μόνο αν το δεδομένο είναι ήδη αποθηκευμένο. Για την αντιμετώπιση της περίπτωσης άγνωστων δεδομένων η μάθηση με απομνημόνευση είναι αδύνατη και το πρόβλημα λύνεται με την μέθοδο της επαγωγικής μάθησης (inductive learning). Η μέθοδος της επαγωγικής μάθησης βασίζεται στην υπόθεση της επαγωγικής μάθησης, σύμφωνα με την οποία κάθε συνάρτηση που προσεγγίζει τη συνάρτηση στόχο ικανοποιητικά για ένα επαρκώς μεγάλο σύνολο εκπαίδευσης, θα την προσεγγίσει το ίδιο και για άγνωστα δεδομένα [2].

Τα προβλήματα που επιλύει η μάθηση με επίβλεψη κατατάσσονται σε δύο κατηγορίες, τα προβλήματα ταξινόμησης (classification) και τα προβλήματα παλινδρόμησης (regression). Στα προβλήματα ταξινόμησης κάθε περίπτωση πρέπει να κατηγοριοποιηθεί σε κάποια κλάση, συνεπώς παράγονται προβλεπτικά μοντέλα. Στα προβλήματα παλινδρόμησης, στόχος είναι η πρόβλεψη κάποιας συνεχούς αριθμητικής τιμής, όπως η τιμή ενός προϊόντος [2].



Σχήμα 2.1: Πρόβλημα ταξινόμησης και παλινδρόμησης.

2.2.1 Γραμμική Παλινδρόμηση

Τα μοντέλα γραμμικής παλινδρόμησης (linear regression) αναπαριστούν τις περιπτώσεις με μία συνάρτηση προσέγγισης τιμής h_{θ} , η οποία συμβολίζεται ως εξής:

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n = \theta \cdot x$$

Η σταθερά θ_0 ονομάζεται μεροληψία (bias) και οι $\theta_1 - \theta_n$ βάρη (weights). Τα $x_1 - x_n$ είναι οι μεταβλητές εισόδου από το σύνολο εκπαίδευσης [2].

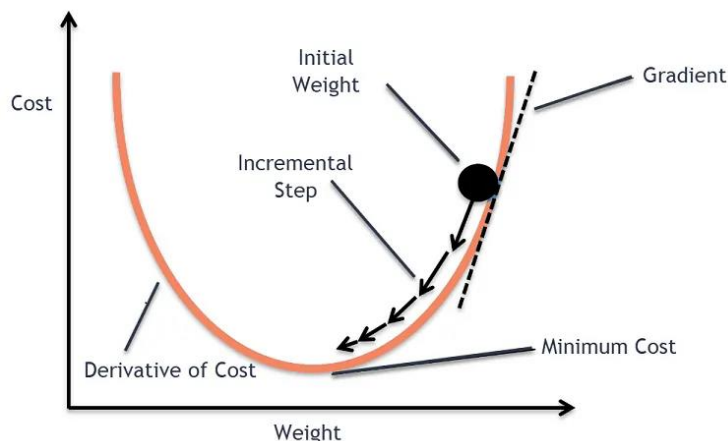
Για τη βελτιστοποίηση του μοντέλου, καθώς και για την αξιολόγηση των σφαλμάτων του, χρησιμοποιείται μία συνάρτηση κόστους J , η οποία υπολογίζει το μέσο τετραγωνικό σφάλμα (mean squared error) διαιρεμένο κατά δύο:

$$J = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Η μεταβλητή m είναι το πλήθος των παραδειγμάτων εκπαίδευσης και η μεταβλητή y η πραγματική τιμή για το παράδειγμα i . Το κόστος του μοντέλου συνεπώς αποτελεί το ένα δεύτερο του αθροίσματος των διαφορών της προσέγγισης της τιμής $h_{\theta}(x^{(i)})$, με την πραγματική τιμή $y^{(i)}$, υψωμένο στο τετράγωνο [2].

Για την βέλτιστη προσέγγιση της συνάρτησης στόχου, πρέπει το κόστος του μοντέλου να ελαχιστοποιηθεί. Αυτό μπορεί να επιτευχθεί με δύο τρόπους. Ο πρώτος τρόπος βελτιστοποίησης είναι με τη μέθοδο Ελαχίστων Τετραγώνων. Ένας άλλος επίσης δημοφιλής αλγόριθμος βελτιστοποίησης που δουλεύει καλά για μεγάλα σύνολα δεδομένων είναι η μέθοδος της Επικλινής Καθόδου (Gradient Descent). Πιο συγκεκριμένα, ο αλγόριθμος αυτός υπολογίζει επαναληπτικά την ενημέρωση των βαρών της h_{θ} για την ελαχιστοποίηση της συνάρτησης κόστους. Κάθε παράμετρος βάρους θ , μεταβάλλεται με βάση το γινόμενο της κλίσης (ή παραγώγου) της συνάρτησης J ως προς θ , με τον ρυθμό μάθησης. Ρυθμός μάθησης ονομάζεται η υπερ-παράμετρος η , που επιλέγεται από τον χρήστη και ορίζει τον ρυθμό μεταβολής των βαρών. Τελικά η ενημέρωση των βαρών ανάγεται στον παρακάτω τύπο:

$$\theta_j := \theta_j - \eta \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$



Σχήμα 2.2: Ο αλγόριθμος της επικλινής καθόδου.

Η παραπάνω διαδικασία εκτελείται έως ότου τα βάρη συγκλίνουν στο καθολικό ελάχιστο της συνάρτησης κόστους ή για ορισμένο αριθμό επαναλήψεων [2].

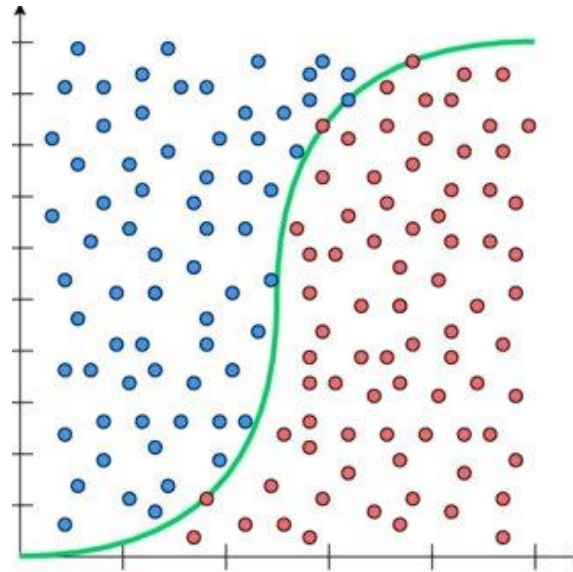
2.2.2 Λογιστική Παλινδρόμηση

Η λογιστική παλινδρόμηση (logistic regression) είναι επέκταση της γραμμικής παλινδρόμησης και χρησιμοποιείται για την επίλυση προβλημάτων δυαδικής ταξινόμησης. Η αναπαράσταση των περιπτώσεων γίνεται ως εξής:

$$h_{\theta}(x) = \sigma(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n) = \sigma(\theta \cdot x)$$

όπου $\sigma(z) = \frac{1}{1+e^{-z}}$ η σιγμοειδής συνάρτηση. Η συνάρτηση h_{θ} επιστρέφει την πιθανότητα μία περίπτωση να ανήκει στην κλάση 1, ενώ η $1 - h_{\theta}$ συμπληρωματικά, την πιθανότητα να ανήκει στην

κλάση 0. Λόγω της φύσης της σιγμοειδούς συνάρτησης, επιστρέφει πάνω από 0.5 και άρα ταξινομεί στην κλάση 1, όταν η είσοδος $\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n > 0$. Αντίστοιχα όταν η είσοδος είναι μικρότερη του μηδενός η περίπτωση ταξινομείται στην κλάση 0. Η περίπτωση μηδενικής εισόδου είναι σημείο αβεβαιότητας [2].



Σχήμα 2.3: Δυαδική Ταξινόμηση. Κλάση 1:Κόκκινο /Κλάση 0: Μπλε

Η αξιολόγηση του μοντέλου γίνεται βάσει της συνάρτησης κόστους:

$$J_{log}(\theta) = \frac{1}{m} \sum_{i=1}^m L_{log}$$

όπου $L_{log}(h_{\theta}(x), y) = -y \log(h_{\theta}(x)) - (1 - y) \log(1 - h_{\theta}(x))$. Η συνάρτηση κόστους υπολογίζει τον μέσο όρο της λογαριθμικής απώλειας, δηλαδή του αρνητικού λογαρίθμου της πιθανότητας που αποδίδεται στην κάθε περίπτωση [2].

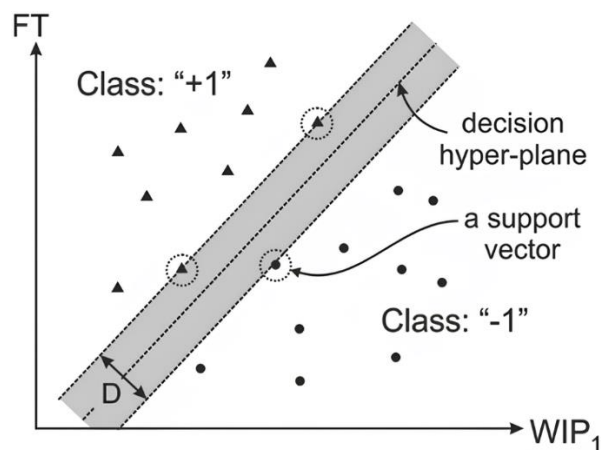
Η ελαχιστοποίηση της συνάρτησης κόστους μπορεί να επιτευχθεί και πάλι με τη μέθοδο της επικλινούς καθόδου, ενημερώνοντας τα βάρη σύμφωνα με τον παρακάτω τύπο:

$$\theta_j := \theta_j - \eta \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

2.2.3 Support Vector Machines

Βασισμένες στη θεωρία στατιστικής μάθησης, οι μηχανές διανυσμάτων υποστήριξης (Support Vector Machines, SVM) αξιοποιούνται στην επίλυση προβλημάτων γραμμικής και μη γραμμικής ταξινόμησης, αλλά και παλινδρόμησης [1]. Η κύρια λειτουργία των SVM, σε μία περίπτωση δυαδικής ταξινόμησης, είναι η δημιουργία ενός διαχωριστικού συνόρου, που να πετυχαίνει τον διαχωρισμό των παραδειγμάτων των κλάσεων. Το διαχωριστικό σύνορο καλείται υπερεπιφάνεια μεγίστου περιθωρίου (maximum margin hypersurface) και καθορίζεται από πεπερασμένο αριθμό δειγμάτων. Τα δείγματα αυτά ονομάζονται διανύσματα υποστήριξης (support vectors). Στόχος της μεθόδου είναι η μεγιστοποίηση της υπερεπιφάνειας, ώστε να απέχει την μέγιστη απόσταση από τα παραδείγματα προς διαχωρισμό. Η τεχνική αυτή επιφέρει ανεκτικότητα στο φαινόμενο της υπερπροσαρμογής [1].

Γραμμικές SVM



Σχήμα 2.4: Παράδειγμα γραμμικού SVM

Σε ένα γραμμικώς διαχωρίσιμο πρόβλημα δύο διαστάσεων, το σύνορο που αναζητείται, εξάγεται μέσω της εξίσωσης $w \cdot x + b = 0$. Συνεπώς το διάνυσμα των βαρών είναι κάθετο στο σύνορο. Σε τέτοιου τύπου προβλήματα ορίζονται δύο ακόμα σύνορα που διαχωρίζουν τις κλάσεις, στην μέγιστη απόσταση, το ένα από το άλλο. Η περιοχή ενδιάμεσα των συνόρων είναι η ζητούμενη υπερεπιφάνεια μεγίστου περιθωρίου, και η ευθεία $w \cdot x + b = 0$ την διχοτομεί. Για τυχαίο σημείο z η ταξινόμηση του έγκειται στο αποτέλεσμα της εξίσωσης συνόρου, δηλαδή:

$$y = \begin{cases} 1 & \text{αν } w \cdot z + b > 0 \\ -1 & \text{αν } w \cdot z + b < 0 \end{cases}$$

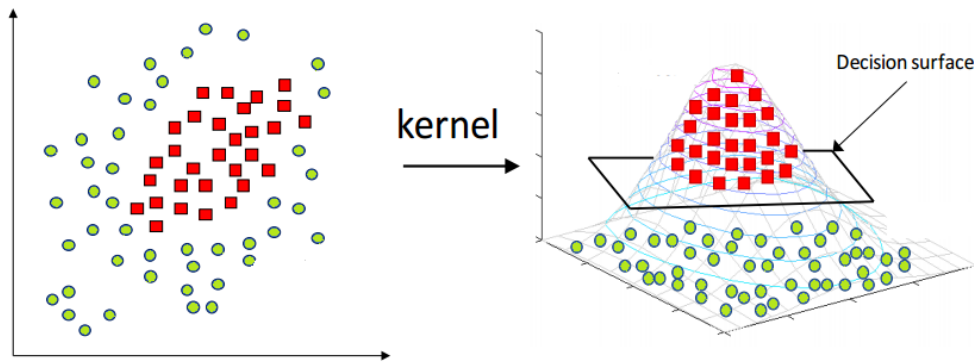
δεδομένου ότι σε δυαδικό πρόβλημα ταξινόμησης, η μία κλάση θεωρείται αρνητική και η άλλη θετική [1].

Η απόσταση μεταξύ των συνόρων υπολογίζεται από την σχέση $d = \frac{2}{||w||}$, επομένως η μεγιστοποίηση της απόστασης d σημαίνει ελαχιστοποίηση του μέτρου διανύσματος των βαρών $||w||$ ή αλλιώς: $\min_w \frac{||w||^2}{2}$. Η έκφραση αυτή είναι η μαθηματική διατύπωση της εκπαίδευσης μίας μηχανής διανυσμάτων υποστήριξης και επιλύεται από πολλαπλασιαστές Lagrange. Συνεπάγεται ότι τα w και b της εξίσωσης συνόρου εξαρτώνται αποκλειστικά από τα διανύσματα υποστήριξης, χωρίς την συμβολή των υπόλοιπων παραδειγμάτων του συνόλου εκπαίδευσης [1].

Μη γραμμικές SVM

Σε ένα πρόβλημα μη γραμμικά διαχωρίσιμων δεδομένων, μπορεί να υπάρξουν περιπτώσεις εσφαλμένης κατηγοριοποίησης δεδομένων. Ως συνέπεια, προστίθεται μια επιπλέον παράμετρος στην συνάρτηση απώλειας που καθορίζει την ανοχή σε σφάλματα [1].

Στις μη γραμμικές SVM εφαρμόζεται ένας μετασχηματισμός $\Phi(x)$, ο οποίος επιτυγχάνει την γραμμική διαχωρισιμότητα των δεδομένων στο μετασχηματισμένο χώρο. Ο υπολογισμός των w και b γίνεται με εσωτερικά γινόμενα της μορφής $\Phi(u) \cdot \Phi(v)$. Εξαιτίας της πολυπλοκότητας των υπολογισμών στις υψηλές διαστάσεις και της ασάφειας ως προς το είδος του μετασχηματισμού, χρησιμοποιείται το κόλπο του πυρήνα (kernel trick) [1].



Σχήμα 2.5: Χρήση kernel trick για γραμμικό διαχωρισμό των δεδομένων

Αποδεικνύεται πως το εσωτερικό γινόμενο $\Phi(u) \cdot \Phi(v)$ είναι δυνατόν να εκφραστεί ως συνάρτηση του εσωτερικού γινομένου των διανυσμάτων $u \cdot v$ στην αρχική διάσταση.

$$\Phi(u) \cdot \Phi(v) = f(u \cdot v)$$

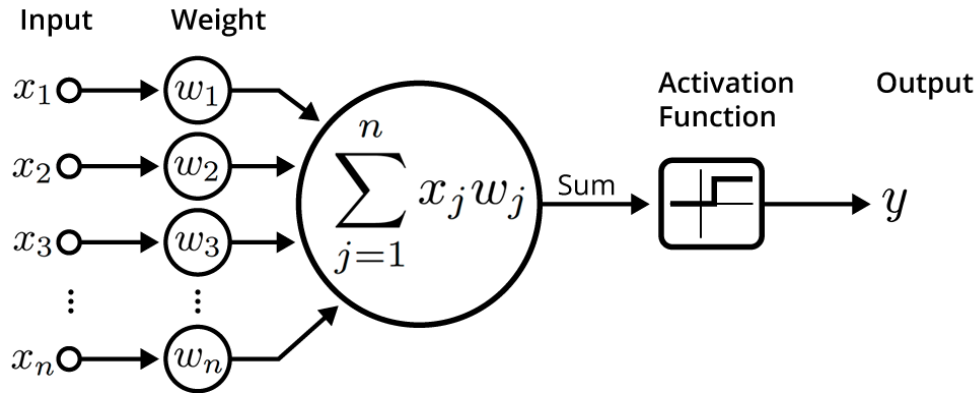
Η συνάρτηση f λέγεται συνάρτηση πυρήνα (kernel function) και συμβολίζεται με $K(u, v)$. Υπάρχουν διάφοροι τύποι συναρτήσεων πυρήνα προς επιλογή, κατάλληλες για διαφορετικού τύπου δεδομένων. Η χρήση μιας τέτοιας συνάρτησης μειώνει την πολυπλοκότητα των υπολογισμών, αφού αυτοί γίνονται στις αρχικές, χαμηλότερες διαστάσεις και εξαλείφει τις ασάφειες στον προσδιορισμό της συνάρτησης Φ [1].

Οι SVM επιτυγχάνουν υψηλές αποδόσεις σε προβλήματα ταξινόμησης τόσο σε γραμμικά, όσο και σε μη γραμμικά σύνολα δεδομένων. Πετυχαίνουν μεγαλύτερη ακρίβεια από απλά νευρωνικά δίκτυα όπως Perceptron λόγω του στόχου του ολικού ελαχίστου, έναντι της σύγκλισης σε τοπικό ελάχιστο. Τέλος παρουσιάζουν ανθεκτικότητα στο φαινόμενο της υπερπροσαρμογής, με μικρές υπολογιστικές απαιτήσεις, παρόλα αυτά, είναι δύσκολο να εφαρμοστούν σε πολυδιάστατα δεδομένα [1].

2.2.4 Τεχνητά Νευρωνικά Δίκτυα

Τα Τεχνητά Νευρωνικά Δίκτυα (Neural Networks) είναι μια υπολογιστική μέθοδος, που πηγάει από τους βιολογικούς νευρώνες του ανθρώπινου εγκεφάλου, και επιλύει αποτελεσματικά πολύπλοκα προβλήματα ταξινόμησης και παλινδρόμησης [1]. Ένα δίκτυο αποτελείται από νευρώνες, διατεταγμένους σε επίπεδα (layers). Κάθε νευρώνας δέχεται πολλαπλές μεταβλητές εισόδου $x_1, x_2, \dots, x_n$, οι οποίες πολλαπλασιάζονται με αντίστοιχα βάρη $w_1, w_2, \dots, w_n$ και μία σταθερά $x_0 = 1$, με βάρος w_0 , που ονομάζεται πόλωση (bias) [1].

Η δομή του τεχνητού νευρώνα διαιρείται σε δύο διακριτά μέρη, τον αθροιστή που συναθροίζει τις πολλαπλασιασμένες από τα βάρη, εισόδους σε μια μεταβλητή S και την συνάρτηση ενεργοποίησης που δέχεται την S και παράγει την έξοδο του νευρώνα. Η έξοδος ενός νευρώνα αποτελεί είσοδο για νευρώνες του επόμενου επιπέδου, μέχρι το τελευταίο επίπεδο, όπου παράγεται η τελική έξοδος του νευρωνικού δικτύου. Στην μάθηση με επίβλεψη, η έξοδος αντιπαραβάλλεται με την πραγματική έξοδο του συνόλου δεδομένων και μεταβάλλονται τα βάρη των νευρώνων αναλογικά με το σφάλμα, ώστε αυτό να ελαχιστοποιηθεί. Αυτός ο αλγόριθμος λέγεται οπισθοδρόμηση σφάλματος (back propagation) και συγκλίνει μετά από ορισμένο αριθμό επαναλήψεων, έως ότου επιτευχθεί ικανοποιητική ακρίβεια [1].



Σχήμα 2.6: Δομή ενός τεχνητού νευρώνα Perceptron

Η συνάρτηση ενεργοποίησης εισάγει μη γραμμικότητα σε ένα νευρωνικό δίκτυο, δίνοντας την δυνατότητα εκμάθησης πολύπλοκων σχέσεων στα δεδομένα εκπαίδευσης και ως εκ τούτου, την ικανότητα επίλυσης προβλημάτων με μη γραμμικώς διαχωρίσιμα δεδομένα. Δύο ιδιαίτερα δημοφιλείς συναρτήσεις ενεργοποίησης είναι η ReLU και η softmax [1]. Η συνάρτηση ReLU χρησιμοποιείται εκτενώς στα βαθιά νευρωνικά δίκτυα και ορίζεται ως εξής:

$$f(S) = \begin{cases} S & \text{για } S > 0 \\ 0 & \text{για } S \leq 0 \end{cases}$$

Η softmax χρησιμοποιείται για απόδοση πιθανότητας στους νευρώνες του τελευταίου επιπέδου του δικτύου, σε προβλήματα ταξινόμησης. Ορίζεται ως:

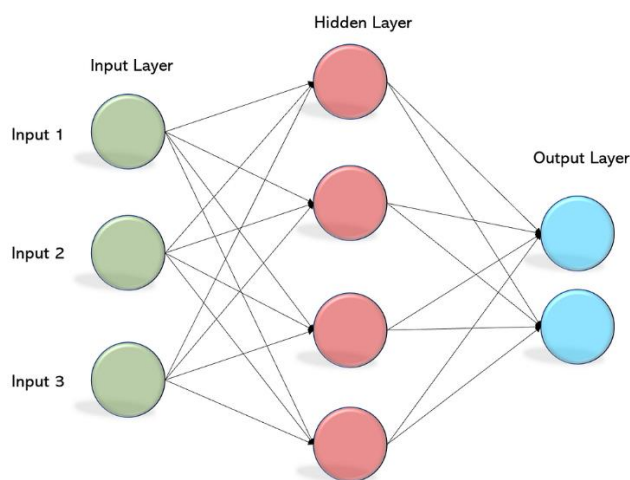
$$\varphi_i = \frac{e^{z_i}}{\sum_{j=1}^k e^{z_j}}$$

Η επιλογή της κατάλληλης συνάρτησης ενεργοποίησης είναι κομβικής σημασίας για την αποφυγή του φαινομένου της εξασθένισης παραγώγου (vanishing gradient) [1].

Multi-Layer Perceptron (MLP)

Το Perceptron συνιστά το απλούστερο είδος νευρωνικού δικτύου, με έναν μόνο νευρώνα, και αξιοποιείται στην επίλυση γραμμικά διαχωρίσιμων προβλημάτων. Για την επίλυση μη γραμμικά διαχωρίσιμων προβλημάτων πρέπει να κατασκευαστούν δίκτυα πολλαπλών επιπέδων (Multi-Layer Perceptron, MLP) [1].

Τα MLP είναι νευρωνικά δίκτυα πρόσθιας τροφοδότησης (feedforward), των οποίων οι νευρώνες είναι διατεταγμένοι σε επίπεδα. Υφίστανται τρεις κατηγορίες επιπέδων, το επίπεδο εισόδου, τα κρυφά επίπεδα (hidden layers) και το επίπεδο εξόδου. Το επίπεδο εισόδου δεν απαρτίζεται από νευρώνες, αλλά από στοιχεία για την εισαγωγή των δεδομένων. Ένα επίπεδο καλείται πλήρως συνδεδεμένο (fully connected) όταν η έξοδος κάθε νευρώνα του επιπέδου είναι είσοδος για κάθε νευρώνα του επόμενου επιπέδου [1].



Σχήμα 2.7: Δομή πλήρως συνδεδεμένου Multilayer Perceptron

Τα δεδομένα εκπαίδευσης μεταφέρονται στο επίπεδο εισόδου και υπολογίζεται το επίπεδο εξόδου μέσα από την διάσχιση των κρυφών επιπέδων. Με βάση το σφάλμα που προκύπτει αναπροσαρμόζονται τα βάρη και επαναλαμβάνεται η διαδικασία για ορισμένο πλήθος επαναλήψεων ή εποχών (epochs) [1].

2.2.5 Συνελικτικά Νευρωνικά Δίκτυα

Τα συνελικτικά νευρωνικά δίκτυα (Convolutional Neural Networks, CNN) είναι κατηγορία βαθιών νευρωνικών δικτύων, κατάλληλα για προβλήματα ταξινόμησης σε δεδομένα εικόνας, καθώς και άλλων τύπων πολυδιάστατων δεδομένων. Οι κατηγορίες επιπέδων σε ένα συνελικτικό δίκτυο είναι τρεις: τα επίπεδα συνέλιξης, το επίπεδο συγκέντρωσης χαρακτηριστικών και το πλήρως συνδεδεμένο [1].

Στόχος του επιπέδου συνέλιξης είναι η αναγνώριση προτύπων στις εικόνες. Χρησιμοποιούνται φίλτρα, τα οποία εφαρμόζονται στον πίνακα δεδομένων μέσω της διαδικασίας του πολλαπλασιασμού πινάκων. Τα φίλτρα αυτά διατρέχουν ολόκληρο τον πίνακα, με την μορφή κυλιόμενου παραθύρου και παράγουν τον πίνακα συνέλιξης, με διαστάσεις μικρότερες του αρχικού. Για την αναγνώριση συγκεκριμένων χαρακτηριστικών χρησιμοποιούνται διαφορετικά φίλτρα, οι παραγόμενοι πίνακες των οποίων, προστίθενται σε έναν πίνακα συνέλιξης [1].

Το επίπεδο συγκέντρωσης (pooling) μειώνει τις διαστάσεις του πίνακα συνέλιξης συγκρατώντας την επικρατέστερη πληροφορία. Υπάρχουν δύο τεχνικές, το max pooling και το average pooling. Στην περίπτωση του πρώτου ένα φίλτρο 2x2 διατρέχει τον πίνακα συνέλιξης, με την μορφή και πάλι του κυλιόμενου παραθύρου, και αποθηκεύει την μέγιστη τιμή από κάθε υπο-πίνακα 2x2 του πίνακα συνέλιξης, σε ένα καινούριο πίνακα. Το average pooling είναι αντίστοιχο αλλά αποθηκεύεται η μέση τιμή κάθε φορά. Το επίπεδο αυτό αντιμετωπίζει την υπερπροσαρμογή και επιταχύνει τους υπολογισμούς [1]. Τέλος, το τελευταίο επίπεδο αποτελείται από ένα πλήρως συνδεδεμένο επίπεδο (γνωστό και ως MLP), που περιλαμβάνει και το επίπεδο εξόδου [1].

2.2.6 Αξιολόγηση Μοντέλων

Τα μοντέλα μηχανικής μάθησης υπόκεινται σε κάποια διαδικασία αξιολόγησης για την εξαγωγή συμπερασμάτων. Η διαδικασία αυτή μπορεί να αφορά στην επιλογή του μοντέλου, με τις κατάλληλες υπερ-παραμέτρους, ή στην εκτίμηση του μοντέλου σε δεδομένα, εκτός του συνόλου εκπαίδευσης

[2]. Ο απλούστερος τρόπος για την αξιολόγηση ενός μοντέλου, είναι η μέτρηση της ακρίβειας που επιτυγχάνει στο σύνολο εκπαίδευσης.

Το σφάλμα που προκύπτει από την αξιολόγηση ονομάζεται σφάλμα επαναντικατάστασης (resubstitution error), και οδηγεί στην επιλογή μοντέλων που υπερπροσαρμόζονται στα δεδομένα εκπαίδευσης. Για την εξάλειψη αυτού του φαινομένου, διασπάται το σύνολο δεδομένων σε σύνολο εκπαίδευσης (training set), σύνολο ελέγχου (test set) και σε συγκεκριμένες περιπτώσεις σε σύνολο επικύρωσης (validation set). Συνεπώς, τα μοντέλα εκπαιδεύονται χρησιμοποιώντας το σύνολο εκπαίδευσης, υπολογίζεται η ακρίβεια του καθενός με βάση το σύνολο επικύρωσης και επιλέγεται το καλύτερο και τέλος το επιλεγμένο μοντέλο αξιολογείται στο σύνολο ελέγχου. Οι πρωταρχικές μέθοδοι αξιολόγησης είναι η παρακράτηση και η σταυρωτή επικύρωση [2].

Η παρακράτηση (holdout) είναι η τεχνική διάσπασης του αρχικού συνόλου δεδομένων, στα υποσύνολα που αναφέρθηκαν παραπάνω. Για την επίτευξη μεγαλύτερης αξιοπιστίας στις εκτιμήσεις υπάρχουν δύο παραλλαγές, η διαστρωματωμένη παρακράτηση (stratified holdout) και η επαναλαμβανόμενη (repeated holdout). Η πρώτη διασφαλίζει την ισοποση αναπαράσταση των κλάσεων σε κάθε υποσύνολο, ενώ στην δεύτερη επαναλαμβάνεται, με τυχαίο τρόπο, η διαδικασία διάσπασης πολλές φορές και υπολογίζεται ο μέσος όρος σφάλματος. Ελλοχεύει ο κίνδυνος επικάλυψης των δεδομένων, φαινόμενο που αντιμετωπίζεται από την σταυρωτή επικύρωση [2].

Η μέθοδος της σταυρωτής επικύρωσης (cross-validation) διαχωρίζει το σύνολο δεδομένων σε k ισομεγέθη διακριτά υποσύνολα. Ένα από αυτά επιλέγεται ως το σύνολο ελέγχου, ενώ τα υπόλοιπα $k-1$ χρησιμοποιούνται για την εκπαίδευση του μοντέλου. Η διαδικασία εκτελείται k φορές, ώστε κάθε υποσύνολο να έχει χρησιμοποιηθεί και ως δεδομένο εκπαίδευσης και ως δεδομένο ελέγχου, και εξάγεται ο μέσος όρος των k αποτελεσμάτων. Σε περιπτώσεις μικρού συνόλου εκπαίδευσης, η τεχνική σταυρωτή επικύρωση άφησε ένα εκτός (leave-one-out cross-validation) αξιοποιεί το μεγαλύτερο ποσοστό δεδομένων εκπαίδευσης. Συγκεκριμένα, το σύνολο εκπαίδευσης χωρίζεται σε m υποσύνολα, όσα και τα δεδομένα εκπαίδευσης, ένα από αυτά αποτελεί το σύνολο ελέγχου και τα υπόλοιπα $m-1$ το σύνολο εκπαίδευσης [2].

Μετρικές Αξιολόγησης Παλινδρόμησης

Υπάρχει πληθώρα μετρικών αξιολόγησης σε προβλήματα παλινδρόμησης. Δύο κύριες είναι το μέσο απόλυτο σφάλμα (mean absolute error – MAE) και η τετραγωνική ρίζα του μέσου τετραγωνικού σφάλματος (root mean squared error – RMSE)[2], οι οποίες ορίζονται με τους παρακάτω τύπους:

$$MAE(h, D) = \frac{1}{m} \sum_{i=1}^m |h(x^{(i)}) - y^{(i)}|$$
$$RMSE(h, D) = \sqrt{\frac{1}{m} \sum_{i=1}^m (h(x^{(i)}) - y^{(i)})^2}$$

Μετρικές Αξιολόγησης Ταξινόμησης

Σε ένα πρόβλημα δυαδικής ταξινόμησης, μία κλάση χαρακτηρίζεται ως θετική και η άλλη ως αρνητική. Η επίδοση ενός μοντέλου αναπαρίσταται σε έναν πίνακα σύγχυσης (confusion matrix) [2].

Πίνακας 2.1: Πίνακας σύγχυσης για πρόβλημα δυαδικής ταξινόμησης

Πρόβλεψη\Πραγματική	Θετική	Αρνητική
Θετική	TP	FP
Αρνητική	FN	TN

Οι τιμές των κελιών του πίνακα 2.1 δηλώνουν τον αριθμό των σωστών και λανθασμένων προβλέψεων, σε σχέση με την πραγματική κλάση. Σύμφωνα με τις παραπάνω τιμές ορίζονται διάφορες μετρικές αξιολόγησης για την θετική κλάση, όπως η ακρίβεια (precision) και η ανάκληση (recall). Υπάρχουν αντίστοιχες μετρικές και για την αρνητική κλάση [2].

Η ακρίβεια υπολογίζει τον αριθμό των στιγμιότυπων που ταξινομήθηκαν σωστά ως θετικά προς όλα όσα ταξινομήθηκαν ως θετικά.

$$P = \frac{TP}{TP + FP}$$

Παράλληλα, η ανάκληση ή αλλιώς ρυθμός αληθώς θετικών (true positive rate) υπολογίζει τον αριθμό των στιγμιότυπων που ταξινομήθηκαν σωστά ως θετικά προς όλα όσα είναι πραγματικά θετικά [2].

$$TPR = \frac{TP}{TP + FN}$$

Οι δύο αυτές μετρικές συγχωνεύονται στην μετρική F_1 που μετρά τον αρμονικό μέσο τους.

$$F_1 = \frac{2 \cdot P \cdot TPR}{P + TPR}$$

Ο δείκτης 1 συμβολίζει την κρισιμότητα της ανάκλησης σε σχέση με την ακρίβεια, και μπορεί να τροποποιηθεί για να δώσει περισσότερο βάρος σε κάποια από τις δύο [2].

2.3 Μάθηση χωρίς Επίβλεψη

Στην μάθηση χωρίς επίβλεψη λαμβάνονται αποκλειστικά δεδομένα εισόδου και σκοπός είναι η εξεύρεση συσχετίσεων και ομαδοποιήσεων μεταξύ τους. Ως εκ τούτου εξάγονται πρότυπα πληροφόρησης όπως είναι οι κανόνες συσχέτισης (association rules) μεταξύ των δεδομένων, και οι παραγόμενες ομάδες (clusters) της μεθόδου της ομαδοποίησης (clustering), οι οποίες ομαδοποιούν παρόμοια παραδείγματα [1].

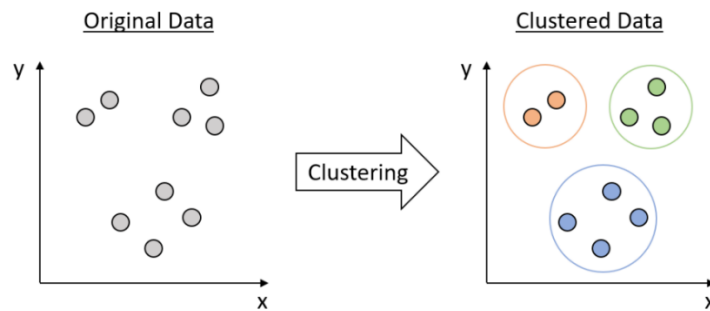
2.3.1 Κανόνες Συσχέτισης

Οι κανόνες συσχέτισης είναι προτάσεις με μορφή $\{X_1, \dots, X_n\} \rightarrow Y$. Η ερμηνεία μιας τέτοιας πρότασης είναι ότι αν έχουμε δεδομένο το σύνολο X , τότε είναι πιθανό να ισχύει και το Y . Η τεχνική αυτή βρίσκει εφαρμογές στην ανάλυση καλαθιού αγορών και γενικότερα, δεδομένων συναλλαγών, με παράδειγμα την σύσταση προϊόντων σε πελάτες [1]. Η αξιολόγηση των κανόνων συσχέτισης γίνεται μέσα από αριθμητικούς δείκτες όπως, η υποστήριξη (support), η εμπιστοσύνη (confidence) και η άντωση (lift). Η υποστήριξη δηλώνει την πιθανότητα να ισχύει ένα σύνολο $\{X_1, \dots, X_n\}$ και η εμπιστοσύνη δηλώνει την πιθανότητα να ισχύει το Y όταν είναι δεδομένο το σύνολο $\{X_1, \dots, X_n\}$. Παράλληλα, η άντωση εκφράζει τον βαθμό συσχέτισης δύο αντικειμένων, συγκρίνοντας την πιθανότητα να εμφανίζονται μαζί, με αυτήν που εμφανίζονται ανεξάρτητα [1].

Ορισμένα πλεονεκτήματα των κανόνων συσχέτισης είναι η επεξηγηματικότητα τους και η διαχείριση μεγάλου όγκου δεδομένων. Το κύριο μειονέκτημα είναι το πρόβλημα της επιλογής των επικρατέστερων κανόνων μέσα από το μεγάλο πλήθος κανόνων που παράγονται [1].

2.3.2 Ομάδες

Οι ομάδες αποτελούν πρότυπα πληροφόρησης που παράγονται από την διαδικασία της ομαδοποίησης (clustering). Η διαδικασία αυτή διαιρεί ένα σύνολο δεδομένων σε ομάδες με βάση την ομοιότητα των χαρακτηριστικών τους [1].



Σχήμα 2.8: Εφαρμογή της διαδικασίας ομαδοποίησης

Οι αλγόριθμοι που υλοποιούν την ομαδοποίηση διακρίνονται σε τρία είδη, τους αλγορίθμους βασισμένους σε διαχωρισμούς (partition based), τους ιεραρχικούς (hierarchical) και τους πιθανοκρατικούς [1].

Ο πιο γνωστός αλγόριθμος ομαδοποίησης, βασισμένης σε διαχωρισμούς, είναι ο αλγόριθμος των K-μέσων (K-means).

1. Ο αλγόριθμος ξεκινά με την επιλογή K τυχαίων στιγμιότυπων, τα οποία αναπαριστούν τα κέντρα των ομάδων.
2. Τα υπόλοιπα δεδομένα ανατίθενται στην ομάδα με το πιο κοντινό κέντρο, με βάση κάποια μετρική όπως η ευκλείδεια απόσταση.
3. Υπολογίζεται για κάθε ομάδα το μέσο διάνυσμα και ορίζεται ως νέο κέντρο της.

Τα βήματα 2 και 3 επαναλαμβάνονται έως ότου τα κέντρα παραμείνουν αμετάβλητα μετά το βήμα 3 [1].

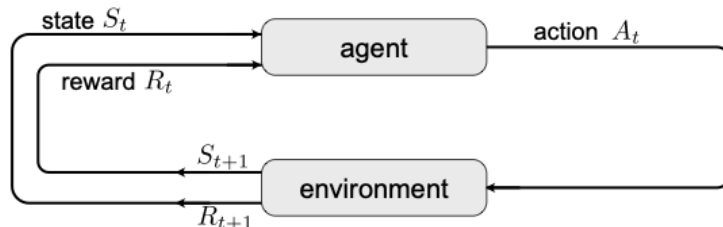
Ο αλγόριθμος K-μέσων είναι αποτελεσματικός εξαιτίας της απλότητας και της ταχύτητας του. Ωστόσο, ο προσδιορισμός της μεταβλητής K είναι ένα μη ντετερμινιστικό πρόβλημα και η μη κατάλληλη αρχικοποίηση της, επηρεάζει αρνητικά την επίδοση του αλγορίθμου [1].

2.4 Ενισχυτική Μάθηση

Η ενισχυτική μάθηση είναι μια τεχνική εμπνευσμένη από τον τρόπο με την οποία εκπαιδεύονται οι βιολογικοί οργανισμοί. Στηρίζεται στην μάθηση μιας συμπεριφοράς ενός πράκτορα, ο οποίος εκπαιδεύεται στο να εκτελεί τις βέλτιστες ενέργειες, με βάση την κατάσταση που βρίσκεται σε κάποιο περιβάλλον. Ενδείκνυται ιδιαίτερα σε πολύπλοκα προβλήματα, με ανεπαρκή και δυσεύρετα δεδομένα εκπαίδευσης, όπως τα αυτοκινούμενα οχήματα [2].

2.4.1 Μαρκοβιανές Διαδικασίες Απόφασης

Ο πράκτορας που εκπαιδεύεται, σε κάθε χρονική στιγμή t , βρίσκεται σε μια κατάσταση S_t . Λαμβάνοντας υπόψιν την τρέχουσα κατάσταση επιλέγει μία ενέργεια A_t , λαμβάνει κάποια ανταμοιβή R_{t+1} και μεταφέρεται στην κατάσταση S_{t+1} . Επομένως μία μαρκοβιανή διαδικασία απόφασης (ΜΔΑ), μοντελοποιεί μαθηματικά τα προβλήματα ενισχυτικής μάθησης με μία σχέση $p(s', r|s, a)$ [2].



Σχήμα 2.9: Αλληλεπίδραση πράκτορα με το περιβάλλον.

Πολιτική (policy) $\pi(s)$, ενός πράκτορα αποτελεί η συνάρτηση που προσδιορίζει τις ενέργειες του σε κάθε χρονική στιγμή. Με βάση την πολιτική του πράκτορα, οι συναρτήσεις αξίας (value function) εκτιμούν την αξία είτε της τρέχουσας κατάστασης (v_π), είτε της επιλογής ενέργειας, δεδομένης της κατάστασης (q_π). Η εκτίμηση σχετίζεται με το αναμενόμενο κέρδος των ενεργειών που θα εκτελεστούν. Στόχος του πράκτορα είναι η βελτιστοποίηση της πολιτικής του μέσα από την μεγιστοποίηση των αναμενόμενων αθροιστικών ανταμοιβών [2].

2.4.2 Δυναμικός Προγραμματισμός

Ο δυναμικός προγραμματισμός συνιστά ένα σύνολο αλγορίθμων με σκοπό την εύρεση της βέλτιστης πολιτικής σε μία πεπερασμένη μαρκοβιανή διαδικασία απόφασης (ΠΜΔΑ). Η αναζήτηση της πολιτικής συνδέεται άρρηκτα με την αναζήτηση των βέλτιστων συναρτήσεων αξίας [2]. Δύο βασικοί αλγόριθμοι δυναμικού προγραμματισμού είναι η επανάληψη πολιτικής (policy iteration) και η επανάληψη αξίας (value iteration). Ο πρώτος ξεκινά από τυχαία πολιτική π , εκτελεί αξιολόγηση πολιτικής (policy evaluation), υπολογίζοντας v_π και q_π , και βελτίωση πολιτικής (policy improvement). Λόγω του μεγάλου πλήθους βημάτων του αλγορίθμου αυτού, μπορεί να χρησιμοποιηθεί ο αλγόριθμος της επανάληψης αξίας, ο οποίος συγχωνεύει την αξιολόγηση και βελτίωση πολιτικής σε ένα μόνο βήμα [2].

Οι αλγόριθμοι δυναμικού προγραμματισμού είναι ανεπαρκείς σε προβλήματα με μεγάλο χώρο καταστάσεων, λόγω μεγάλου υπολογιστικού κόστους [2].

2.4.3 Μέθοδοι Χρονικών Διαφορών

Οι μέθοδοι χρονικών διαφορών (ΧΔ) είναι κομβικής σημασίας στην ενισχυτική μάθηση. Η ενημέρωση των εκτιμήσεων v_π και q_π , γίνεται μέσω εκτιμήσεων αξιών διαφορετικών αξιών κατάστασης και ζεύγους κατάστασης και ενέργειας. Επομένως δεν απαιτείται μοντέλο του περιβάλλοντος αλλά εμπειρία, γεγονός που επιτυγχάνει την εκμάθηση του πράκτορα εντός κατά την διάρκεια του επεισοδίου [2].

Για κάθε βήμα η ενημέρωση των $V(s_t)$, $Q(s_t, a_t)$ δίνεται από τους παρακάτω τύπους:

$$V(s_t) \leftarrow V(s_t) + a[R_{t+1} + \gamma V(s_{t+1}) - V(s_t)]$$
$$Q(s_t, A_t) \leftarrow Q(s_t, A_t) + a[R_{t+1} + \gamma Q(s_{t+1}, A_{t+1}) - Q(s_t, A_t)]$$

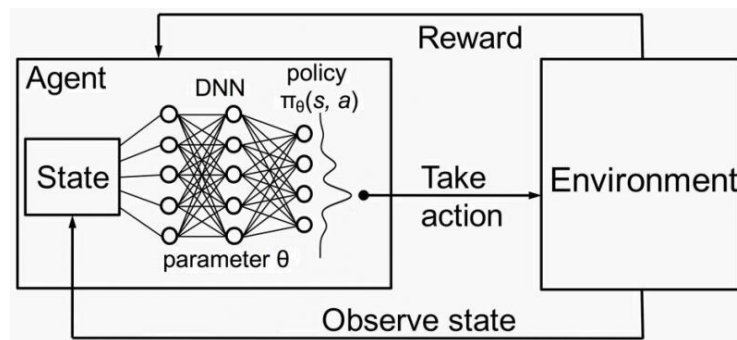
Μια διάκριση των αλγορίθμων της ενισχυτικής μάθησης είναι σε εντός πολιτικής (on policy) και εκτός πολιτικής (off policy). Οι αλγόριθμοι εντός πολιτικής ακολουθούν κάποια πολιτική και προσπαθούν να βελτιώσουν την ίδια. Αντίθετα οι εκτός πολιτικής ακολουθούν διαφορετική πολιτική από αυτή που προσπαθούν να βελτιώσουν, γεγονός που συμβάλλει σε περισσότερο άπληστη προσέγγιση για την βελτιστοποίηση της πολιτικής [2].

Ένας πολύ σημαντικός αλγόριθμος χρονικών διαφορών εκτός πολιτικής είναι ο Q-Learning. Η σύγκλιση του στην βέλτιστη πολιτική γίνεται αφού έχει επισκεφτεί κάθε ζεύγος κατάστασης και ενέργειας για άπειρες περιπτώσεις. Η ενημέρωση των αξιών πραγματοποιείται με τον ίδιο τρόπο, με την διαφορά να είναι η επιλογή της μέγιστης αξίας ως την επόμενη κατάσταση S' έναντι της αξίας $q_\pi(S_{t+1}, A_{t+1})$, που αποτελεί την προσέγγιση εντός πολιτικής [2]. Δηλαδή με την σχέση:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma \max_A Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

2.4.4 Βαθιά Ενισχυτική Μάθηση

Τα μοντέλα βαθιάς ενισχυτικής μάθησης (deep reinforcement learning, DRL) αξιοποιούν βαθιά νευρωνικά δίκτυα, με πολλαπλά επίπεδα νευρώνων για την προσέγγιση των συναρτήσεων αξίας $v_\pi(S; \theta)$ και $q_\pi(S, A; \theta)$ και της πολιτικής $\pi(A|S; \theta)$. Η παράμετρος θ αποτελεί τις τιμές των βαρών του νευρωνικού δικτύου. Η επιλεγόμενη συνάρτηση βελτιστοποίησης είναι η στοχαστική επικλινής κάθοδος [4].



Σχήμα 2.10: Αλληλεπίδραση πράκτορα βαθιάς ενισχυτικής μάθησης, με το περιβάλλον.

2.4.5 Deep Q-Network

Ο αλγόριθμος Deep Q-Network (DQN) ανήκει σε μία οικογένεια αλγορίθμων βασισμένων στις αξίες (value based), οι οποίοι προσεγγίζουν την τιμή της συνάρτησης αξίας Q , με την χρήση βαθιών νευρωνικών δικτύων [3]. Συγκεκριμένα, ο DQN αξιοποιεί δύο βαθιά νευρωνικά δίκτυα με ίδια δομή αλλά διαφορετικές παραμέτρους, το Q-Network και το Target Network. Οι παράμετροι του πρώτου ενημερώνονται μετά από κάθε βήμα, ενώ του δεύτερου μετά από N βήματα [3].

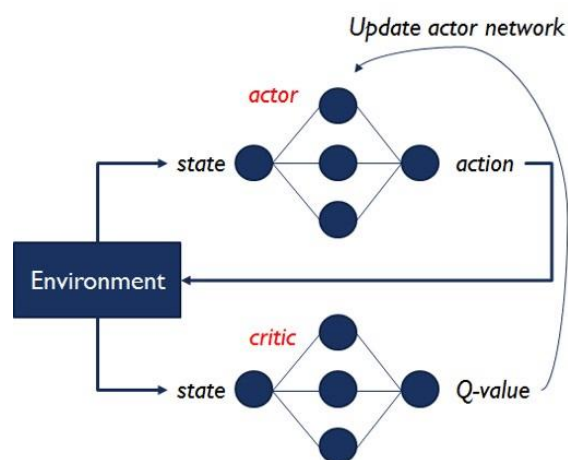
Η συλλογή των δεδομένων εκπαίδευσης γίνεται με την διάσχιση του Q-Network. Στην μνήμη, που ονομάζεται experience replay memory, αποθηκεύονται τετράδες (s_t, a_t, r_t, s_{t+1}) όσο ο πράκτορας αλληλεπιδρά με το περιβάλλον. Οι καταστάσεις s_{t+1} λαμβάνονται ως είσοδοι από το Target Network και με βάση την απώλεια που προκύπτει από τα δύο νευρωνικά, ενημερώνονται ανάλογα οι παράμετροι τους [3].

2.4.6 Μέθοδοι Policy Gradient

Οι αλγόριθμοι Policy Gradient έχουν ως στόχο την μεγιστοποίηση της αναμενόμενης συνολικής ανταμοιβής, σε αντίθεση με τους value based, που το επιτυγχάνουν έμμεσα με την μεγιστοποίηση του Q. Είναι ιδανικοί για μεγάλα πολύπλοκα προβλήματα με συνεχή χώρο καταστάσεων, όμως είναι περισσότερο ασταθείς και επιρρεπείς σε τοπικά μέγιστα. Δύο βασικοί αλγόριθμοι Policy Gradient είναι ο Actor Critic και ο Proximal Policy Optimization (PPO) [3].

Actor Critic

Ο αλγόριθμος Actor Critic χρησιμοποιεί δύο νευρωνικά δίκτυα. Το πρώτο ονομάζεται Actor και λαμβάνει την τρέχουσα κατάσταση του περιβάλλοντος για να υπολογίσει την βέλτιστη ενέργεια. Η ενέργεια που προκύπτει, μαζί με την τρέχουσα κατάσταση, αποτελούν είσοδο για το δεύτερο δίκτυο που ονομάζεται Critic. Το δίκτυο αυτό αξιολογεί την ενέργεια που επιλέχθηκε και επιστρέφει την αξία του Q, ώστε το δίκτυο Actor να βελτιώσει την πολιτική του [3].



Σχήμα 2.11: Δομή του Actor Critic αλγόριθμου.

Τα βασικά του μειονεκτήματα είναι η αδυναμία του να αποφύγει τοπικά μέγιστα, καθώς και η χαμηλή του επίδοση σε δυναμικά περιβάλλοντα με άγνωστα δεδομένα. Τα προβλήματα αυτά αντιμετωπίζονται με τον βελτιωμένο αλγόριθμο Soft Actor Critic. Στόχος του αλγορίθμου είναι και πάλι η μεγιστοποίηση του κέρδους, αλλά παράλληλα και η μεγιστοποίηση της εντροπίας. Η στοχαστικότητα που προσδίδει η εντροπία κάνει την πολιτική ιδιαίτερα ανθεκτική σε άγνωστα δεδομένα [3].

Proximal Policy Optimization (PPO)

Ο αλγόριθμος PPO έχει πολλές ομοιότητες με τον Actor Critic. Αξιοποιούνται δύο νευρωνικά δίκτυα, το Policy Network και το Value Function Network. Όπως στον Actor Critic το πρώτο υπολογίζει την βέλτιστη ενέργεια, ενώ το δεύτερο παράγει την αξία Q, της ενέργειας βάση της κατάστασης [3]. Η διαφορά των δύο αλγορίθμων έγκειται στον τρόπο ενημέρωσης των βαρών των δικτύων, το Value Function ενημερώνεται βάσει του advantage: $A(s, a) = Q(s, a) - V(s)$, ενώ στην ενημέρωση του Policy υφίσταται “κλάδεμα” (clipping) και γίνεται σύμφωνα με τον παρακάτω τύπο:

$$L(\theta) = \hat{E}_t \left[\min \left(\frac{\pi_{\theta}(a|s)}{\pi_{\theta_{old}}(a|s)} A_t, \text{clip} \left(\frac{\pi_{\theta}(a|s)}{\pi_{\theta_{old}}(a|s)}, 1 - \epsilon, 1 + \epsilon \right) A_t \right) \right]$$

Η τεχνική του clipping μειώνει το μέγεθος των αλλαγών στα βάρη του δικτύου και το καθιστά πιο σταθερό και αξιόπιστο [3].

2.4.7 Μέθοδοι Model-Based

Η Model-Based ενισχυτική μάθηση αναφέρεται σε αλγορίθμους που μαθαίνουν ένα μοντέλο του περιβάλλοντος με σκοπό τον σχεδιασμό ενεργειών. Το μοντέλο λαμβάνει ως είσοδο μια κατάσταση και μία ενέργεια και παράγει μια πρόβλεψη για την επόμενη κατάσταση και ανταμοιβή. Όταν το μοντέλο εκπαιδευτεί, ο πράκτορας το χρησιμοποιεί για να προσομοιώσει την αλληλεπίδραση με το περιβάλλον και να σχεδιάσει τις μελλοντικές του ενέργειες στο αληθινό περιβάλλον [5].

Οι μέθοδοι Model-Based κάνουν αποδοτική χρήση των δειγμάτων, καθώς χρησιμοποιούν το προσομοιωμένο περιβάλλον και συνεπώς μπορούν να εκπαιδευτούν με λιγότερα δεδομένα. Ωστόσο ανακρίβειες στο μοντέλο που εκπαιδεύεται, μπορεί να οδηγήσουν σε λανθασμένες προβλέψεις και αναποτελεσματικό σχεδιασμό ενεργειών [5]. Παραδείγματα των αλγορίθμων αυτής της κατηγορίας είναι οι Dreamer-v3 και AlphaZero.

Κεφάλαιο 3

Προηγμένοι Αλγόριθμοι Βαθιάς Ενισχυτικής Μάθησης

Την τελευταία δεκαετία ο τομέας της ενισχυτικής μάθησης έχει γνωρίσει τεράστια ανάπτυξη, με την υλοποίηση καινοτόμων αλγορίθμων που επιλύουν πολύπλοκα προβλήματα. Στόχος του κεφαλαίου αυτού είναι η ανάλυση ορισμένων προηγμένων αλγορίθμων βαθιάς ενισχυτικής μάθησης, σε θεωρητικό επίπεδο, προκειμένου να γίνει κατανοητός ο τρόπος λειτουργίας τους και οι περιπτώσεις εφαρμογής τους. Οι αλγόριθμοι που θα αναλυθούν, κατηγοριοποιούνται σε Value-Based, Policy Gradient, Model-Based και σε αυτούς που επικεντρώνονται στην εξερεύνηση, Exploration Module.

3.1 Value-Based

Οι αλγόριθμοι ενισχυτικής μάθησης που βασίζονται στη συνάρτηση αξίας (Value-Based) στοχεύουν στην εκτίμηση της συνάρτησης $Q(s, a)$, η οποία αποτιμά την αναμενόμενη συνολική ανταμοιβή του πράκτορα, μετά την λήψη μιας ενέργειας στην δεδομένη κατάσταση. Οι προηγμένοι αλγόριθμοι αυτής της κατηγορίας αξιοποιούν τεχνικές όπως αναδρομικά δίκτυα (RNN) και prioritized experience replay αλλά και μεθόδους κατανεμημένης εκπαίδευσης, ώστε να αντιμετωπίσουν αποδοτικά σύνθετα και δυναμικά περιβάλλοντα.

3.1.1 R2D2

Ο αλγόριθμος Recurrent Replay Distributed DQN (R2D2) είναι ένας αλγόριθμος ενισχυτικής μάθησης, ο οποίος συνδυάζει τεχνικές αναδρομικών νευρωνικών δικτύων (Recurrent Neural Networks, RNNs), experience replay memory και κατανεμημένης εκπαίδευσης (distributed training) [6]. Ο R2D2 είναι βασισμένος στο prioritized distributed replay και το n-step double Q-learning. Αναφορικά με το πρώτο, από τα δεδομένα που αποθηκεύονται στην replay memory, μέσω της αλληλεπίδρασης του πράκτορα με το περιβάλλον, οι περιπτώσεις με το μεγαλύτερο σφάλμα, λαμβάνουν την μεγαλύτερη προτεραιότητα στην εκπαίδευση του πράκτορα. Επιπρόσθετα, η συλλογή των δεδομένων κατανέμεται σε πολλαπλούς actors, με έναν learner που εκπαιδεύεται από κοινό buffer. Το n-step double Q-learning αναφέρεται στην συνεισφορά των συνολικών ανταμοιβών, μετά από n βήματα και συμβάλλει στην καλύτερη μακροχρόνια εκτίμηση των ανταμοιβών [6].

Χρησιμοποιείται η αρχιτεκτονική dueling DQN με ένα LSTM επίπεδο μετά από τα συνελκτικά επίπεδα. Το dueling DQN διαχωρίζει την εκτίμηση της τιμής Q στον υπολογισμό της συνάρτησης Advantage $A(s,a)$ και της τιμής $V(s)$. Το LSTM επίπεδο είναι τύπου RNN και είναι σχεδιασμένο ώστε να εντοπίζει μακροχρόνιες συσχετίσεις μεταξύ καταστάσεων, ενεργειών και ανταμοιβών και να αντιμετωπίσει το πρόβλημα των vanishing gradients [6]. Οι μεταβάσεις που αποθηκεύονται στην experience replay δεν είναι της μορφής (s_t, a_t, r_t, s_{t+1}) αλλά (s_t, a_t, r_t) . Συγκεκριμένα, αποθηκεύονται ακολουθίες μεταβάσεων, οι οποίες αλληλεπικαλύπτονται στα πλαίσια του κάθε επεισοδίου [6].

3.1.2 Agent57

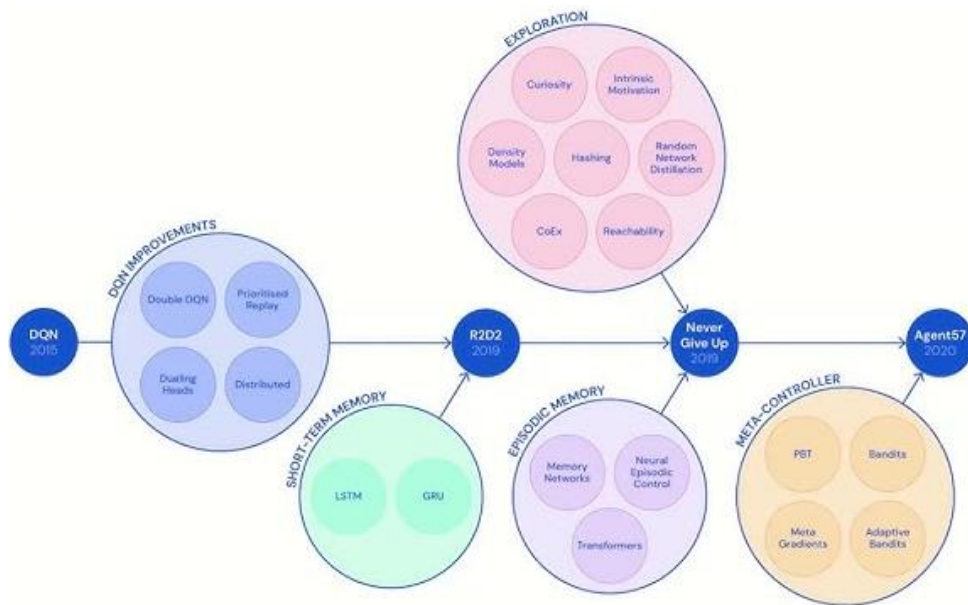
Ο αλγόριθμος Agent57 είναι ένας προηγμένος αλγόριθμος ενισχυτικής μάθησης, ο οποίος επεκτείνει τις δυνατότητες του R2D2 και του Never Give Up. Είναι ο πρώτος που κατάφερε να ξεπεράσει τις

επιδόσεις ενός μέσου ανθρώπου και στα 57 παιχνίδια Atari [7]. Ο υπολογισμός της αξίας Q διαχωρίζεται στον υπολογισμό της εσωτερικής (intrinsic) και εξωτερικής (extrinsic) ανταμοιβής και περιγράφεται από τον παρακάτω τύπο [7].

$$Q(x, a, j; \theta) = Q(x, a, j; \theta^e) + \beta_j Q(x, a, j; \theta^i)$$

Η παράμετρος β καθορίζει την βαρύτητα που έχει στον υπολογισμό της Q το intrinsic reward. Όσο μεγαλύτερο είναι το β , τόσο ο πράκτορας οδηγείται σε εξερεύνηση (exploration) του περιβάλλοντος, ενώ όσο μικρότερο είναι τόσο οδηγείται στην εκμετάλλευση (exploitation) του. Παράλληλα, στον υπολογισμό της αξίας V χρησιμοποιείται η παράμετρος γ , η οποία καθορίζει την αξία των μακροχρόνιων ανταμοιβών σε σχέση με τις άμεσες ανταμοιβές. Αυτό είναι ιδιαίτερα χρήσιμο για ορισμένα παιχνίδια Atari, όπου οι συνολικές ανταμοιβές θα είναι μεγαλύτερες αν ο πράκτορας δεν επιλέγει άπληστα την ενέργεια με την μεγαλύτερη ανταμοιβή σε κάθε κατάσταση [7].

Η κεντρική ιδέα του Agent57 είναι η δημιουργία ενός μετα-ελεγκτή (meta-controller), ο οποίος εκπαιδεύεται για να επιλέγει σε κάθε επεισόδιο το κατάλληλο ζευγάρι υπερ-παραμέτρων (β_i, γ_j) για κάθε actor που ενεργεί στο περιβάλλον, από ένα προκαθορισμένο σύνολο επιλογών. Κατά τα αρχικά στάδια της εκπαίδευσης είναι σύνηθες η επιλογή μεγάλου β και μικρού γ , ώστε ο πράκτορας να μάθει γρήγορα το περιβάλλον μέσα από εξερεύνηση. Όσο εξελίσσεται η εκπαίδευση επιλέγονται χαμηλές τιμές στο β και υψηλές στο γ , ώστε ο πράκτορας να εκμεταλλευτεί την γνώση που απέκτησε [7].



Σχήμα 3.1: Χρονοδιάγραμμα προγενέστερων αλγορίθμων του Agent57 και βελτιώσεις.

Συμπερασματικά, η επιτυχία του Agent57 έγκειται στην ικανότητα να ισορροπεί την σχέση exploration-exploitation καθ' όλη τη διάρκεια της εκπαίδευσης και να προσαρμόζεται σε προβλήματα με διαφορετικούς χρονικούς ορίζοντες ανταμοιβών [7].

3.2 Policy Gradient

Οι αλγόριθμοι ενισχυτικής μάθησης Policy Gradient έχουν ως κύριο στόχο την άμεση βελτιστοποίηση της πολιτικής του πράκτορα. Σε αντίθεση με τους Value-Based δεν βασίζονται στην πολιτική στην εκτίμηση της συνάρτησης αξίας, αλλά στην αναμενόμενη ανταμοιβή που πρόκειται να λάβει ο πράκτορας. Οι τελευταίοι αλγόριθμοι εστιάζουν σε καλύτερη εξερεύνηση του πράκτορα στο

περιβάλλον, προκειμένου να περιοριστούν προβλήματα σταθερότητας, χωρίς όμως να υστερούν σε απόδοση.

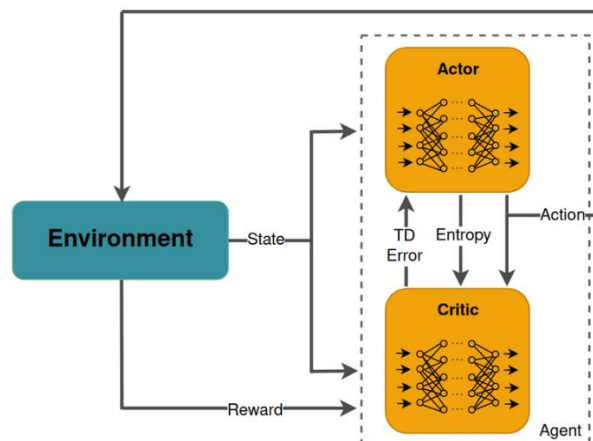
3.2.1 Soft Actor-Critic (SAC)

Ο Soft Actor-Critic αποτελεί μια βελτίωση του αλγορίθμου Actor-Critic που αναφέρθηκε στο κεφάλαιο 2. Ο κύριος στόχος του είναι η μεγιστοποίηση της αναμενόμενης ανταμοιβής, μεγιστοποιώντας την εντροπία. Συγκεκριμένα, επιδιώκεται η μέγιστη ανταμοιβή όσο ο πράκτορας ενεργεί όσο το δυνατόν πιο τυχαία [8].

Τυπικά στην ενισχυτική μάθηση μεγιστοποιείται το άθροισμα των αναμενόμενων ανταμοιβών: $J(\pi) = \sum_t E(s_t, a_t) \sim \rho_\pi[r(s_t, a_t)]$. Ο SAC όμως εισάγει την μεγιστοποίηση της εντροπίας ως εξής:

$$J(\pi) = \sum_{t=0}^T E(s_t, a_t) \sim \rho_\pi[r(s_t, a_t) + \alpha H(\pi(\cdot | s_t))]$$

Η συνάρτηση H είναι η εντροπία της πολιτικής στην κατάσταση s_t , ενώ η παράμετρος α ονομάζεται θερμοκρασία και ελέγχει την ισορροπία μεταξύ μεγιστοποίησης ανταμοιβής και εξερεύνησης [8]. Ο διπλός στόχος ενισχύει την εξερεύνηση του πράκτορα, αποτρέποντας τον από πρόωρη σύγκλιση σε τοπικά μέγιστα, και σταθεροποιεί την διαδικασία εκπαίδευσης [8].



Σχήμα 3.2: Αρχιτεκτονική Soft Actor-Critic για την αλληλεπίδραση με το περιβάλλον.

3.2.2 Phasic Policy Gradient (PPG)

Ο Phasic Policy Gradient είναι ένας προηγμένος Actor-Critic αλγόριθμος, ο οποίος διαχωρίζει την εκπαίδευση του policy και value function network μέσω δύο διακριτών εναλλασσόμενων φάσεων εκπαίδευσης, την φάση της πολιτικής (policy phase) και την βοηθητική φάση (auxiliary phase) [9].

Στην policy phase ο πράκτορας εκπαιδεύεται με μεθόδους policy gradient, όπως ο Proximal Policy Optimization (PPO). Συγκεκριμένα, εκπαιδεύονται δύο νευρωνικά δίκτυα που αντιπροσωπεύουν τα policy και value function, χρησιμοποιώντας την τεχνική του clipping για την ενημέρωση των βαρών [9].

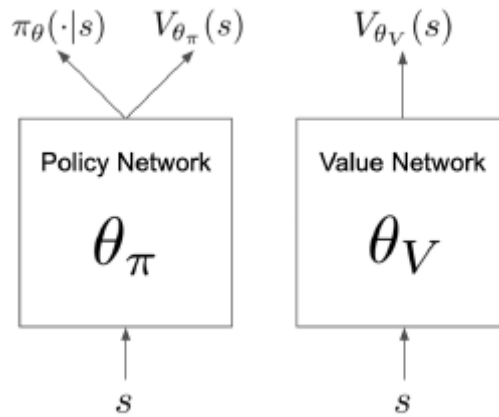
Στην auxiliary phase ο PPG εισάγει μια κοινή απώλεια (joint loss) που αποτελείται από κάποια auxiliary απώλεια και την απώλεια κλωνοποίησης συμπεριφοράς (behavioral cloning loss) [9].

$$L^{joint} = L^{aux} + \beta_{clone} \cdot \hat{E}_t[KL[\pi_{\theta_{old}}(\cdot | s_t), \pi_{\theta}(\cdot | s_t)]]$$

Ο όρος $\hat{E}_t[KL[\pi_{\theta_{old}}(\cdot | s_t), \pi_{\theta}(\cdot | s_t)]]$ ορίζει την μέση τιμή της Kullback-Leibler απόκλισης μεταξύ της πολιτικής πριν την auxiliary phase, $\pi_{\theta_{old}}$ και της νέας πολιτικής π_{θ} . Η υπερ-παραμέτρος β_{clone} ελέγχει την βαρύτητα αυτού του όρου στον υπολογισμό της κοινής απώλειας. Όσο μεγαλύτερη είναι η τιμή της, τόσο λιγότερο επηρεάζεται η νέα πολιτική σε σχέση με την παλιά. Η auxiliary απώλεια μπορεί να είναι η συνάρτηση απώλειας της value function, ή οποιαδήποτε βοηθητική συνάρτηση απώλειας [9].

$$L^{aux} = \frac{1}{2} \cdot \hat{E}_t[(V_{\theta_{\pi}}(s_t) - \hat{V}_t^{targ})^2]$$

Συμπερασματικά, η auxiliary phase επεμβαίνει στις ενημερώσεις των νευρωνικών, βελτιώνοντας τις εκτιμήσεις χωρίς όμως να μεταβάλλει απότομα την πολιτική. Συνεπώς ο PPG επιτυγχάνει επαναχρησιμοποίηση των δειγμάτων εκπαίδευσης αλλά και αποφυγή παρεμβολών στις ενημερώσεις των policy και value function δικτύων [9].

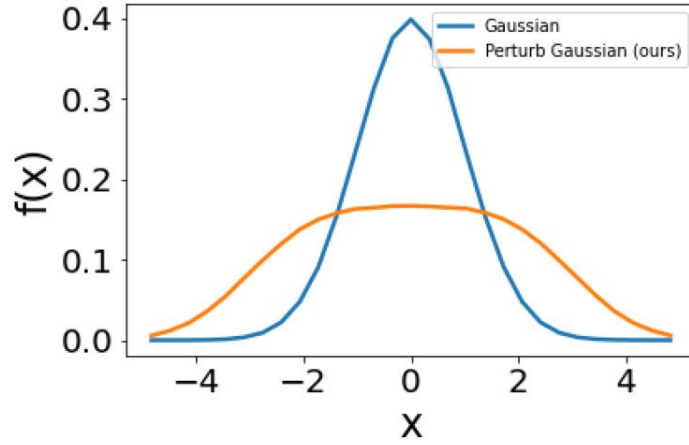


Σχήμα 3.3: Διαχωρισμός της εκπαίδευσης σε Policy και Value network

3.2.3 Robust Policy Optimization (RPO)

Κατά την εκπαίδευση ενός πράκτορα τυπικά η εντροπία, δηλαδή ο ρυθμός εξερεύνησης, είναι μεγάλη στα αρχικά στάδια και εξασθενεί όσο ο πράκτορας μαθαίνει το περιβάλλον. Σε συνεχή περιβάλλοντα αυτό οφείλεται στην κανονική (ή Gaussian) κατανομή των ενεργειών. Όσο ο πράκτορας επιλέγει τις βέλτιστες ενέργειες τόσο η κατανομή αυτή στενεύει προς την μέση τιμή και μειώνονται οι πιθανότητες επιλογής άλλης ενέργειας για περεταίρω εξερεύνηση [10].

Ο RPO έχει στόχο να διατηρήσει την τιμή της εντροπίας υψηλή καθ' όλη την διάρκεια της εκπαίδευσης. Αυτό επιτυγχάνεται μέσω του συνδυασμού κανονικής με ομοιόμορφη κατανομή (uniform). Σε κάθε βήμα προστίθεται ένας τυχαίος αριθμός από την ομοιόμορφη κατανομή $U(-\alpha, \alpha)$ στον μέσο όρο μ της κανονικής κατανομής $N(\mu, \sigma)$ και αυτό έχει ως αποτέλεσμα τον «διαταραγμένο» (perturbed) μέσο όρο μ' και την νέα κατανομή $N(\mu', \sigma)$. Αντίθετα με την κανονική κατανομή η ομοιόμορφη κατανομή είναι ανεξάρτητη της κατάστασης, επομένως η εντροπία παραμένει υψηλή και η δυνατότητα εξερεύνησης του πράκτορα διατηρείται κατά την διάρκεια της εκπαίδευσης [10].



Σχήμα 3.4: Κατανομή της RPO σε σχέση με την κανονική. Παρατηρείται μικρότερη συγκέντρωση σε μια τιμή και άρα μεγαλύτερη εντροπία.

3.2.4 IMPALA

Η μέθοδος Importance Weighted Actor-Learner Architecture (IMPALA) αποτελεί μια αρχιτεκτονική που μπορεί να εφαρμοστεί σε αλγορίθμους βαθιάς ενισχυτικής μάθησης. Είναι μια κατανεμημένη προσέγγιση με δυνατότητες επεκτασιμότητας (scalability), που υποστηρίζει χιλιάδες μηχανήματα χωρίς απώλειες σταθερότητας στην εκπαίδευση και απόδοσης χρήσης των δεδομένων [11].

Ο IMPALA αποτελείται από ένα σύνολο actors και έναν ή περισσότερους learners. Οι actors ενημερώνουν την τοπική πολιτική τους μ ανεξάρτητα, με βάση την πολιτική του learner π και εκτελούν η βήματα στο περιβάλλον. Μετά τα η βήματα, ο κάθε actor στέλνει ένα σύνολο εμπειριών με καταστάσεις, ενέργειες και ανταμοιβές $(x_1, a_1, r_1, \dots, x_n, a_n, r_n)$, μαζί με την πολιτική του $\mu(a_t|x_t)$ στην ουρά του learner. Στην συνέχεια ο learner ενημερώνει την δική του πολιτική π με τα batches που δέχεται από τους actors [11].

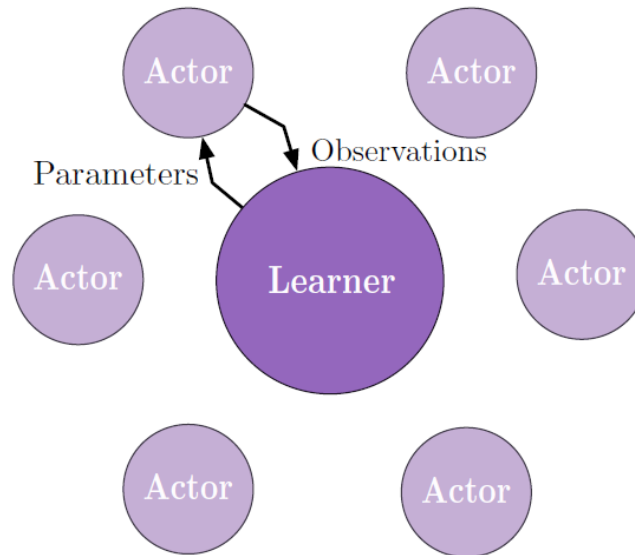
Ωστόσο επειδή η διαδικασία γίνεται ασύγχρονα, η πολιτική π του learner ενδέχεται να είναι κάποιες ενημερώσεις μπροστά σε σχέση με αυτή του actor, δημιουργώντας προβλήματα συνέπειας. Λύση στο παραπάνω πρόβλημα δίνει η εισαγωγή ενός διορθωτικού όρου στις ενημερώσεις της αξίας V και πολιτικής π , το V-trace. Το V-trace ορίζεται ως εξής:

$$v_s = V(x_s) + \sum_{t=s}^{s+n-1} \gamma^{t-s} \left(\prod_{i=s}^{t-1} c_i \right) \delta_t V$$

όπου $\delta_t V = \rho_t(r_t + \gamma V(x_{t+1}) - V(x_t))$, $\rho_t = \min\left(\bar{\rho}, \frac{\pi(a_t|x_t)}{\mu(a_t|x_t)}\right)$ και $c_i = \min\left(\bar{c}, \frac{\pi(a_i|x_i)}{\mu(a_i|x_i)}\right)$, $\bar{c}=1$. Η

κύρια συνιστώσα από τους παραπάνω τύπους, είναι πως ο λόγος $\frac{\pi(a_t|x_t)}{\mu(a_t|x_t)}$, που αποτελεί την διαφορά μεταξύ πολιτικής actor και learner για μια ενέργεια, μεταβάλλει αναλόγως την αξία αυτής της ενέργειας. Δηλαδή αν η μη ενημερωμένη πολιτική μ αξιολογεί υψηλά μια ενέργεια, ενώ η ενημερωμένη πολιτική π την αξιολογεί χαμηλά, η ενέργεια αυτή θα υποβαθμιστεί ανάλογα [11]. Τελικά η ενημέρωση των V και π γίνονται με τον εξής τρόπο για την αποφυγή τυχών ασυνεπειών:

$$(v_s - V_\theta(x_s)) \nabla_\theta V_\theta(x_s) \text{ και } \rho_s \nabla_\omega \log \pi_\omega(a_s|x_s)(r_s + \gamma v_{s+1} - V_\theta(x_s)).$$

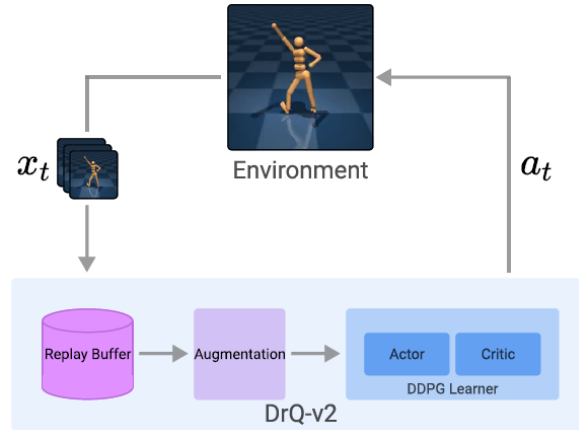


Σχήμα 3.5: Κάθε actor στέλνει τις παρατηρήσεις του στον Learner και δέχεται την πολιτική του.

3.2.5 DrQ-v2

Ο DrQ-v2 είναι ένας actor-critic αλγόριθμος ενισχυτικής μάθησης, επέκταση του DrQ, προσαρμοσμένος στην επέκταση δεδομένων εικόνας (data augmentation) [12]. Όπως και στον DrQ εφαρμόζονται τυχαίες μετατοπίσεις σε εικόνες παρατήρησης του περιβάλλοντος. Συγκεκριμένα, σε κάθε 84x84 εικόνα προστίθενται 4 pixel σε κάθε πλευρά, επαναλαμβάνοντας τα pixel στα άκρα, και από την 92x92 εικόνα που προκύπτει, γίνεται τυχαία αποκοπή σε 84x84. Η παραγόμενη εικόνα είναι η αρχική αλλά μετατοπισμένη κατά ± 4 pixel. Επιπρόσθετα, εφαρμόζεται διγραμμική παρεμβολή (bilinear interpolation) στην μετατοπισμένη εικόνα, δηλαδή η τιμή κάθε pixel αντικαθίσταται από την μέση τιμή των τεσσάρων γειτονικών pixel. Οι παραπάνω τροποποιήσεις συμβάλλουν στην καλύτερη επίδοση και ικανότητα γενίκευσης του αλγορίθμου [12].

Οι «επαυξημένες» εικόνες αποτελούν είσοδο σε ένα συνελικτικό κωδικοποιητή (convolutional encoder) ώστε να μειωθεί η διάστασή τους, χωρίς κάποια απώλεια. Η διαδικασία μπορεί να αναπαρασταθεί με την σχέση: $h = f_{\xi}(aug(x))$, όπου f_{ξ} είναι ο κωδικοποιητής, aug είναι οι μετατοπίσεις στην εικόνα και x η αρχική εικόνα παρατήρησης [12]. Μετά την προεπεξεργασία των δεδομένων, ο πράκτορας εκπαιδεύεται χρησιμοποιώντας τον actor-critic αλγόριθμο DDPG με ορισμένες βελτιώσεις. Ειδικότερα αυξάνεται το μέγεθος του replay buffer και μειώνεται το μέγεθος των mini-batch καθώς και του ρυθμού μάθησης. Οι αλλαγές αυτές βελτιώνουν σημαντικά την ταχύτητα, σταθερότητα και επίδοση του προγενέστερου αλγορίθμου DrQ [12].



Σχήμα 3.6: Ροή διεργασιών του DrQ-v2.

3.3 Model-Based

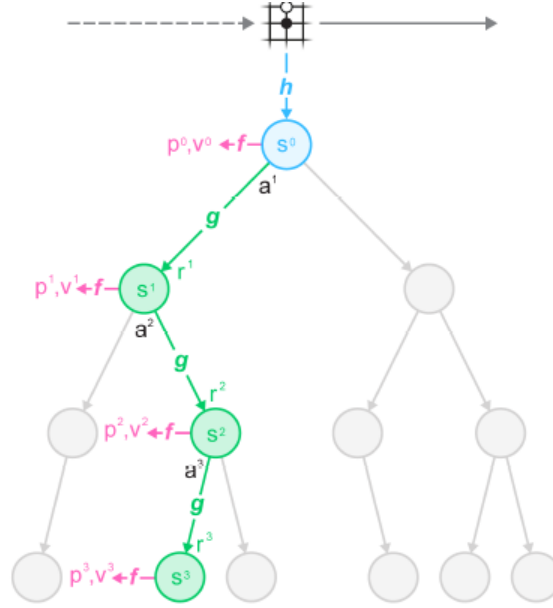
Οι Model-Based αλγόριθμοι αποτελούν μία κατηγορία αλγορίθμων ενισχυτικής μάθησης που αποσκοπούν στην κατασκευή ενός μοντέλου του περιβάλλοντος. Το μοντέλο αυτό επιτρέπει στον πράκτορα να προβλέπει τα αποτελέσματα των ενεργειών του, χωρίς την ανάγκη άμεσης αλληλεπίδρασης και κατά συνέπεια μεγάλου πλήθους δειγμάτων εκπαίδευσης. Οι αλγόριθμοι Model-Based είναι κατάλληλοι για πολύπλοκα προβλήματα και υπερτερούν έναντι model-free μεθόδων σε περιπτώσεις όπου η πληροφορία του πράκτορα είναι περιορισμένη και απαιτείται σχεδιασμός μακροπρόθεσμων ενεργειών.

3.3.1 MuZero

Ο MuZero είναι ένας model-based αλγόριθμος ενισχυτικής μάθησης, ο οποίος εν αντιθέσει άλλων model-based προσεγγίσεων, δεν μαθαίνει ολόκληρο το περιβάλλον, παρά μόνο προβλέπει τα πλήρως απαραίτητα στοιχεία του μέσω κρυφών καταστάσεων [13]. Σε κάθε χρονικό βήμα του αλγορίθμου γίνονται προβλέψεις πολιτικής $p_t^k \approx \pi(a_{t+k+1} | o_1, \dots, o_t, a_{t+1}, \dots, a_{t+k})$, συνάρτησης αξίας $v_t^k \approx E[u_{t+k+1} + \gamma u_{t+k+2} + \dots | o_1, \dots, o_t, a_{t+k}]$ και άμεσης ανταμοιβής $r_t^k \approx u_{t+k}$. Οι προβλέψεις γίνονται με δεδομένη κάποια ενέργεια και βασίζονται στις παρατηρήσεις που έχει συλλέξει ο πράκτορας από την αλληλεπίδραση του με το περιβάλλον [13].

Το μοντέλο εσωτερικά αναπαρίσταται από τον συνδυασμό των συναρτήσεων αναπαράστασης (representation), δυναμικής (dynamics) και πρόβλεψης (prediction). Η συνάρτηση δυναμικής $r^k, s^k = g_\theta(s^{k-1}, a^k)$ αποτελεί μία επαναλαμβανόμενη διαδικασία που υπολογίζει σε κάθε υποθετικό βήμα k , μία άμεση ανταμοιβή r^k και μια κρυφή κατάσταση s^k . Η συνάρτηση πρόβλεψης, $p^k, v^k = f_\theta(s^k)$, λαμβάνει την s^k και υπολογίζει την πολιτική και την συνάρτηση αξίας, όπως ο προγενέστερος αλγόριθμος AlphaZero. Τέλος, η αρχική κρυφή κατάσταση s^0 αρχικοποιείται από την συνάρτηση αναπαράστασης, $s^0 = h_\theta(o_1, \dots, o_t)$, που δέχεται τις προηγούμενες παρατηρήσεις του πράκτορα [13].

Το δέντρο κρυφών καταστάσεων που σχηματίζεται, όπως φαίνεται και στο σχήμα 3.5, διασχίζεται με μαρκοβιανούς αλγορίθμους σχεδιασμού ενεργειών για την επιλογή βέλτιστου μονοπατιού. Συγκεκριμένα χρησιμοποιείται ένας Monte-Carlo αλγόριθμος αναζήτησης σε δέντρο (Monte-Carlo Tree Search, MCTS) που αξιοποιεί την πολιτική, αξία και ανταμοιβή κάθε κρυφής κατάστασης και παράγει μια προτεινόμενη πολιτική και εκτιμώμενη αξία, ώστε να επιλεγεί η κατάλληλη ενέργεια [13].



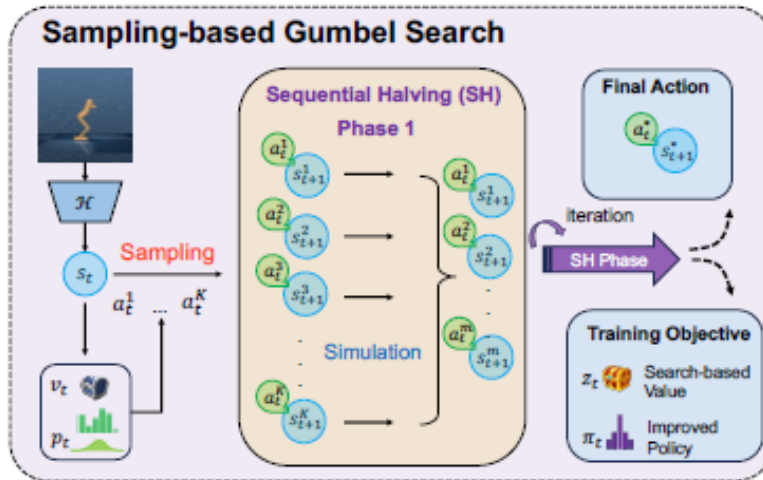
Σχήμα 3.7: Αναπαράσταση μοντέλου του περιβάλλοντος στον MuZero με συνδυασμό συναρτήσεων αναπαράστασης, δυναμικής και πρόβλεψης.

3.3.2 EfficientZero V2

Ο αλγόριθμος EfficientZero V2 αποτελεί επέκταση του EfficientZero, ο οποίος στοχεύει στην καλύτερη επαναχρησιμοποίηση των δειγμάτων εκπαίδευσης (sample efficiency) από άλλους model-based αλγορίθμους, όπως ο MuZero. Ομοίως με τον MuZero, ο EZ-V2 για την αναπαράσταση του μοντέλου του περιβάλλοντος, απαρτίζεται από συναρτήσεις (νευρωνικά δίκτυα) αναπαράστασης, δυναμικής, πρόβλεψης πολιτικής και αξίας [14].

Για την συνάρτηση δυναμικής, ο EfficientZero χρησιμοποιεί μάθηση με επίβλεψη. Συγκεκριμένα, η προβλεπόμενη κρυφή κατάσταση και ανταμοιβή που παράγεται, συγκρίνεται με την πραγματική κατάσταση και ανταμοιβή που προκύπτουν αν εκτελεστεί η ενέργεια στο περιβάλλον. Συνεπώς το μοντέλο του περιβάλλοντος μπορεί να βελτιωθεί ελαχιστοποιώντας την διαφορά μεταξύ πρόβλεψης και πραγματικής τιμής και ως εκ τούτου να προκύψουν ακριβέστερες προβλέψεις ενεργειών. Η διάσχιση του εκπαιδευμένου μοντέλου γίνεται με τον αλγόριθμο MCTS, ο οποίος εξάγει την προβλεπόμενη πολιτική και αξία της κατάστασης αλλά και την προτεινόμενη ενέργεια. Ο EfficientZero V2 κληρονομεί τα παραπάνω χαρακτηριστικά, όμως αντικαθιστά τον MCTS με αναζήτηση Gumbel βασισμένη σε δειγματοληψία (Gumbel sampling-based search), η οποία εγγυάται βελτίωση της πολιτικής σε συνεχή περιβάλλοντα [14].

Η αναζήτηση Gumbel επιλέγει δειγματοληπτικά K ενέργειες χρησιμοποιώντας το Gumbel-Top-k trick. Μέσω αυτού, επιλέγονται οι K επικρατέστερες ενέργειες μετά την προσθήκη τυχαίου θορύβου Gumbel για την ενίσχυση της εξερεύνησης του πράκτορα. Στην συνέχεια, μοντελοποιεί την επιλογή της κατάλληλης, από τις K , ενέργειας σε ένα πρόβλημα bandit που στοχεύει στην επιλογή της ενέργειας με την υψηλότερη αξία Q . Για την επίλυση του, επιστρατεύεται ένας bandit αλγόριθμος που ονομάζεται Sequential Halving. Δεδομένης μιας κατάστασης s , οι K υποψήφιας ενέργειες επιλέγονται τόσο από την τρέχουσα πολιτική, όσο και από προγενέστερη. Από το σύνολο των ενεργειών ο Sequential Halving σε κάθε επαναληπτική φάση αξιολογεί τις ενέργειες με την αξία Q και απορρίπτει το μισό σύνολο με τις χαμηλότερες αξίες. Η διαδικασία συνεχίζεται έως ότου παραμείνει μία ενέργεια. Συνεπώς η δειγματοληπτική αναζήτηση Gumbel ολοκληρώνεται επιστρέφοντας την βέλτιστη ενέργεια με την αξία της, καθώς και την βελτιωμένη πολιτική του πράκτορα [14].



Σχήμα 3.8: Διαδικασία της δειγματοληπτικής αναζήτησης Gumbel, για την διάσχιση του δέντρου του μοντέλου στον EfficientZero V2.

Η παραπάνω αναζήτηση πετυχαίνει ισορροπία μεταξύ εξερεύνησης και εκμετάλλευσης του περιβάλλοντος, που είναι ιδιαίτερα σημαντική στην επίδοση διάσχισης δέντρου, ειδικότερα όταν τα δεδομένα εκπαίδευσης είναι περιορισμένα [14].

3.3.3 DreamerV3

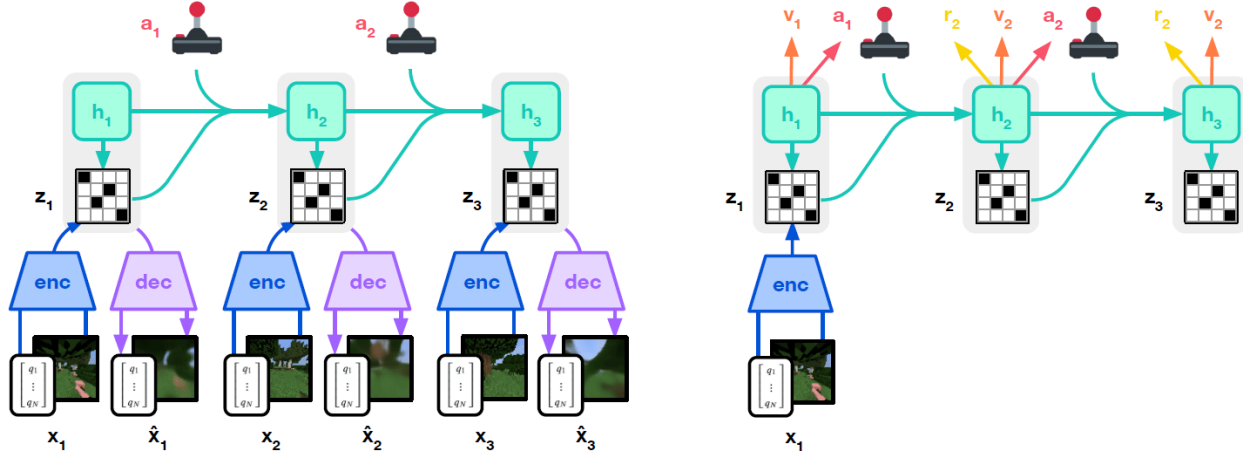
Ο DreamerV3 είναι ένας προηγμένος αλγόριθμος ενισχυτικής μάθησης με πολύ υψηλές αποδόσεις σε διαφορετικού τύπου περιβάλλοντα. Βασίζεται στην μάθηση ενός μοντέλου κόσμου, το οποίο χρησιμοποιεί ο πράκτορας για την αξιολόγηση των μελλοντικών ενεργειών του [15]. Ο αλγόριθμος αυτός αποτελείται ουσιαστικά από τρία νευρωνικά δίκτυα: το μοντέλο κόσμου (world model) που προβλέπει τις συνέπειες πιθανών ενεργειών, τον critic που αξιολογεί την αξία των καταστάσεων που προκύπτουν και τον actor που επιλέγει ενέργειες στοχεύοντας στο μεγαλύτερο κέρδος. Η εκπαίδευση τους γίνεται παράλληλα από την experience replay memory, όσο ο πράκτορας αλληλεπιδρά με το περιβάλλον [15].

Το μοντέλο κόσμου αναπαρίσταται ως ένα αναδρομικό μοντέλο χώρου καταστάσεων (Recurrent State-Space Model, RSSM) που αποτελείται από ορισμένα συστατικά. Αρχικά ένας κωδικοποιητής (encoder) λαμβάνει ένα δεδομένο παρατήρησης, όπως μία εικόνα του περιβάλλοντος, x_t και δίνει ως έξοδο την διακριτή αναπαράσταση του σε μικρότερη διάσταση z_t . Στην συνέχεια, ένα μοντέλο ακολουθιών (sequence model) δεδομένου της αναπαράστασης αυτής, μίας προηγούμενης ενέργειας a_{t-1} και μίας αναδρομικής κατάστασης h_t προβλέπει την επόμενη αναδρομική κατάσταση. Συνεπώς κάθε αναδρομική κατάσταση εξαρτάται από όλες τις προηγούμενες. Αντίστροφα, υπάρχει ένα νευρωνικό δίκτυο πρόβλεψης δυναμικής (dynamics predictor), το οποίο δεδομένης αναδρομικής κατάστασης, προβλέπει την z_t αναπαράσταση της. Επιπρόσθετα, σε κάθε αναδρομική κατάσταση προβλέπεται η ανταμοιβή (reward predictor) και το αν αυτή θα είναι τερματική (continue predictor). Τέλος, υπάρχει ένας αποκωδικοποιητής (decoder) που δέχεται h_t και z_t και προσπαθεί να ανακατασκευάσει την αρχική παρατήρηση [15].

Η εκπαίδευση του νευρωνικού του μοντέλου κόσμου, πραγματοποιείται ελαχιστοποιώντας την παρακάτω συνάρτηση απώλειας:

$$L(\varphi) = E_{q_\varphi} \left[\sum_{t=1}^T (\beta_{pred} L_{pred}(\varphi) + \beta_{dyn} L_{dyn}(\varphi) + \beta_{rep} L_{rep}(\varphi)) \right]$$

Παράλληλα, οι actor-critic αξιοποιούν τις προβλέψεις του μοντέλου κόσμου, ώστε ο critic να αξιολογεί τις προβλεπόμενες αναδρομικές καταστάσεις, ενώ ο actor να επιλέγει την βέλτιστη ενέργεια προς την επόμενη κατάσταση, που θα επιφέρει την μέγιστη ανταμοιβή [15].



Σχήμα 3.9: Αριστερά: Εκπαίδευση μοντέλου κόσμου. Δεξιά: Εκπαίδευση actor-critic.

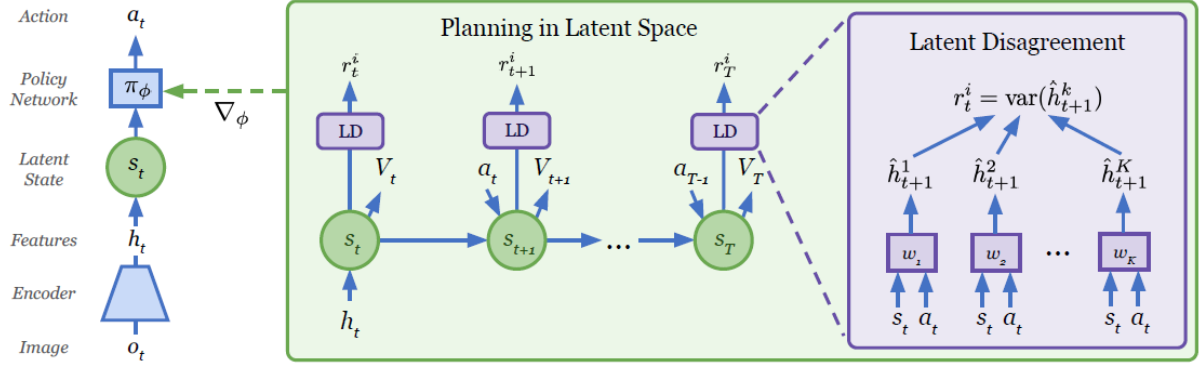
3.3.4 Plan2Explore

Ο αλγόριθμος Plan2Explore είναι μία διαφορετική προσέγγιση στους model-based αλγορίθμους ενισχυτικής μάθησης. Ο κύριος στόχος του είναι η εκπαίδευση ενός πράκτορα που κάνει εξερεύνηση του περιβάλλοντος, χωρίς να δέχεται ανταμοιβές ενεργειών [16]. Ο Plan2Explore αρχικά δέχεται μια παρατήρηση του περιβάλλοντος o_t και χρησιμοποιεί έναν κωδικοποιητή, δηλαδή ένα συνελκτικό νευρωνικό δίκτυο, για να εξάγει τα χαρακτηριστικά της h_t .

Από τον συνδυασμό των χαρακτηριστικών και της προηγούμενης κατάστασης, προκύπτει η λανθάνουσα κατάσταση s_t , η οποία αποτελεί μια αναπαράσταση της παρατήρησης σε μικρότερη διάσταση. Για την επιλογή ενέργειας από την κατάσταση αυτή, δεδομένου ότι δεν υπάρχουν τυπικές ανταμοιβές, θα πρέπει να εκπαιδευτεί ένα δίκτυο πολιτικής π_ϕ μέσω μιας διαδικασίας σχεδιασμού ενεργειών. Προσεγγίσεις χωρίς κάποιο μοντέλο του περιβάλλοντος είναι αναποτελεσματικές, καθώς οι καινούριες καταστάσεις που επισκέπτεται ο πράκτορας, παύουν να είναι σημαντικές στην εξερεύνηση αφού έχουν ήδη εξερευνηθεί. Συνεπώς συνίσταται η εκπαίδευση ενός μοντέλου του περιβάλλοντος, το οποίο θα προβλέπει τις καταστάσεις χωρίς να απαιτείται η επίσκεψη τους [16].

Το μοντέλο περιβάλλοντος, δέχεται ως είσοδο τα χαρακτηριστικά h_t και προβλέπει τις εκτιμώμενες μελλοντικές κρυφές λανθάνουσες καταστάσεις s_t χρησιμοποιώντας τον αλγόριθμο Dreamer. Για κάθε κρυφή κατάσταση υπολογίζεται η εσωτερική (intrinsic) ανταμοιβή, μέσω μιας διαδικασίας λανθάνουσας ασυμφωνίας (latent disagreement). Οπότε, ο πράκτορας εκπαιδεύει την πολιτική του προσπαθώντας να μεγιστοποιήσει το άθροισμα των intrinsic ανταμοιβών των προβλεπόμενων μελλοντικών του καταστάσεων, προκειμένου να επιλέξει την κατάλληλη ενέργεια προς εξερεύνηση του περιβάλλοντος [16].

Η Latent Disagreement είναι ένας συγκεκριασμός μοντέλων $w_1, \dots, w_K$ με διαφορετικές παραμέτρους, τα οποία δέχονται την ίδια κατάσταση και ενέργεια s_t, a_t . Το καθένα παράγει ως έξοδο μία πρόβλεψη h_t , μόνο των χαρακτηριστικών της επόμενης κατάστασης που προκύπτει μετά την ενέργεια a_t . Η intrinsic ανταμοιβή τελικά προκύπτει από την διακύμανση των προβλέψεων: $r_t^i = \text{var}\{\widehat{h_{t+1}^k}\}$. Αν αυτή είναι υψηλή, τότε υπάρχει αβεβαιότητα για τις συνέπειες της επόμενης κατάστασης, γεγονός που αποτελεί τον στόχο του Plan2Explore [16].



Σχήμα 3.10: Επισκόπηση του αλγορίθμου Plan2Explore.

3.4 Exploration Module

Η αποτελεσματική εξερεύνηση του περιβάλλοντος είναι ένα θεμελιώδες ζήτημα στον τομέα της ενισχυτικής μάθησης, κυρίως σε περιπτώσεις που οι ανταμοιβές είναι σπάνιες και ο πράκτορας δεν λαμβάνει την απαραίτητη ανατροφοδότηση για να εκπαιδευτεί. Ως εκ τούτου, έχουν αναπτυχθεί μέθοδοι ώστε ο πράκτορας να λαμβάνει εσωτερικές ανταμοιβές όταν αυτός επιλέγει ενέργειες προκειμένου να εξερευνήσει το περιβάλλον. Η προσάρτηση μεθόδων exploration εξισορροπούν τη σχέση exploration-exploitation και ενισχύουν την ικανότητα γενίκευσης των πρακτόρων.

3.4.1 Intrinsic Curiosity Module (ICM)

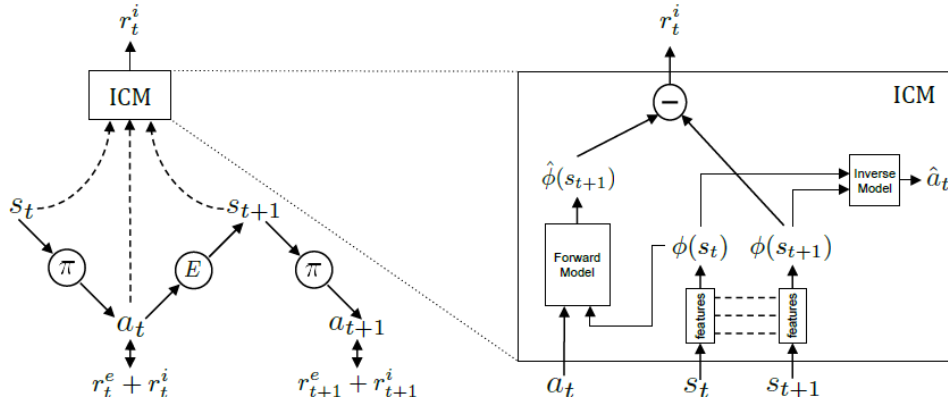
Σε πολλαπλά σενάρια πραγματικού κόσμου, οι εξωτερικές ανταμοιβές ενός πράκτορα που αλληλεπιδρά με ένα περιβάλλον, είναι σπάνιες ή ακόμα και μηδενικές. Σε αυτές τις περιπτώσεις, η «περιέργεια» (curiosity) μπορεί να αποτελέσει την εσωτερική ανταμοιβή, ώστε ο πράκτορας να μπορέσει να εξερευνήσει το περιβάλλον. Ως «περιέργεια» ορίζεται η αδυναμία ενός πράκτορα να προβλέψει τις συνέπειες των ενεργειών του [17].

Ο πράκτορας ξεκινά από μία κατάσταση s_t , χρησιμοποιεί μία πολιτική π και εκτελεί μία ενέργεια a_t στο περιβάλλον $\mathcal{E}$. Αποτέλεσμα της ενέργειας είναι η κατάσταση s_{t+1} και η ενέργεια a_{t+1} . Η πολιτική που παράγει τις ενέργειες, εκπαιδεύεται ώστε να μεγιστοποιήσει το άθροισμα των εσωτερικών και εξωτερικών (αν υπάρχουν) ανταμοιβών. Οι εσωτερικές ανταμοιβές παράγονται από την μονάδα εσωτερικής περιέργειας (Intrinsic Curiosity Module, ICM) [17].

Η ICM αρχικά κωδικοποιεί τις καταστάσεις s_t, s_{t+1} στα κύρια χαρακτηριστικά τους $\phi(s_t), \phi(s_{t+1})$. Ο κωδικοποιητής εκπαιδεύεται μαζί με ένα αντίστροφο μοντέλο (inverse model), το οποίο προβλέπει την ενέργεια που εκτελέστηκε από την $\phi(s_t)$ στην $\phi(s_{t+1})$. Με αυτόν τον τρόπο τα χαρακτηριστικά των καταστάσεων που παράγει ο κωδικοποιητής περιορίζονται μόνο σε αυτά που χρειάζονται για να υπολογιστεί η ενδιάμεση ενέργεια. Στην συνέχεια, ένα μοντέλο προώθησης (forward model) δέχεται την ενέργεια a_t , που υπολόγισε η πολιτική και την αναπαράσταση $\phi(s_t)$ και προβλέπει τα κύρια χαρακτηριστικά της επόμενης κατάστασης $\hat{\phi}(s_{t+1})$. Η διαφορά μεταξύ της προβλεπόμενης αναπαράστασης στα κύρια χαρακτηριστικά και της πραγματικής αναπαράστασης από τον κωδικοποιητή, ισούται με την εσωτερική ανταμοιβή:

$$r_t^i = \frac{\eta}{2} \|\hat{\phi}(s_{t+1}) - \phi(s_{t+1})\|_2^2$$

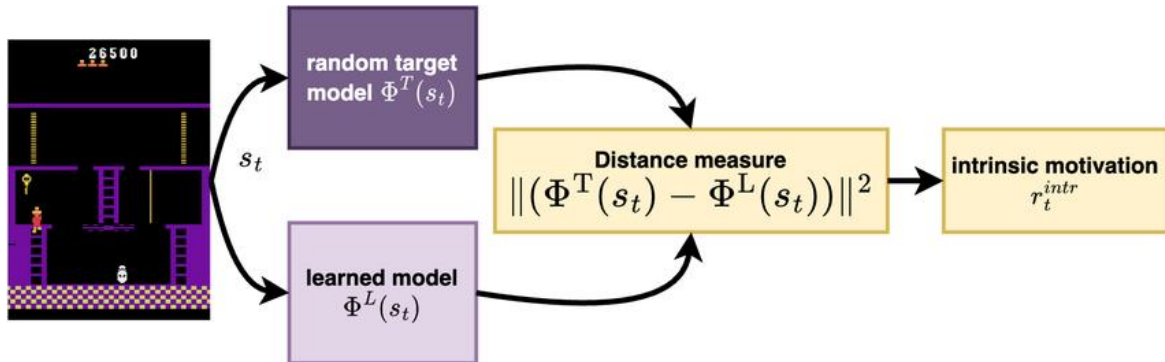
Όσο μεγαλύτερη είναι αυτή η διαφορά, τόσο μεγαλύτερη είναι η αβεβαιότητα για αυτήν την κατάσταση και άρα τόσο μεγαλύτερη πρέπει να είναι η εσωτερική ανταμοιβή που θα λάβει ο πράκτορας, προκειμένου να εξερευνήσει νέες καταστάσεις [17].



Σχήμα 3.11: Διαδικασία παραγωγής intrinsic reward μέσω ICM.

3.4.2 Random Network Distillation (RND)

Μία τεχνική που ενθαρρύνει έναν πράκτορα να εξερευνήσει ένα περιβάλλον ενισχυτικής μάθησης, είναι η Random Network Distillation. Περιλαμβάνει δύο νευρωνικά δίκτυα: ένα σταθερό δίκτυο στόχος (target network) με τυχαίες αρχικές παραμέτρους και ένα δίκτυο πρόβλεψης (predictor network) που προσπαθεί να προβλέψει την έξοδο του δικτύου στόχου. Συγκεκριμένα, το δίκτυο πρόβλεψης εκπαιδεύεται με την μέθοδο της επικλινής καθόδου (gradient descent) ώστε να ελαχιστοποιήσει το εκτιμώμενο μέσο τετραγωνικό σφάλμα: $\|\hat{f}(x; \theta) - f(x)\|^2$ μεταξύ των δύο δικτύων. Το σφάλμα πρόβλεψης αποτελεί την εσωτερική ανταμοιβή και αναμένεται να είναι υψηλότερο για ανεξερευνήτες καταστάσεις, διαφορετικές από αυτές που έχει εκπαιδευτεί το δίκτυο πρόβλεψης [18].



Σχήμα 3.12: Διαμόρφωση Intrinsic reward (motivation) με Random Network Distillation.

Ένα καίριο πρόβλημα στην εξερεύνηση που αντιμετωπίζουν οι πράκτορες, είναι όταν το περιβάλλον είναι στοχαστικό. Όταν υπάρχουν θορυβώδη στιγμιότυπα, όπως για παράδειγμα μία τηλεόραση που προβάλλει συνεχώς τυχαίες εικόνες, ο πράκτορας μπορεί να αποπροσανατολιστεί εξερευνώντας καταστάσεις που δεν βοηθούν στην επίλυση του προβλήματος. Εν αντιθέσει της ICM, η RND επιλύει αυτό το πρόβλημα, καθώς λόγω της αμεταβλητότητας του δικτύου στόχου, το δίκτυο πρόβλεψης καταφέρνει να προσεγγίσει την έξοδο του για συνεχόμενα τυχαία δεδομένα [18].

3.4.3 Efficient Random Encoders (RE3)

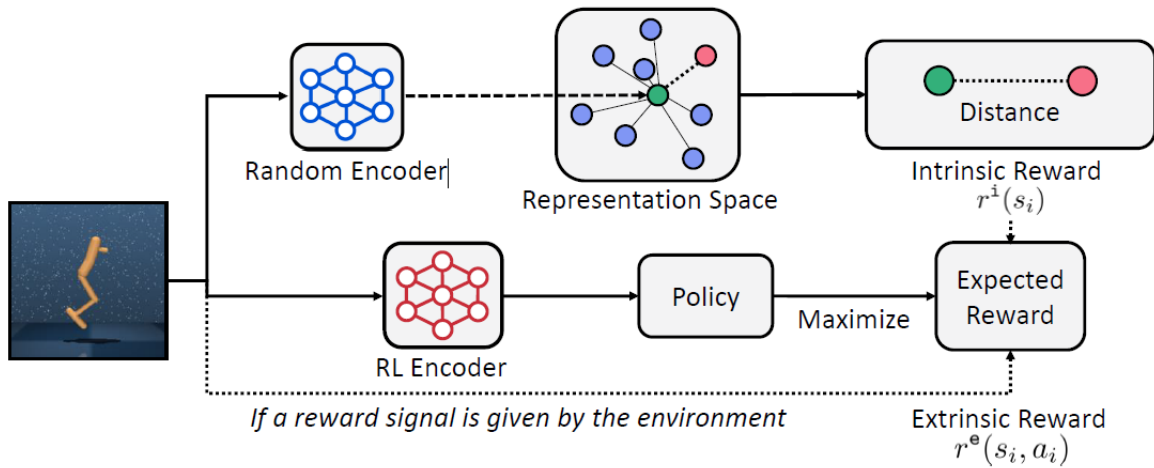
Η μέθοδος των αποδοτικών τυχαίων κωδικοποιητών ή RE3 στοχεύει στην βελτίωση της αποδοτικότητας δειγματοληψίας, ενθαρρύνοντας τον πράκτορα να εξερευνήσει νέες καταστάσεις, μεγιστοποιώντας την εντροπία της κατανομής καταστάσεων [19].

Πραγματοποιείται τυχαία αρχικοποίηση ενός κωδικοποιητή, που δεν εκπαιδεύεται, ο οποίος υπολογίζει την αναπαράσταση μίας κατάστασης σε μικρότερη διάσταση. Η εσωτερική ανταμοιβή, υπολογίζεται από την απόσταση μεταξύ της εν λόγω παρατήρησης y_i και της k πλησιέστερης της y_i^{k-NN} στον χώρο αναπαράστασης, σύμφωνα με τον παρακάτω τύπο:

$$r^i(s_i) := \log(\|y_i - y_i^{k-NN}\|_2 + 1)$$

Ο υπολογισμός της απόστασης μεταξύ καταστάσεων στον χώρο αναπαράστασης έχει ως αποτέλεσμα πιο σταθερές εσωτερικές ανταμοιβές, καθώς η απόσταση αυτή δεν μεταβάλλεται κατά την διάρκεια της εκπαίδευσης. Για την αποφυγή πολύπλοκων υπολογισμών σε κάθε βήμα, οι αναπαραστάσεις των καταστάσεων αποθηκεύονται στο replay buffer [19].

Η πολιτική που θα ακολουθήσει ο πράκτορας εξάγεται από έναν κωδικοποιητή RL. Η εκπαίδευση της βασίζεται στην μεγιστοποίηση των συνολικών εκτιμώμενων εσωτερικών και εξωτερικών ανταμοιβών, με κάποιον συντελεστή που καθορίζει την ισορροπία μεταξύ εξερεύνησης και εκμετάλλευσης [19].



Σχήμα 3.13: Επισκόπηση της RE3 για τον υπολογισμό *intrinsic reward*.

Κεφάλαιο 4

Σύγκριση και αξιολόγηση Προηγμένων Αλγορίθμων Βαθιάς Ενισχυτικής Μάθησης

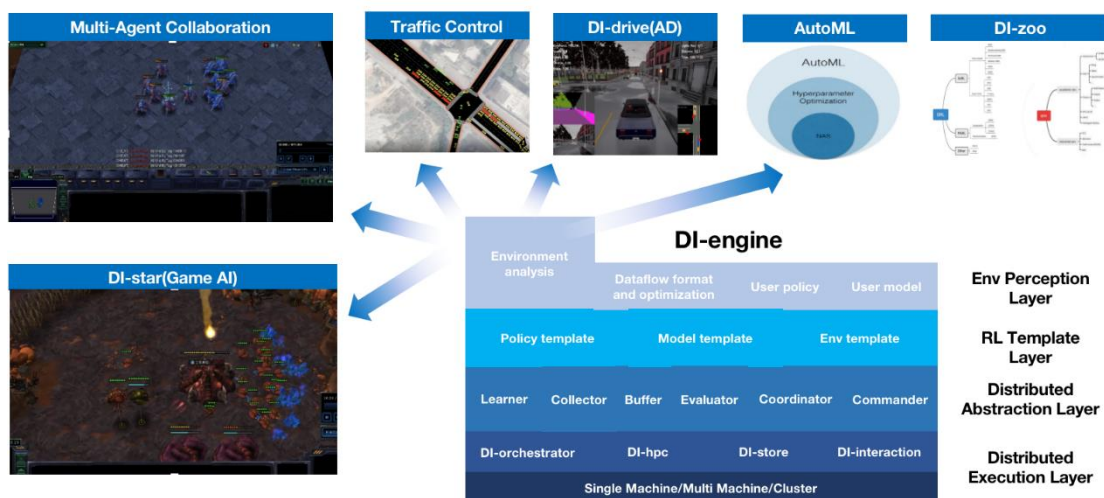
Στο παρόν κεφάλαιο, γίνεται καταγραφή των απαραίτητων βημάτων και μεθόδων που απαιτούνται για την εφαρμογή προηγμένων αλγορίθμων ενισχυτικής μάθησης στα επιλεγμένα περιβάλλοντα εκπαίδευσης. Αρχικά, παρουσιάζονται οι βιβλιοθήκες και framework που αξιοποιήθηκαν με ιδιαίτερη έμφαση στο DI-engine, μέσω του οποίου επιτρέπεται η παραμετροποίηση και εκτέλεση των αλγορίθμων. Στην συνέχεια, περιγράφονται τα περιβάλλοντα εκπαίδευσης στα οποία ενεργούν οι πράκτορες, ποιοι αλγόριθμοι επιλέχθηκαν και με ποιες υπερ-παραμέτρους. Επιπλέον, αναλύεται η αλυσίδα διεργασιών που απαιτείται προκειμένου να εκτελεστεί και να αξιολογηθεί ένας αλγόριθμος. Τέλος, παρουσιάζονται τα πειράματα που διεξήχθησαν και τα αποτελέσματα εκπαίδευσης και γίνονται συγκρίσεις μεταξύ των ελεγμένων αλγορίθμων.

4.1 Βιβλιοθήκες

Για την εκτέλεση των πειραμάτων αξιοποιήθηκαν αρκετές βιβλιοθήκες και framework που προσφέρει η Python. Η βασικότερη εξ αυτών είναι το DI-engine, το οποίο παρείχε τους αλγορίθμους ενισχυτικής μάθησης προς πειραματισμό και αξιολόγηση, καθώς και την αλυσίδα επεξεργασίας (pipeline) της εκπαίδευσης. Στόχος αυτής της ενότητας είναι η αποδόμηση του DI-engine αλλά και ορισμένων βασικών βιβλιοθηκών που αυτό εμπεριέχει προκειμένου να υλοποιήσει τις παραπάνω διαδικασίες.

4.1.1 DI-Engine

Το DI-engine είναι ένα ανοιχτού κώδικα πλαίσιο (framework) ενισχυτικής μάθησης, γραμμένο σε Python και ανεπτυγμένο από το OpenDILab. Στόχος του είναι η παροχή ενός ενοποιημένου, ευέλικτου και επεκτάσιμου πλαισίου ανάπτυξης, εφαρμογής και αξιολόγησης αλγορίθμων ενισχυτικής μάθησης. Απευθύνεται τόσο σε ακαδημαϊκή έρευνα όσο και σε εφαρμογές λογισμικού, όπως αυτόνομη οδήγηση ή παιχνίδια στρατηγικής [20].



Σχήμα 4.1: Αρχιτεκτονική του DI-engine και περιπτώσεις χρήσης.

Αρχιτεκτονική

Η αρχιτεκτονική του είναι βασισμένη σε middleware, όπου κάθε στάδιο εκπαίδευσης και αξιολόγησης υλοποιείται ως ανεξάρτητο και επαναχρησιμοποιήσιμο στοιχείο, επιτρέποντας την ευελιξία στην παραμετροποίηση του pipeline. Η προσέγγιση αυτή διευκολύνει την τροποποίηση επιμέρους συστατικών, χωρίς να απαιτείται ανασχεδιασμός ολόκληρου του συστήματος. Τα δομικά στοιχεία που περιλαμβάνει το pipeline μιας διαδικασίας ενισχυτικής μάθησης είναι τα εξής:

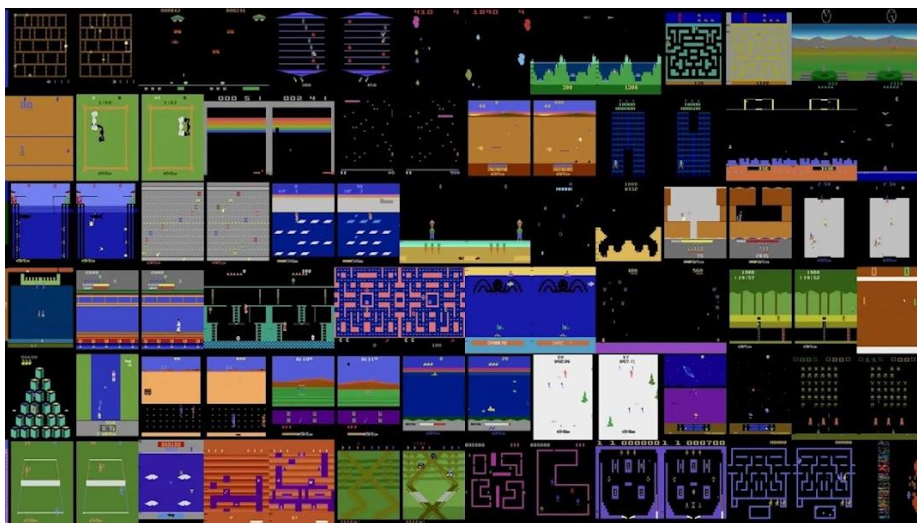
- Συλλογή εμπειριών (data collection)
- Εκπαίδευση (training)
- Ενημέρωση παραμέτρων (update)
- Αξιολόγηση (evaluation)
- Αποθήκευση ή φόρτωση μοντέλων (save/load)

Τροποποιώντας κατάλληλα τις διάφορες παραμέτρους των παραπάνω στοιχείων, το DI-engine μπορεί να υποστηρίξει αλγορίθμους on-policy και off-policy ή να δημιουργήσει κατάλληλη μνήμη buffer όπως replay buffer. Ακόμα δίνεται η δυνατότητα ενσωμάτωσης μηχανισμών εξερεύνησης (exploration module) του περιβάλλοντος του πράκτορα, ταυτόχρονη εκπαίδευση πολλαπλών πρακτόρων (multi-agent) και πολλές άλλες λειτουργίες [20].

Περιβάλλοντα εκπαίδευσης

Το DI-engine υποστηρίζει ένα μεγάλο σύνολο από περιβάλλοντα εκπαίδευσης διαφορετικού τύπου, επιτρέποντας την αξιολόγηση αλγορίθμων σε διακριτά και συνεχή περιβάλλοντα αλλά και σε περιβάλλοντα διαβαθμισμένης δυσκολίας. Εμπεριέχει τόσο απλά περιβάλλοντα ελέγχου και κινηματικής όσο και σύνθετα παιχνίδια στρατηγικής [20]. Μερικά παραδείγματα αποτελούν τα εξής:

- Atari
- MuJoCo
- CartPole
- Pendulum
- LunarLander
- Gfootball
- PettingZoo
- SMAC



Σχήμα 4.2: Περιβάλλοντα εκπαίδευσης Atari.

Αλγόριθμοι

Το framework περιλαμβάνει πληθώρα αλγορίθμων και επεκτάσεων τους, μέσω της βιβλιοθήκης DI-zoo που παρέχει έτοιμες υλοποιήσεις και παραδείγματα αξιολόγησης (benchmarks). Υποστηρίζονται σχεδόν όλες οι κατηγορίες αλγορίθμων ενισχυτικής μάθησης, συμπεριλαμβανομένων των τριών βασικών που έχουν προαναφερθεί, δίνοντας την ευκαιρία στον χρήστη να πειραματιστεί επιλέγοντας τον αλγόριθμο που επιθυμεί και συνδυάζοντας τον με το κατάλληλο περιβάλλον εκπαίδευσης. Συγκεκριμένα παρέχονται αλγόριθμοι Q-learning, Actor-Critic, Exploration, Imitation Learning, Offline RL, Memory Based, Multi-Agent, Model-Based και Generalization [20].

Πίνακας 4.1: Πίνακας αλγορίθμων που υποστηρίζει το DI-engine.

Κατηγορία Αλγορίθμων	Ενδεικτικοί Αλγόριθμοι
Q-learning	DQN,C51,QRDQN,Rainbow,IQN,FQF,SQL,SQLN,MDQN,Averaged-DQN
Actor-Critic	A2C,PPO,ACER,IMPALA,PPG,DDPG,D4PG,TD3,SAC
Exploration	RND,HER,ICM
Imitation-Learning	DQfD,SQLIL,GAIL,TREX,R2D3
Offline RL	CQL,TD3BC,EDAC,DT,QGPO,Diffuser
Memory Based	R2D2,GTrXL
Multi-Agent	QMIX,COMA,WQMIX,QTRAN,CollaQ,ATOC
Model-Based	MBPO,VPN
Generalization	PLR

4.1.2 PyTorch

Η PyTorch συνιστά ένα πλαίσιο βαθιάς μάθησης (deep learning framework) ανοιχτού κώδικα, γραμμένο σε Python, με στόχο την ανάπτυξη και εκπαίδευση νευρωνικών δικτύων. Παρέχει ένα εύχρηστο, ευέλικτο και δυναμικό περιβάλλον που υποστηρίζει τόσο την ακαδημαϊκή έρευνα όσο και την βιομηχανία, καθώς επιτρέπει την διεξαγωγή πειραμάτων χωρίς μεγάλη πολυπλοκότητα από την πλευρά του χρήστη. Ακολουθεί πιστά τις προγραμματιστικές αρχές της γλώσσας Python, με την εκτέλεση του κώδικα ανά γραμμή, διευκολύνοντας την διαδικασία της αποσφαλμάτωσης (debugging), ενώ παράλληλα υποστηρίζει μεθόδους επιτάχυνσης υλικού (hardware acceleration) για την πλήρη αξιοποίηση της υπολογιστικής ισχύος που διαθέτουν οι κάρτες γραφικών (GPU) [21].

Το DI-engine χρησιμοποιεί την PyTorch ως βασικό στοιχείο για την υλοποίηση και εκπαίδευση των μοντέλων ενισχυτικής μάθησης. Συγκεκριμένα αξιοποιείται στην δημιουργία των μοντέλων, διαμορφώνοντας τα με τα κατάλληλα επίπεδα (layers), συναρτήσεις ενεργοποίησης και γενικότερα συμβάλλοντας στο χτίσιμο της αρχιτεκτονικής του κάθε μοντέλου. Κατά συνέπεια τα πειράματα που εκτελέστηκαν με το DI-engine επωφελούνται των πλεονεκτημάτων που παρέχει η PyTorch, όπως η ταχεία ανάπτυξη και εκτέλεση των αλγορίθμων με χρήση κάρτας γραφικών [20].

4.1.3 NumPy

Η βιβλιοθήκη NumPy αποτελεί την κυρίαρχη βιβλιοθήκη προγραμματισμού πινάκων (array programming) της γλώσσας προγραμματισμού Python. Ο προγραμματισμός πινάκων που υλοποιεί η NumPy έχει κομβικό ρόλο σε διαδικασίες ανάλυσης δεδομένων από πολλαπλούς τομείς, ενώ υποστηρίζει πολύ υψηλές διαστάσεις πινάκων και μετασχηματισμό αυτών σε γρήγορες ταχύτητες. Πιο συγκεκριμένα περιλαμβάνει λειτουργίες γραμμικής άλγεβρας και στατιστικής για την αποτελεσματική διαχείριση μεγάλων δεδομένων διευκολύνοντας ερευνητές αλλά και απλούς χρήστες [22].

Η NumPy χρησιμοποιείται κατά κόρον στους αριθμητικούς υπολογισμούς που απαιτούνται στην εκπαίδευση των μοντέλων του DI-engine και στον χειρισμό των δεδομένων. Η αποθήκευση των παρατηρήσεων (observations), ενεργειών, ανταμοιβών και μελλοντικών καταστάσεων επιτελείται με την χρήση της NumPy, ενώ παράλληλα αναλαμβάνει στοχαστικές διεργασίες με παραγωγή τυχαίων αριθμών, μετασχηματισμούς και υπολογισμούς σε πίνακες, όπως υπολογισμός μέσου όρου. Αξιοσημείωτη είναι η συμβατότητα που έχει με την PyTorch, επιτρέποντας μετασχηματισμούς από NumPy πίνακα, δηλαδή ndarray, σε Tensor και αντιστρόφως [20].

4.2 Περιβάλλοντα εκπαίδευσης

Για την αξιολόγηση ενός πράκτορα ενισχυτικής μάθησης απαιτείται η ύπαρξη ενός περιβάλλοντος μέσα στο οποίο ο πράκτορας μπορεί να εκτελέσει ενέργειες, να λάβει ανταμοιβές, είτε θετικές είτε αρνητικές, και να μεταβεί σε επόμενες καταστάσεις με κύριο σκοπό την μεγιστοποίηση των ανταμοιβών που λαμβάνει. Η OpenAI έχει δημιουργήσει ένα ερευνητικό εργαλείο (toolkit/API) ανοιχτού κώδικα, το οποίο παρέχει μία ευρεία συλλογή από περιβάλλοντα αξιολόγησης και ονομάζεται Gym (Gymnasium). Σκοπός του είναι να διευκολύνει την σύγκριση και αξιολόγηση αλγορίθμων ενισχυτικής μάθησης μέσα από τα προσφερόμενα περιβάλλοντα που λειτουργούν ως benchmarks για την αξιολόγηση της επίδοσης των πρακτόρων. Περιλαμβάνει περιβάλλοντα διαφόρων κατηγοριών όπως classic control, παιχνιδιών Atari και επιτραπέζιων παιχνιδιών (πχ. Go), ενώ συνεχώς γίνονται νέες προσθήκες [23].

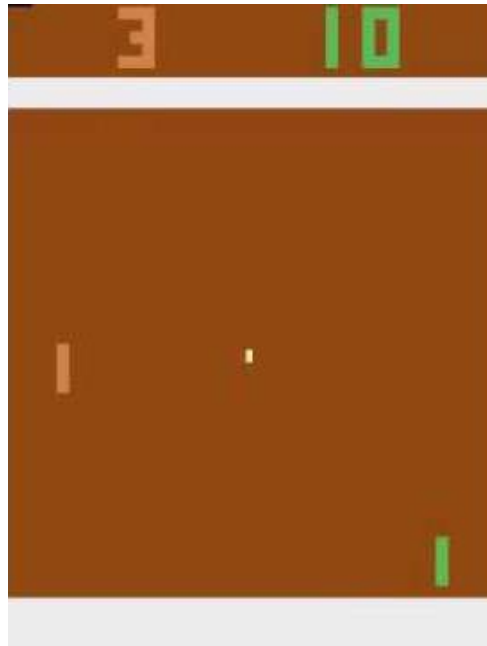
Το Gym παρέχει μία κοινή διεπαφή η οποία επιτρέπει στους χρήστες να χρησιμοποιήσουν τον ίδιο κώδικα αλγορίθμου σε διαφορετικά περιβάλλοντα. Η εναλλαγή σε κάποιο άλλο περιβάλλον είναι επίσης απλή, αφού έχοντας εγκατεστημένες τις απαραίτητες βιβλιοθήκες, απαιτείται μία γραμμή κώδικα για την επιλογή του, για παράδειγμα: `env = gym.make('CartPole-v1')`. Σε περίπτωση που ο χρήστης θελήσει να τροποποιήσει την λογική του περιβάλλοντος, έρχεται αντιμέτωπος με την κλάση του και ορισμένες βασικές συναρτήσεις που υλοποιούνται σε αυτήν, όπως την `__init__`, `reset`, `step` και `render` [23].

Αρχικά, η συνάρτηση `__init__` καλείται κατά την δημιουργία του περιβάλλοντος και ορίζει το όνομα του περιβάλλοντος που θα δημιουργηθεί, τον χώρο των παρατηρήσεων (observation space), τον χώρο των ενεργειών (action space) και προαιρετικά χώρο ανταμοιβών (reward space) και άλλα εργαλεία όπως ο ορισμός τυχαίας κατανομής (seed). Στην αρχή κάθε επεισοδίου καλείται η συνάρτηση `reset`. Σκοπός της είναι να δειγματοληπτήσει την αρχική κατάσταση του περιβάλλοντος και να επιστρέψει την πρώτη παρατήρηση. Σε κάθε βήμα ενός επεισοδίου καλείται η συνάρτηση `step`. Λαμβάνει ως είσοδο την επιλεγόμενη ενέργεια του πράκτορα και την εφαρμόζει στο περιβάλλον, μεταβαίνοντας στην επόμενη κατάσταση και επιστρέφοντας την επόμενη παρατήρηση, την ανταμοιβή της ενέργειας, μία λογική (boolean) μεταβλητή `done` που ορίζει αν ο πράκτορας έφτασε σε τερματική κατάσταση, και τέλος ένα λεξικό `info` με πρόσθετες πληροφορίες (πχ. debugging). Καταληκτικά, η συνάρτηση `render` υλοποιεί την οπτική αναπαράσταση του περιβάλλοντος, δηλαδή απεικονίζει την κατάσταση του περιβάλλοντος όσο εκπαιδεύεται με μορφή γραφικών για παράδειγμα, προκειμένου να βοηθήσει τον χρήστη να κατανοήσει καλύτερα πως γίνεται η εκπαίδευση και αν παρουσιάζονται προβλήματα [23].

PongNoFrameskip-v4

Το Pong είναι το πρώτο βιντεοπαιχνίδι στην ιστορία, και πρόκειται για ένα απλό παιχνίδι πινγκ πονγκ. Το περιβάλλον που παρέχει το Gym προσομοιώνει μία έκδοση του παιχνιδιού στην κονσόλα Atari. Στο περιβάλλον ο πράκτορας ελέγχει την δεξιά ρακέτα που απεικονίζεται στο σχήμα 4.3 και

προσπαθεί να κατευθύνει την μπάλα προς την πλευρά του αντιπάλου και να "σκοράρει" ενώ παράλληλα προστατεύει το δικό του τέρμα, αποκρούοντας την μπάλα [24].



Σχήμα 4.3: Το περιβάλλον Atari Pong του Gym.

Οι ενέργειες που μπορεί να εκτελέσει ο πράκτορας αποτελούν τον χώρο ενεργειών του περιβάλλοντος. Συγκεκριμένα, έχει την επιλογή μέσα από 6 διακριτές ενέργειες σε κάθε βήμα:

0. NOOP: Καμία ενέργεια
1. FIRE: Πυροδότηση (καμία εφαρμογή στο Pong)
2. RIGHT: Κίνηση προς τα δεξιά (πάνω)
3. LEFT: Κίνηση προς τα αριστερά (κάτω)
4. RIGHTFIRE: Κίνηση προς τα δεξιά (πάνω) και πυροδότηση (ίδια με την 2)
5. LEFTFIRE: Κίνηση προς τα αριστερά (κάτω) και πυροδότηση (ίδια με την 3)

Ουσιαστικά ο πράκτορας είτε θα επιλέξει να κινηθεί προς τα πάνω είτε προς τα κάτω, είτε θα μείνει ακίνητος [24].

Ο χώρος των παρατηρήσεων που χρησιμοποιήθηκε στα πειράματα αντιστοιχεί σε μία εικόνα RGB, διαστάσεων 210x160 πίξελ και 3 κανάλια χρώματος, δηλαδή `Box(0, 255, (210, 160, 3), np.uint8)`. Ο τρόπος που ο πράκτορας δέχεται ανταμοιβές είναι με την μορφή πόντων που κερδίζει κάθε φορά που σκοράρει. Συνεπώς οι ανταμοιβές που δέχεται κυμαίνονται από -21 σε 21, καθώς ο πρώτος παίκτης που αποκτήσει 21 πόντους κερδίζει το παιχνίδι [24].

SpaceInvadersNoFrameskip-v4

Το SpaceInvaders αποτελεί ένα κλασικό βιντεοπαιχνίδι της κονσόλας Atari, στο οποίο ο παίκτης αντιμετωπίζει κύματα εξωγήινων. Συγκριμένα ελέγχει ένα κανόνι laser, όπως φαίνεται στο κάτω μέρος του σχήματος 4.4, και προσπαθεί να εξαλείψει τους εξωγήινους που κατεβαίνουν συνεχώς προς το μέρος του. Οι επιτρεπόμενες ενέργειες που έχει ο πράκτορας είναι ίδιες με αυτές του Pong, με την διαφορά ότι πλέον η ενέργεια 1: FIRE είναι κρίσιμης σημασίας για το παιχνίδι, ενώ ο χώρος των παρατηρήσεων είναι επίσης όμοιος με αυτόν του περιβάλλοντος Pong. Ο πράκτορας λαμβάνει ανταμοιβές κάθε φορά που καταστρέφει έναν εξωγήινο και το παιχνίδι τερματίζεται είτε όταν φτάσουν οι εξωγήινοι στο κάτω μέρος της οθόνης, είτε όταν ο πράκτορας δεχθεί εχθρικά πυρά πάνω

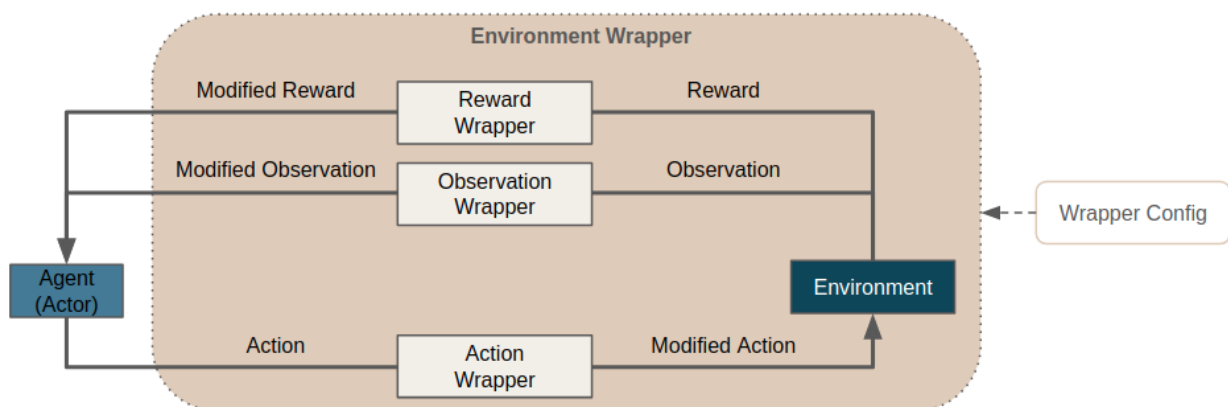
από δύο φορές. Συνεπώς στόχος του είναι η εξάλειψη όσο το δυνατόν περισσότερων εξωγήινων, όσο παράλληλα αποφεύγει εχθρικά πυρά [24].



Σχήμα 4.4: Το περιβάλλον Atari SpaceInvaders του Gym.

Προκειμένου να χρησιμοποιηθούν τα περιβάλλοντα μαζί με το DI-engine απαιτείται η εφαρμογή κάποιων συναρτήσεων wrapper που θα τα διαμορφώσουν κατάλληλα και θα ενισχύσουν την απόδοσή τους. Για το λόγο αυτό, επιλέχθηκαν οι εκδόσεις «NoFrameskip-v4» που παύουν την αυτόματη αγνόηση των καρτέ (frames) και επιτρέπουν τον πλήρη έλεγχο μέσα από τα wrappers. Πιο συγκεκριμένα τα wrappers που δέχονται τα περιβάλλοντα είναι τα εξής: MaxAndSkipWrapper, EpisodicLifeWrapper, WarpFrameWrapper, ScaledFloatFrameWrapper, FrameStackWrapper [20].

Αρχικά, χρησιμοποιείται ο MaxAndSkipWrapper, με τον οποίο εκτελείται μία ενέργεια για skip διαδοχικά καρτέ (skip=4) και επιστρέφει το φωτεινότερο από αυτά καρτέ προκειμένου να μειώσει τον θόρυβο που δημιουργείται στην εικόνα και να επιταχύνει την προσομοίωση. Στην συνέχεια, εφαρμόζεται ο EpisodicLifeWrapper, ο οποίος δίνει τέλος στο επεισόδιο κάθε φορά που για παράδειγμα δέχεται ένα γκολ ο πράκτορας ή γενικότερα χάνει μία ζωή, χωρίς όμως να κάνει επαναφορά κατάστασης, βοηθώντας με αυτό τον τρόπο τον πράκτορα να εκπαιδευτεί και από την επιβάρυνση στο σκορ και όχι μόνο στο τελικό αποτέλεσμα. Ακολουθεί ο WarpFrameWrapper που αναπροσαρμόζει το μέγεθος των εικόνων-παρατηρήσεων σε 84x84 πίξελ, έτσι ώστε να αφαιρέσει ασήμαντες πληροφορίες που αυξάνουν την πολυπλοκότητα των νευρωνικών δικτύων. Τέλος, αξιοποιείται ο ScaledFloatFrameWrapper που κανονικοποιεί τις παρατηρήσεις σε 0-1 για να βελτιώσει την σταθερότητα της εκπαίδευσης, και ο FrameStackWrapper που ομαδοποιεί ορισμένα καρτέ σε ένα ώστε να επιτραπεί στον πράκτορα να έχει μια αίσθηση της δυναμικής και της κίνησης, παρά ενός στατικού καρτέ [20].



Σχήμα 4.5: Εφαρμογή προσαρμοσμένου wrapper σε περιβάλλον ενισχυτικής μάθησης.

4.3 Αλυσίδα διεργασιών (Pipeline)

Η εκπαίδευση των αλγορίθμων γίνεται μέσω της συνάρτησης `serial_pipeline` που προσφέρει το DI-engine. Η `serial_pipeline` είναι η βασική μέθοδος που χρησιμοποιείται για την εκπαίδευση αλγορίθμων εκτός πολιτικής (*off-policy*) και αποσκοπεί στην ενορχήστρωση της διαδικασίας συλλογής δεδομένων, εκπαίδευσης ενός μοντέλου ενισχυτικής μάθησης και τελικά αξιολόγησής του, επιστρέφοντας την εκπαιδευμένη πολιτική. Η αλυσίδα διεργασιών απαρτίζεται από τα εξής στάδια: σύνταξη και φόρτωση των ρυθμίσεων (`config`), δημιουργία περιβαλλόντων και ορισμός τυχαίου «σπόρου» (`random seed`), δημιουργία πολιτικής, κατασκευή των εργατών (`workers`) που εκτελούν σημαντικές λειτουργίες της αλυσίδας, κύριος βρόχος εκπαίδευσης (`main loop`), αποθήκευση αποτελεσμάτων και επιστροφή πολιτικής [20].

Config

Το πρώτο στάδιο της αλυσίδας διεργασιών είναι ο ορισμός των παραμέτρων που δέχεται η `serial_pipeline`. Τα ορίσματα που λαμβάνει είναι δύο λεξικά ως `tuple`, `main_config` και `create_config`, επιθυμητό `seed` και κάποια προαιρετικά ορίσματα όπως εισαγωγή προσαρμοσμένου μοντέλου ή μέγιστος αριθμός επαναλήψεων. Το `main_config` είναι ένα λεξικό που αποτελείται από άλλα λεξικά τα οποία περιλαμβάνουν όλες τις παραμέτρους εκπαίδευσης που αφορούν για παράδειγμα δημιουργία των περιβαλλόντων, της πολιτικής, του μοντέλου ή των επιμέρους `worker` [20]. Συγκεκριμένα, αναφέρονται τα εξής λεξικά που περιέχονται καθώς και ορισμένες παραμέτρους που αυτά εμπεριέχουν:

- `env`: ρυθμίζει τον αριθμό των περιβαλλόντων που δημιουργούνται για συλλογή εμπειριών και αξιολόγηση, την συνθήκη τερματισμού της εκπαίδευσης και άλλες παραμέτρους όπως το `frame_stack`.
- `policy`: καθορίζει αν θα αξιοποιηθούν οι δυνατότητες της κάρτας γραφικών, αν θα χρησιμοποιηθεί η τεχνική της προτεραιότητας στις εμπειρίες του πράκτορα και τα περιλαμβάνει τα λεξικά `model`, `learn`, `collect`, `eval` και `other`.
 - `model`: ορίζει το σχήμα των παρατηρήσεων, το πλήθος των ενεργειών, τις διαστάσεις των κρυμμένων επιπέδων MLP που ακολουθούν συνελικτικά επίπεδα στο νευρωνικό δίκτυο και άλλες παραμέτρους ανάλογα με τον αλγόριθμο που εκπαιδεύεται.
 - `learn`: ρυθμίσεις που αφορούν τον `learner`, όπως ρυθμός εκπαίδευσης (`learning rate`), συχνότητα ενημέρωσης του νευρωνικού στόχου (`target network`) και `batch size`.
 - `collect`: ρυθμίσεις που αφορούν τον `collector`, και καθορίζουν το πώς συλλέγονται δεδομένα από το περιβάλλον, όπως για παράδειγμα η παράμετρος `n_sample` που ορίζει πόσα δείγματα συλλέγονται ανά βήμα του `collector`.
 - `eval`: ρύθμιση συχνότητας αξιολόγησης του `evaluator` και άλλες σχετικές παράμετροι.
 - `other`: λοιπές ρυθμίσεις βάσει του αλγορίθμου εκπαίδευσης.
 - `eps`: αφορά την πολιτική εξερεύνησης του πράκτορα με μεθόδους όπως η `ε-greedy` που επιλέγει την βέλτιστη ενέργεια «άπληστα» με πιθανότητα $1-\epsilon$.
 - `replay_buffer`: ορισμός μεγέθους του `buffer` και μεταβλητών `alpha`, `beta`, `anneal_step`.

Το `create_config` ορίζει τον τύπο του περιβάλλοντος που θα δημιουργηθεί και την βιβλιοθήκη στην οποία πηγάζει, τον διαχειριστή των περιβαλλόντων, όπως `subprocess`, στον οποίο κάθε αντίγραφο του περιβάλλοντος εκτελείται σαν ξεχωριστή διαδικασία, αλλά και τον τύπο της πολιτικής, δηλαδή τον αλγόριθμο προς εκπαίδευση. Προτού δοθούν ως ορίσματα στην `serial_pipeline`, μετατρέπονται σε EasyDict για την ευκολότερη πρόσβαση στα πεδία τους [20].

Η `serial_pipeline` αντιγράφει σε βάθος την πλειάδα των λεξικών και τα αποθηκεύει στην μεταβλητή `cfg` και `create_cfg` αντίστοιχα. Στην συνέχεια τροποποιείται ο τύπος της πολιτικής στον οποίο προστίθεται η κατάληξη `"_command"`, προκειμένου να επιτρέψει την δυνατότητα `command mode` του `commander` που θα επεξηγηθεί παρακάτω. Τέλος, με την συνάρτηση `compile_config` γίνεται ένας συγκερασμός των `cfg` και `create_cfg`, που αποθηκεύεται στην μεταβλητή `cfg` αλλά και κάποιων επιπρόσθετων ρυθμίσεων, όπως το αν θα γίνει αυτόματη συμπλήρωση ελλειπών πεδίων, αν θα αποθηκευτεί το `config` σε κάποιο αρχείο ή αν πρόκειται για προεκπαιδευμένο μοντέλο στο οποίο θα συνεχίσει η εκπαίδευση [20].

```
cfg = compile_config(
    cfg,
    seed=seed,
    env=env_fn,
    auto=True,
    create_cfg=create_cfg,
    save_cfg=True,
    renew_dir=not cfg.policy.learn.get('resume_training', False)
)
```

Περιβάλλοντα και Πολιτική

Στο στάδιο αυτό δημιουργούνται τα κύρια συστατικά της αλυσίδας διεργασιών, τα περιβάλλοντα και η πολιτική. Αρχικά, αν δεν έχουν δοθεί ρυθμίσεις για το περιβάλλον καλείται η συνάρτηση `get_vec_env_setting` η οποία επιστρέφει τα `env_fn`, `collector_env_cfg` και `evaluator_env_cfg` που συμβάλλουν στην κατάλληλη ρύθμιση των περιβαλλόντων συλλογής και αξιολόγησης. Τα περιβάλλοντα συλλογής είναι αυτά στα οποία ο πράκτορας ενεργεί και εκπαιδεύεται, ενώ αντίθετα τα περιβάλλοντα αξιολόγησης χρησιμοποιούνται μόνο κατά το στάδιο της αξιολόγησης του. Η διαδικασία συνεχίζεται με την δημιουργία του διαχειριστή περιβάλλοντος, τόσο για τον `collector` όσο και για τον `evaluator`, που μέσω παραμέτρων του `cfg` καθορίζει τον αριθμό των περιβαλλόντων που θα δημιουργηθούν και άλλες πρόσθετες ρυθμίσεις και παράλληλα εφαρμόζεται το `seed` για την επίτευξη της αναπαραγωγιμότητας. Το στάδιο αυτό καταλήγει στην δημιουργία της πολιτικής χρησιμοποιώντας τις παραμέτρους που έχουν δοθεί για τον τύπο της και το μοντέλο εκπαίδευσης, ενώ δηλώνονται οι καταστάσεις λειτουργίας (`modes`) που θέλει ο χρήστης να ενεργοποιήσει, όπως `learn`, `collect`, `eval`, `command`, οι οποίες ενεργοποιούν τους αντίστοιχους `worker` στην συνέχεια [20].

```
# Create main components: env, policy
if env_setting is None:
    env_fn, collector_env_cfg, evaluator_env_cfg = get_vec_env_setting(cfg.env)
else:
    env_fn, collector_env_cfg, evaluator_env_cfg = env_setting
collector_env = create_env_manager(cfg.env.manager, [partial(env_fn, cfg=c) for c in collector_env_cfg])
evaluator_env = create_env_manager(cfg.env.manager, [partial(env_fn, cfg=c) for c in evaluator_env_cfg])
collector_env.seed(cfg.seed, dynamic_seed=dynamic_seed)
evaluator_env.seed(cfg.seed, dynamic_seed=False)
set_pkg_seed(cfg.seed, use_cuda=cfg.policy.cuda)
policy = create_policy(cfg.policy, model=model, enable_field=['learn', 'collect', 'eval', 'command'])
```

Workers

Το στάδιο αυτό αφορά την δημιουργία των επιμέρους συστατικών που εκτελούν κύριες λειτουργίες της αλυσίδας διεργασιών, όπως μάθηση, συλλογή, αξιολόγηση και αποθήκευση δεδομένων αλλά και τον συντονισμό όλων αυτών των συστατικών προκειμένου να υλοποιηθεί ο κύκλος εκπαίδευσης και

να καταλήξει σε εκπαιδευμένη πολιτική. Οι λειτουργίες αυτές υποστηρίζονται μέσω των συστατικών learner, collector, evaluator, replay_buffer και commander [20].

Μέσω του learner επιτελείται η διαδικασία της εκπαίδευσης της πολιτικής. Αποτελεί αντικείμενο της κλάσης BaseLearner, για την δημιουργία του οποίου δίνονται κάποια ορίσματα, όπως σχετικές ρυθμίσεις από το cfg, την κατάσταση λειτουργίας learn της πολιτικής, που είναι μία διεπαφή ή οποία περιέχει βασικές συναρτήσεις, τύπου forward_learn, τον τρόπο αποθήκευσης των αποτελεσμάτων και το όνομα του παραγόμενου αρχείου [20].

```
learner = BaseLearner(cfg.policy.learn.learner, policy.learn_mode, tb_logger, exp_name=cfg.exp_name)
```

Ο collector έχει τον ρόλο της συλλογής εμπειριών από τα παράλληλα περιβάλλοντα, στα οποία εφαρμόζεται η πολιτική της κατάστασης λειτουργίας collect, και της αποθήκευσης τους στον replay buffer προς εκπαίδευση μέσω του learner. Οι εμπειρίες που παράγονται έχουν την μορφή (obs, action, reward, next_obs, done, info). Κατά την διάρκεια της εκπαίδευσης ο evaluator είναι υπεύθυνος για την αξιολόγηση της τρέχουσας απόδοσης της πολιτικής. Συγκεκριμένα, εκτελεί την τρέχουσα πολιτική με ντετερμινιστική συμπεριφορά, χωρίς εξερεύνηση, και ενημερώνει για το αν πρέπει να σταματήσει η εκπαίδευση με βάση το αν ο μέσος όρος των ανταμοιβών που επιστρέφει η πολιτική είναι μεγαλύτερος ή ίσος του stop_value που έχει οριστεί στο config. Η δημιουργία του collector γίνεται μέσω της συνάρτησης create_serial_collector, ενώ ο evaluator δημιουργείται από την create_serial_evaluator. Οι δύο συναρτήσεις δέχονται παραμέτρους αντίστοιχες με αυτές του learner αλλά που σχετίζονται με την λειτουργία του collector και evaluator αντίστοιχα [20].

```
collector = create_serial_collector( evaluator = create_serial_evaluator(
    cfg.policy.collect.collector,      cfg.policy.eval.evaluator,
    env=collector_env,                 env=evaluator_env,
    policy=policy.collect_mode,        policy=policy.eval_mode,
    tb_logger=tb_logger,               tb_logger=tb_logger,
    exp_name=cfg.exp_name               exp_name=cfg.exp_name
)                                     )
```

Η αποθήκευση των εμπειριών που παράγει ο collector γίνεται στον replay buffer, ο οποίος επιστρέφει δείγματα σε δεσμίδες (batches) στον learner προκειμένου να εκπαιδευτεί. Υπάρχουν αρκετές παραλλαγές στην διαδικασία αυτή, από τις οποίες προκύπτουν διάφορα είδη replay buffer. Ο Prioritized Replay Buffer (PER) είναι από τις πιο δημοφιλείς επιλογές buffer και η διαφορά του από τον κλασικό buffer έγκειται στην δειγματοληψία των εμπειριών. Πιο συγκεκριμένα, σταθμίζει τις εμπειρίες, δίνοντας προτεραιότητα σε αυτές με την μεγαλύτερη μαθησιακή αξία. Η προτεραιότητα αυτή ρυθμίζεται μέσω της παραμέτρου alpha που κυμαίνεται από 0 έως 1. Το bias που προκαλείται από την εισαγωγή της προτεραιότητας αντισταθμίζεται με την παράμετρο beta, η οποία αυξάνεται σταδιακά μέσω της anneal_step. Στην αλυσίδα διεργασιών, ο replay buffer ορίζεται μέσω της συνάρτησης create_buffer, η οποία λαμβάνοντας τις αντίστοιχες ρυθμίσεις του cfg δημιουργεί τον κατάλληλο replay buffer [20].

```
replay_buffer = create_buffer(cfg.policy.other.replay_buffer, tb_logger=tb_logger, exp_name=cfg.exp_name)
```

Τον συντονισμό όλων των επιμέρους συστατικών που αναφέρονται στο στάδιο αυτό, αναλαμβάνει ο commander, ένα αντικείμενο της κλάσης BaseSerialCommander. Η κύρια λειτουργία του είναι να ελέγχει πότε πρέπει να συλλεχθούν νέες εμπειρίες από το περιβάλλον μέσω του collector, πότε να αποθηκευτούν στον replay buffer, πότε να ξεκινήσει η διαδικασία της εκπαίδευσης του learner και πότε να γίνει η αξιολόγηση του evaluator, μεταφέροντας παραμέτρους από το ένα συστατικό προς

το άλλο. Κατά συνέπεια αποτελεί τον εντοπιστή της αλυσίδας διεργασιών και δημιουργείται λαμβάνοντας ως ορίσματα τα συστατικά που συντονίζει και ρυθμίσεις του `cfg` και της πολιτικής [20].

```
commander = BaseSerialCommander(  
    cfg.policy.other.commander, learner, collector, evaluator, replay_buffer, policy.command_mode  
)
```

Κύκλος εκπαίδευσης

Προτού ξεκινήσει η εκπαίδευση, αν έχει οριστεί η παράμετρος `random_collect_size` στο `config`, γίνεται συλλογή τυχαίων δεδομένων μέσω του `collector` και αποθηκεύονται στον `replay buffer`, προκειμένου να αντιμετωπιστούν φαινόμενα πρόωρης υπερπροσαρμογής (*early overfitting*). Στην συνέχεια ακολουθεί ο βρόχος εκπαίδευσης που ξεκινά με τον `commander` να επιστρέφει παραμέτρους για την συλλογή δεδομένων `collect_kwargs`. Ακολουθεί η αξιολόγηση της πολιτικής του `learner` (αν χρειάζεται σε αυτό το βήμα) και συνεπώς η συνθήκη τερματισμού, η οποία αν είναι αληθής τερματίζει τον βρόχο της εκπαίδευσης. Έπεται η συλλογή νέων δεδομένων μέσω του `collector` που χρησιμοποιεί τα `collect_kwargs` που παρήγαγε ο `commander`, καθώς και η προσθήκη τους στον `replay buffer` [20].

```
if cfg.policy.get('random_collect_size', 0) > 0:  
    random_collect(cfg.policy, policy, collector, collector_env, commander, replay_buffer)  
while True:  
    collect_kwargs = commander.step()  
    # Evaluate policy performance  
    if evaluator.should_eval(learner.train_iter):  
        stop, eval_info = evaluator.eval(learner.save_checkpoint, learner.train_iter, collector.envstep)  
        if stop:  
            break  
    # Collect data by default config n_sample/n_episode  
    new_data = collector.collect(train_iter=learner.train_iter, policy_kwargs=collect_kwargs)  
    replay_buffer.push(new_data, cur_collector_envstep=collector.envstep)
```

Στο σημείο αυτό ορίζεται ο βασικός βρόχος, στον οποίο ο `learner` εκπαιδεύεται πολλαπλές φορές, όσες έχουν οριστεί με την παράμετρο `update_per_collect`. Συγκεκριμένα, σε κάθε επανάληψη ο πράκτορας δειγματοληπτεί μία δεσμίδα εμπειριών από τον `replay buffer`, εφόσον υπάρχει αποθηκευμένο το ζητούμενο πλήθος εμπειριών, και εκπαιδεύεται μέσω της `learner.train`. Αν χρησιμοποιείται PER ενημερώνονται οι προτεραιότητες των δειγμάτων στο `replay buffer`. Σε περίπτωση που ο κύκλος εκπαίδευσης δεν έχει τερματίσει μέσω του `evaluator`, τερματίζει μετά το πέρας του μέγιστου πλήθους βημάτων στο περιβάλλον ή του μέγιστου πλήθους επαναλήψεων που προαιρετικά ορίζονται κατά την κλήση του `serial_pipeline` [20].

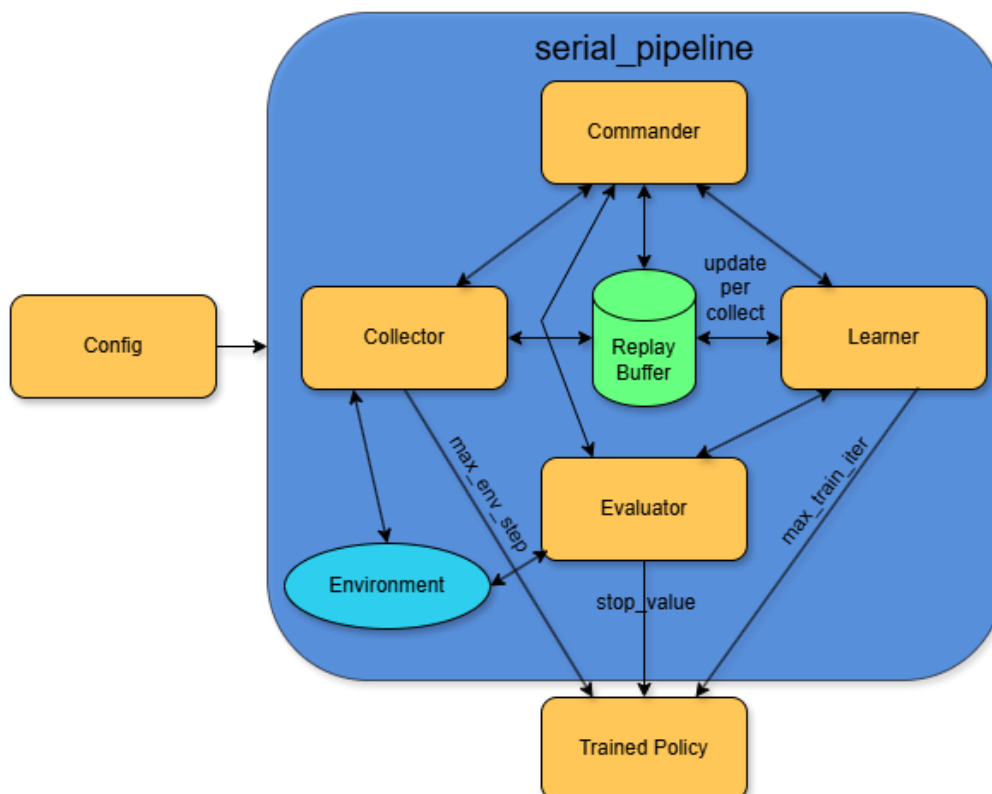
```

for i in range(cfg.policy.learn.update_per_collect):
    # Learner will train ``update_per_collect`` times in one iteration.
    train_data = replay_buffer.sample(learner.policy.get_attribute('batch_size'), learner.train_iter)
    if train_data is None:
        # It is possible that replay buffer's data count is too few to train ``update_per_collect`` times
        logging.warning(
            "Replay buffer's data can only train for {} steps. ".format(i) +
            "You can modify data collect config, e.g. increasing n_sample, n_episode."
        )
        break
    learner.train(train_data, collector.envstep)
    if learner.policy.get_attribute('priority'):
        replay_buffer.update(learner.priority_info)
if collector.envstep >= max_env_step or learner.train_iter >= max_train_iter:
    break

```

Αποθήκευση αποτελεσμάτων

Την επιστροφή της εκπαιδευμένης πολιτικής από την `serial_pipeline` συνοδεύουν αρχεία (logs) που καταγράφουν τα αποτελέσματα της εκπαίδευσης. Αναλυτικότερα αποθηκεύονται πληροφορίες που αφορούν την μέση ανταμοιβή που έλαβε ο πράκτορας, τα βήματα που εκτέλεσε στο περιβάλλον, τον χρόνο που χρειάστηκε και άλλες παρόμοιες μετρικές. Κατά την διάρκεια της αλυσίδας διεργασιών έχουν αποθηκευτεί ενδιάμεσες καταστάσεις (checkpoints), από τα οποία είναι δυνατή η συνέχεια της εκπαίδευσης. Για την οπτικοποίηση των αποτελεσμάτων σε γραφήματα αξιοποιείται από το DL-engine, το TensorBoard. Το TensorBoard παρέχει ένα διαδικτυακό διαδραστικό περιβάλλον για την παρακολούθηση και ανάλυση πειραμάτων μηχανικής μάθησης, επιτρέποντας τον έλεγχο των επιδόσεων της πολιτικής, απώλειες εκπαίδευσης και πολλά άλλα [20].



Σχήμα 4.6: Αλυσίδα διεργασιών `serial_pipeline`.

4.4 Πειράματα και Αποτελέσματα

Στην παρούσα ενότητα περιγράφονται τα πειράματα που διεξήχθησαν, δηλαδή οι αλγόριθμοι που εκπαιδεύτηκαν μέσω του framework DI-engine. Αρχικά, παρουσιάζονται συνοπτικά οι αλγόριθμοι και οι υπερ-παράμετροι που επιλέχθηκαν για τον καθένα, ενώ διασαφηνίζεται η σημασία τους στην εκπαίδευση του. Στην συνέχεια φαίνονται τα αποτελέσματα της εκπαίδευσης μέσα από σχετικά γραφήματα και τέλος γίνεται μελέτη και εξαγωγή συμπερασμάτων από τα πειράματα.

4.4.1 Αλγόριθμοι και Υπερ-παράμετροι

Οι αλγόριθμοι στους οποίους έγιναν πειράματα, όπως προαναφέρθηκε εκπαιδεύτηκαν μέσω του DI-engine και εφαρμόστηκαν στα περιβάλλοντα PongNoFrameskip-v4 και SpaceInvadersNoFrameskip-v4 της OpenAI Gym. Στην προηγούμενη ενότητα εξηγήθηκε ενδελεχώς ο κύκλος διεργασιών για την εκπαίδευση με πρώτο βήμα την είσοδο που δέχεται το `serial_pipeline`. Το `main_config` που περιλαμβάνεται στα ορίσματα, περιέχει όλες τις απαραίτητες υπερ-παραμέτρους για την κατάλληλη ρύθμιση κάθε αλγόριθμου, οργανωμένες στα επιμέρους λεξικά. Ένα μεγάλο μέρος από αυτές είναι κοινό στους περισσότερους αλγόριθμους και για αυτό τον λόγο θα παρουσιαστούν αρχικά οι κοινές παράμετροι και έπειτα κατά την αναφορά σε κάθε αλγόριθμο, θα περιγράφονται ξεχωριστά οι επιμέρους παράμετροι που τον αφορούν.

Γενικές υπερ-παράμετροι

Αρχικά, το λεξικό `env` αφορά κατά κύριο λόγο την οργάνωση και έλεγχο των περιβαλλόντων συλλογής και αξιολόγησης. Η παράμετρος `collector_env_num` ορίζει τον αριθμό των παράλληλων διεργασιών περιβάλλοντος χρησιμοποιούνται από τον `collector` για την συλλογή εμπειριών, επηρεάζοντας την ταχύτητα συλλογής. Αντίστοιχα, η παράμετρος `evaluator_env_num` ορίζει τον αριθμό των περιβαλλόντων που χρησιμοποιούνται στην περιοδική αξιολόγηση του πράκτορα από τον `evaluator`, ενώ η `n_evaluator_episode` καθορίζει το πλήθος των επεισοδίων κάθε κύκλου αξιολόγησης, συμβάλλοντας στην ακρίβεια της. Για τις τρεις αυτές παραμέτρους επιλέγεται στους περισσότερους αλγόριθμους ο αριθμός 8. Ο τερματισμός της εκπαίδευσης από τον `evaluator` πραγματοποιείται όταν η μέση ανταμοιβή του πράκτορα υπερβεί την τιμή της παραμέτρου `stop_value=20` για το περιβάλλον Pong, ενώ στο SpaceInvaders επιλέγεται μία τιμή που τείνει στο άπειρο. Τέλος, η παράμετρος `frame_stack` καθορίζει πόσα συνεχόμενα καρέ (frames) ομαδοποιούνται (`frame_stack=4`) και δίνονται ως είσοδος ώστε ο πράκτορας να μπορεί να αντιληφθεί χρονική πληροφορία [20].

Το λεξικό `policy` αφορά παραμέτρους που διαμορφώνουν βασικές αρχές λειτουργίας του πράκτορα. Η `random_collect_size` ορίζει πόσα βήματα θα κάνει ο πράκτορας, εκτελώντας τυχαίες ενέργειες πριν ξεκινήσει η εκπαίδευση, προκειμένου να γεμίσει τον αποθηκευτικό χώρο εμπειριών (replay buffer) και να εξασφαλίσει την ύπαρξη αντιπροσωπευτικών δειγμάτων που διευκολύνουν την εκπαίδευση (`random_collect_size=10000`). Επιτάχυνση της εκπαίδευσης επιτυγχάνεται με την ενεργοποίηση της GPU που επισπεύδει τους υπολογισμούς, μέσω του ορισμού της boolean παραμέτρου `cuda` σε `True`. Επιπλέον, με την παράμετρο `priority=True` δίνεται προτεραιότητα στις εμπειρίες που είναι πιο σημαντικές για την διόρθωση των σφαλμάτων του πράκτορα και κατά συνέπεια για την εκπαίδευση του. Η σημασία που δίνεται στις μελλοντικές ανταμοιβές ελέγχεται από την παράμετρο `discount_factor` ή αλλιώς `gamma`. Κυμαίνεται από το 0 έως το 1 και όσο πιο κοντά είναι στο 1, τόσο μεγαλύτερη σημασία δίνεται στις μελλοντικές ανταμοιβές σε αντίθεση με τις άμεσες. Για το περιβάλλον Pong επιλέγονται υψηλές τιμές όπως 0.99 που ενισχύουν την μακροπρόθεσμη στρατηγική του πράκτορα. Τέλος, η παράμετρος `nstep` δηλώνει για πόσα βήματα υπολογίζεται το

άθροισμα της συνολικής ανταμοιβής ώστε ο πράκτορας να μάθει από μία ακολουθία, τριών στην προκειμένη εμπειριών, αντί με μόνο ένα βήμα [20].

Η βασική αρχιτεκτονική του νευρωνικού δικτύου που εκπαιδεύεται καθορίζεται από τις παραμέτρους του λεξικού `policy.model`. Ορίζεται αρχικά το σχήμα των παρατηρήσεων `obs_shape`, το οποίο είναι [4,84,84] λόγω του `frame_stack` που επιλέγεται ως 4 και του `WarpFrameWrapper` που αναπροσαρμόζει τις διαστάσεις εισόδου του περιβάλλοντος σε 84x84. Λόγω της ύπαρξης 6 δυνατών ενεργειών στα περιβάλλοντα, ορίζεται `action_shape=6`. Μετά τον καθορισμό του επιπέδου εισόδου του νευρωνικού ακολουθεί η προσαρμογή του πλήθους νευρώνων στα τρία κρυφά επίπεδα του κωδικοποιητή με την παράμετρο `encoder_hidden_size_list`. Με βάση την επιθυμητή πολυπλοκότητα που απαιτείται και μπορεί να υποστηριχθεί από το υλικό, επιλέχθηκαν τα μεγέθη [128, 128, 512] [20].

Στο λεξικό `policy.learn` περιλαμβάνονται παράμετροι που ρυθμίζουν την λειτουργία του `learner` και επηρεάζουν τον τρόπο που ο πράκτορας μαθαίνει μία πολιτική. Η παράμετρος `update_per_collect` καθορίζει πόσες φορές θα ενημερωθεί το νευρωνικό δίκτυο μετά από κάθε κύκλο συλλογής εμπειριών του `collector`. Μεγαλύτερες τιμές, όπως η επιλεγόμενη τιμή 10, εντείνουν τον χαρακτήρα εκτός πολιτικής του αλγορίθμου, δηλαδή μειώνουν την εξάρτηση από την τρέχουσα πολιτική. Το `batch_size` αναφέρεται στον αριθμό εμπειριών που χρησιμοποιούνται ανά βήμα εκπαίδευσης (`batch_size=32`), ενώ το `learning_rate` αποτελεί τον ρυθμό εκμάθησης του δικτύου, συμβάλλοντας στην ταχύτητα σύγκλισης. Τέλος, σε αλγορίθμους που διαθέτουν `target network` η παράμετρος `target_update_freq` ορίζει ανά πόσα βήματα γίνεται η ενημέρωση του (`target_update_freq=500`) [20].

Το πλήθος των εμπειριών που συλλέγονται σε κάθε φάση εκπαίδευσης καθορίζεται από την παράμετρο `n_sample` του `policy.collect` με αρχική τιμή 100, ενώ η παράμετρος `eval_freq` του `policy.eval` με τιμή 4000, ορίζει την συχνότητα αξιολόγησης της πολιτικής μετά από ορισμένα βήματα περιβάλλοντος. Το τελευταίο λεξικό που περιλαμβάνει το `main_config` είναι το `policy.other` με τα υπολεξικά `eps` και `replay_buffer`. Στο πρώτο διαμορφώνεται η στρατηγική εξερεύνησης ϵ -greedy, όπου ϵ η πιθανότητα τυχαίας επιλογής με σκοπό την εξερεύνηση. Ορίζεται ο τύπος μείωσης του ϵ με το `type`, όπως για παράδειγμα `exp`, δηλαδή εκθετικά, η αρχική τιμή του ϵ (`start`), η τελική τιμή του (`end`) και τα απαιτούμενα βήματα για αυτή την μετάβαση. Στο λεξικό `replay_buffer` δηλώνεται το συνολικό μέγεθος του χώρου αποθήκευσης εμπειριών (`replay_buffer_size=10000`) και σε περίπτωση ενεργοποιημένης προτεραιοποίησης, οι παράμετροι `alpha`, `beta` και `anneal_step` που έχουν διασαφηνιστεί σε προηγούμενη ενότητα, με τιμές 0.6, 0.4 και 20000 [20].

Αναφορικά με τα πειράματα που εκτελέστηκαν στο περιβάλλον `SpaceInvaders` αξίζει να σημειωθεί πως λόγω της μεγαλύτερης πολυπλοκότητας του, απαιτούνταν και ανάλογες τιμές στις παραμέτρους. Εξαιτίας έλλειψης επαρκών υπολογιστικών πόρων ορισμένες από αυτές, όπως το μέγεθος του `replay buffer`, έπρεπε να περιοριστούν έτσι ώστε να γίνει δυνατή η εκτέλεση τους. Συνεπώς στην εξέταση των αποτελεσμάτων θα πρέπει να ληφθούν υπόψη αυτοί οι περιορισμοί που ίσως επιβαρύνουν τις πραγματικές δυνατότητες κάποιου αλγορίθμου.

R2D2

Ο R2D2 είναι ένας Value-Based αλγόριθμος, βασισμένος στον DQN που συνδυάζει μεθόδους αναδρομικών νευρωνικών δικτύων RNN, `experience replay memory` και κατανεμημένης εκπαίδευσης [6]. Λόγω της υψηλής πολυπλοκότητας που έχουν τα RNN χρησιμοποιήθηκαν 2 παράλληλα περιβάλλοντα για τον `collector` και τον `evaluator`. Εισάγονται μερικές νέες παράμετροι όπως η `priorityISweight`, με τιμή `True`, στο λεξικό `policy` που αφορά την ενεργοποίηση ή μη της διόρθωσης

του bias που προκύπτει από την χρήση προτεραιότητας. Για την διόρθωση χρησιμοποιούνται Importance Sampling Weights από τον τύπο: $w_i = \left(\frac{1}{N} \cdot \frac{1}{P(i)}\right)^\beta$, όπου β η παράμετρος beta. Επιπλέον εισάγονται οι παράμετροι burnin_step και learn_unroll_len με τιμή 20, όπου η πρώτη ελέγχει τον αριθμό βημάτων του RNN που γίνεται ενημέρωση των κρυφών επιπέδων, χωρίς να υπολογίζεται η απώλεια (loss) και η δεύτερη ορίζει τον αριθμό βημάτων που ακολουθούν, στα οποία γίνεται κανονική εκπαίδευση με υπολογισμό απώλειας. Μια πολύ σημαντική παράμετρος για τον R2D2 και τα RNN είναι η traj_len_inf, με τιμή True, που εξασφαλίζει την διατήρηση μίας ολόκληρης ακολουθίας εμπειριών μέχρι να γίνει αληθής η μεταβλητή-σημαία done [20]. Το πείραμα εκτελέστηκε με collector_env_num=2, evaluator_env_num=2, n_evaluator_episode=2, nstep=3, update_per_collect=8, batch_size=32, learning_rate= 0.0005 και n_sample=32.

Rainbow

Ο αλγόριθμος Rainbow αποτελεί μια ενισχυμένη έκδοση του DQN, ο οποίος περιλαμβάνει τις εξής επεκτάσεις:

- Double DQN: Χρησιμοποιούνται δύο δίκτυα για την εκτίμηση του Q-value
- Prioritized Experience Replay
- Αρχιτεκτονική Dueling
- Multi-step learning: με χρήση nstep
- Distributional Q-Learning (C51)
- Noisy Nets: εισαγωγή θορύβου στα βάρη για ώθηση του πράκτορα στην εξερεύνηση (αντικαθιστά την ϵ -greedy) [25].

Στο λεξικό policy.model συναντώνται τρεις παράμετροι που αφορούν την Distributional Q-Learning, η v_min, v_max και n_atom. Η Q-value μετατρέπεται σε μία διακριτή κατανομή από n_atoms σημεία και v_min, v_max τα κάτω και πάνω όρια της [20]. Θέτεται v_min=-10, v_max=10 και n_atom=51. Η εκτέλεση του πειράματος έγινε με learning_rate=0.0001.

QRDQN

Ο Quantile Regression DQN (QRDQN) είναι ένας Value-Based αλγόριθμος που αποτελεί μία κατανεμημένη μορφή του DQN. Συγκεκριμένα, αντί να προβλέπει μία μόνο τιμή Q, μαθαίνει ολόκληρη κατανομή τιμών μέσω καθορισμένου αριθμού ποσοστημορίων (quantiles). Επιλέγονται δηλαδή 64 σταθερά quantiles με σκοπό την προσέγγιση της κατανομής των πιθανών τιμών Q για κάθε ενέργεια. Αντίθετα με τον DQN επίσης, χρησιμοποιείται quantile huber loss έναντι του απλού loss. Ο αλγόριθμος πετυχαίνει καλύτερη μοντελοποίηση της αβεβαιότητας και συχνά αποδίδει καλύτερα του DQN και C51 [20]. Το πείραμα εκτελέστηκε με learning_rate= 0.0001.

FQF

Ο αλγόριθμος Fully-Parameterized Quantile Function (FQF) είναι μία εξελιγμένη μορφή του QRDQN που δεν μαθαίνει μόνο τις τιμές των quantiles αλλά και τις θέσεις τους στην κατανομή. Εν αντιθέσει με τα σταθερά quantiles του QRDQN, ο FQF μαθαίνει δυναμικά τις θέσεις των quantiles, μέσω του Quantile Fraction Proposal Network, έτσι ώστε να προσαρμόζονται καλύτερα στα δεδομένα. Με αυτό τον τρόπο εστιάζει σε περιοχές υψηλής μεταβλητότητας των ανταμοιβών και επιτυγχάνει καλύτερη απόδοση σε σύνθετα περιβάλλοντα [20].

Ο ρυθμός μάθησης του Quantile Fraction Proposal Network ρυθμίζεται από την παράμετρο learning_rate_fraction με τιμή 2.5e-9, ενώ ο κλασικός ρυθμός μάθησης αντικαθίσταται από την παράμετρο learning_rate_quantile, με τιμή 0.00005, που αφορά το κύριο δίκτυο για την πρόβλεψη

της τιμής των quantiles. Επίσης, η παράμετρος kappa, με τιμή 1, ρυθμίζει το Huber Loss. Για σφάλματα μικρότερα του kappa το loss είναι τετραγωνικό (MSE), ενώ αντίθετα είναι γραμμικό (MAE) [20]. Η εκτέλεση του πειράματος έγινε με num_quantiles=32.

PPO

Ο PPO είναι αλγόριθμος Policy Gradient που χρησιμοποιεί την αρχιτεκτονική Actor-Critic. Αξιοποιούνται δύο νευρωνικά δίκτυα, το Policy Network και το Value Function Network. Το πρώτο υπολογίζει την βέλτιστη ενέργεια, ενώ το δεύτερο παράγει την αξία Q, της ενέργειας βάση της κατάστασης. Χρησιμοποιούνται τεχνικές clipping για μειώσουν τις ασταθείς αλλαγές ανά βήμα εκπαίδευσης [3].

Στο policy.model ρυθμίζεται το πλήθος των νευρώνων στα κρυφά επίπεδα των actor και critic δικτύων σε 128 με τις παραμέτρους actor_head_hidden_size και critic_head_hidden_size. Παράλληλα στο λεξικό policy.learn ελέγχεται πόσες φορές θα γίνει η εκπαίδευση σε κάθε δεσμίδα δεδομένων που συλλέγεται από το περιβάλλον με την παράμετρο epoch_per_collect, με τιμή 10 καθώς και η σημαντικότητα που δίνεται στο value function (value_weight=0.5) και στην εξερεύνηση (entropy_weight=0.001). Επίσης ορίζεται ο λόγος μεταξύ παλιάς και νέας πολιτικής στο clipping (clip_ratio=0.2), η κανονικοποίηση του advantage σε κάθε δεσμίδα δεδομένων (adv_norm=True), η κανονικοποίηση της εξόδου του value function (value_norm=True) και η μέγιστη επιτρεπτή τιμή κανονικοποίησης των gradients (grad_clip_value=0.5) [20]. Το πείραμα γίνεται με encoder_hidden_size_list=[64, 64, 128], batch_size=320, learning_rate=3e-4, n_sample=3200 και eval_freq=5000.

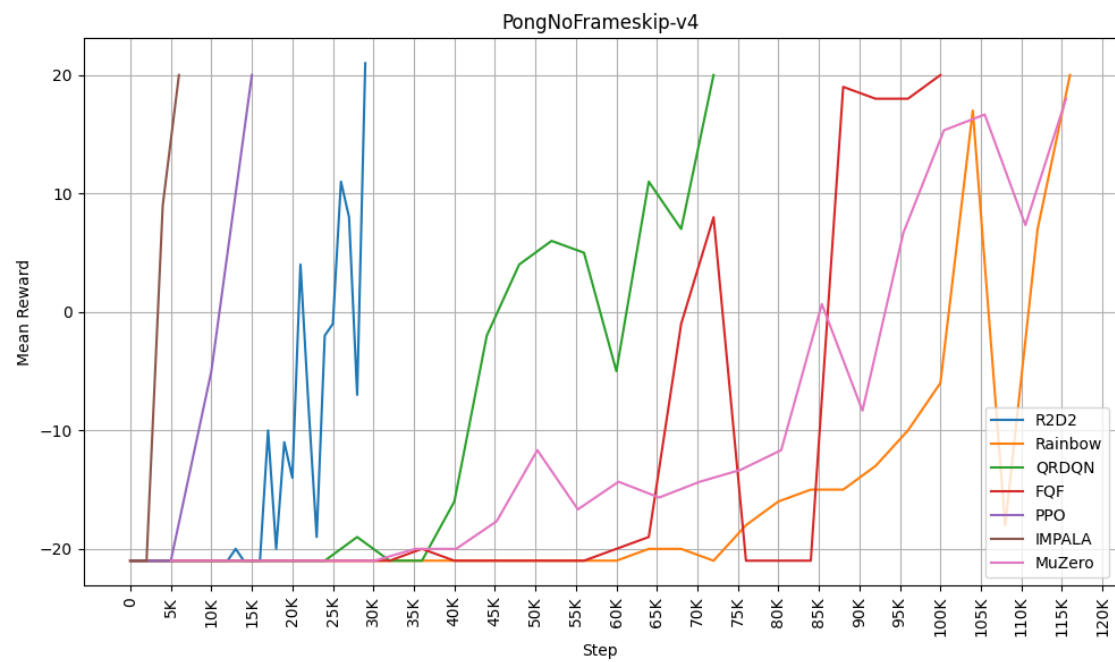
IMPALA

Ο IMPALA είναι ένας Policy Gradient αλγόριθμος που αποτελείται από ένα σύνολο actors και έναν ή περισσότερους learners, αποτελεί δηλαδή μία κατανεμημένη προσέγγιση με μεγάλες δυνατότητες επεκτασιμότητας [11]. Στο μοντέλο που εκπαιδεύεται ορίζονται δύο κρυφά επίπεδα στα δίκτυα actor και critic (critic_head_layer_num= actor_head_layer_num=2), επιπλέον συντελεστής discount (lambda= 0.950) καθώς και ορίζονται επιμέρους μεγέθη της εξίσωσης του μηχανισμού V-trace (rho_clip_ratio=1.0, c_clip_ratio=1.0, rho_pg_clip_ratio=1.0) [20]. Οι υπόλοιπες παράμετροι που εφαρμόστηκαν και αποκλίνουν από τις προκαθορισμένες είναι collector_env_num = 12, unroll_len=64, encoder_hidden_size_list=[64, 128, 256], batch_size=128, learning_rate=0.0006, value_weight=0.5, entropy_weight=0.01, n_sample=16 και eval_freq=2000.

MuZero

Ο MuZero αποτελεί ένας Model-Based αλγόριθμος ενισχυτικής μάθησης, ο οποίος προβλέπει αποκλειστικά απαραίτητα στοιχεία του περιβάλλοντος μέσω κρυφών καταστάσεων, χωρίς να χρειάζεται να μάθει ολόκληρο το περιβάλλον [13]. Ο αλγόριθμος εκπαιδεύεται με την βοήθεια ενός διαφορετικού framework από το OpenDILab, το LightZero, που λειτουργεί με παρόμοιο τρόπο με το DI-engine, εστιάζοντας ωστόσο σε Model-Based τεχνικές. Για την εκπαίδευση του αλγορίθμου στα περιβάλλοντα, χρησιμοποιούνται ανάλογες παράμετροι με αυτές του DI-engine, με learning_rate=0.2, batch_size=256 και target_update_freq=100 για τα πειράματα.

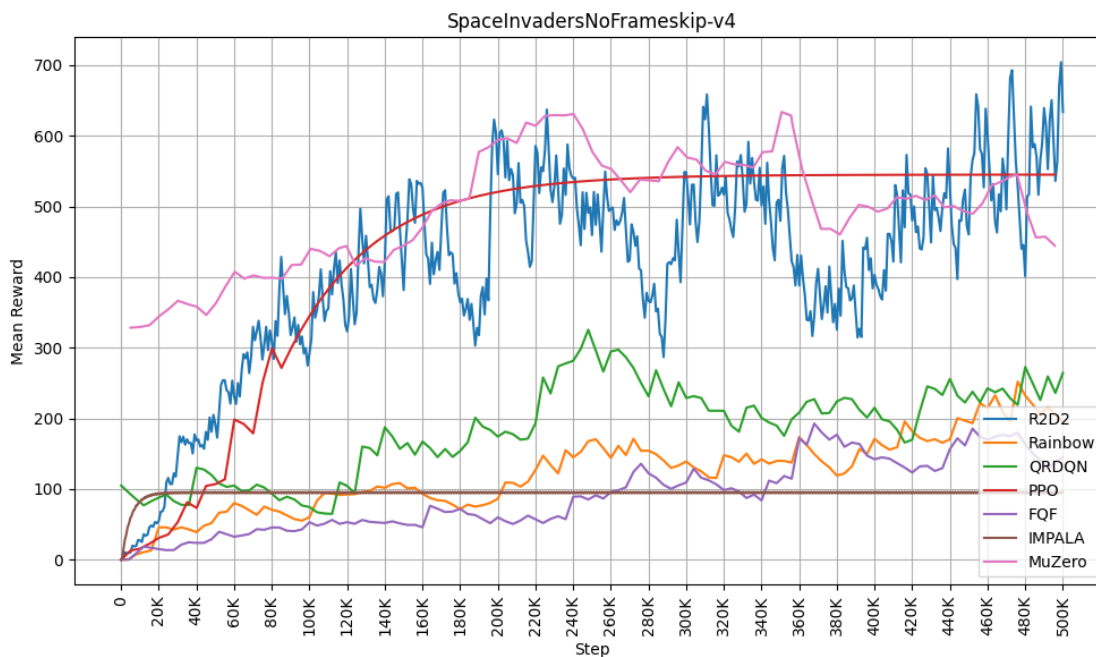
4.4.2 Αποτελέσματα πειραμάτων



Σχήμα 4.6: Σύγκριση αλγορίθμων στο περιβάλλον PongNoFrameskip-v4.

Πίνακας 4.2: Ανάλυση ανταμοιβών του καλύτερου αλγορίθμου, IMPALA.

Βήματα	Ελάχιστη Ανταμοιβή	Μέση Ανταμοιβή	Μέγιστη Ανταμοιβή
0	-21	-21	-21
1000	-21	-21	-21
2000	-21	-21	-21
3000	-6	-6	-6
4000	9	9	9
5000	14.5	14.5	14.5
6000	20	20	20



Σχήμα 4.7: Σύγκριση αλγορίθμων στο περιβάλλον *SpaceInvadersNoFrameskip-v4*.

4.4.3 Συμπεράσματα

PongNoFrameskip-v4

Στο σχήμα 4.6 παρουσιάζεται η εξέλιξη της μέσης ανταμοιβής (mean reward) για επτά διαφορετικούς αλγορίθμους ενισχυτικής μάθησης στο περιβάλλον *PongNoFrameskip-v4*. Ως γενική παρατήρηση, όλοι οι αλγόριθμοι επιτυγχάνουν στην εκμάθηση του περιβάλλοντος σε ικανοποιητικό βαθμό, καθώς καταλήγουν στην μέση ανταμοιβή 20, που αποτελεί την συνθήκη τερματισμού. Ωστόσο ο ρυθμός μάθησης και η σταθερότητα της επίδοσης έχουν σημαντικές διαφορές.

Οι αλγόριθμοι IMPALA και PPO ξεχωρίζουν από τους υπόλοιπους για την γρήγορη σύγκλιση τους, η οποία πραγματοποιείται μέσα σε μόλις 10 χιλιάδες βήματα. Αυτό οφείλεται λόγω της παραλληλοποίησης του IMPALA και την εντός πολιτικής (on-policy) συμπεριφορά του PPO, που αποβαίνουν ιδιαίτερα αποτελεσματικές σε απλά περιβάλλοντα όπως το *Pong*, που επιστρέφουν συχνές ανταμοιβές.

Ο R2D2 χρειάζεται περισσότερα βήματα για να εκπαιδευτεί και παρουσιάζει μία αστάθεια στην εκπαίδευση του, ωστόσο πετυχαίνει αρκετά καλή απόδοση, λιγότερο από 30 χιλιάδες βήματα, με κάποια επιβάρυνση στον χρόνο εκπαίδευσης λόγω της πολυπλοκότητας του μοντέλου που χρησιμοποιεί. Οι αλγόριθμοι QRDQN, FQF, Rainbow δίνουν μεγαλύτερη έμφαση στην εξερεύνηση του περιβάλλοντος και για αυτό το λόγο εμφανίζουν καθυστέρηση στην έναρξη μάθησης, αλλά προσεγγίζουν υψηλά επίπεδα απόδοσης σε ικανοποιητικό αριθμό βημάτων. Ιδιαίτερα ασταθής φαίνεται η εκπαίδευση του FQF πιθανότατα εξαιτίας της ευαισθησίας σε υπερ-παραμέτρους σχετικές με την εξισορρόπηση εξερεύνησης-εκμετάλλευσης του περιβάλλοντος (exploration-exploitation). Ο Model-Based MuZero επιτυγχάνει απόδοση όμοια με αυτή του Rainbow. Παρά το γεγονός ότι πρόκειται για έναν πολύ ισχυρό αλγόριθμο, το περιβάλλον *Pong* μπορεί να λυθεί περισσότερο αποδοτικά χωρίς να απαιτείται κάποια διαδικασία μοντελοποίησης του.

Συμπερασματικά, το *Pong* αποδεικνύεται ένα περιβάλλον που δεν απαιτεί μεγάλη εξερεύνηση και συλλογή εμπειριών προκαταβολικά. Η «άπληστη» επιλογή της καλύτερης ενέργειας κάθε φορά,

χαρακτηριστικό των Policy Gradient αλγορίθμων, αποβαίνει αρκετή ώστε να επιτευχθεί η ταχύτερη σύγκλιση στο επιθυμητό αποτέλεσμα. Έτσι αλγόριθμοι όπως ο IMPALA και ο PPO ξεπερνούν την επίδοση αλγορίθμων υψηλότερης πολυπλοκότητας με στρατηγικές εξερεύνησης.

SpaceInvadersNoFrameskip-v4

Στο σχήμα 4.7 εμφανίζεται αντίστοιχα η εξέλιξη της μέσης ανταμοιβής για τους ίδιους αλγορίθμους ενισχυτικής μάθησης με ορισμένες αλλαγές στις υπερ-παραμέτρους για να υποστηρίξουν το πιο απαιτητικό περιβάλλον SpaceInvadersNoFrameskip-v4. Σε αντίθεση με το Pong, δεν υπάρχει κάποιος τερματικός στόχος για τις ανταμοιβές και συνεπώς αξιολογείται η επίδοση των αλγορίθμων μετά από πεντακόσιες χιλιάδες βήματα.

Ο R2D2 πετυχαίνει σαφώς την καλύτερη επίδοση και επιτυγχάνει ανταμοιβές μεγαλύτερες από 600 παρά την έντονη αστάθεια που παρουσιάζει κατά την εκπαίδευση. Η επιστράτευση των RNN στο μοντέλο του αλγορίθμου είναι κρίσιμη καθώς το περιβάλλον απαιτεί μακροπρόθεσμες στρατηγικές από τον πράκτορα, προκειμένου να πετύχει την μεγαλύτερη δυνατή ανταμοιβή. Η ανοδική πορεία του γραφήματος φανερώνει ακόμα καλύτερες επιδόσεις όσο εκτυλίσσεται η εκπαίδευση. Παράλληλα η επίδοση του PPO είναι εντυπωσιακή καθώς σε λιγότερο από 200 χιλιάδες βήματα επιτυγχάνει ανταμοιβή που ξεπερνά τις 500 μονάδες. Το αποτέλεσμα αυτό προκύπτει επίσης σε σχετικά μικρό χρόνο εκπαίδευσης, ωστόσο η άπληστη πολιτική του PPO οδηγεί την εκπαίδευση να σταθεροποιηθεί λίγο πάνω από τις 500 μονάδες χωρίς περιθώρια βελτίωσης με περαιτέρω εκπαίδευση. Αντίθετα με τον PPO, ο MuZero έχει μεγάλες προοπτικές εξέλιξης αφού αν εκπαιδευτεί για περισσότερα βήματα ξεπερνά και τον R2D2. Η φύση του MuZero απαιτεί μεγαλύτερες περιόδους εκπαίδευσης προκειμένου να επιλύσει πολύπλοκα προβλήματα με αρκετά υψηλές ανταμοιβές.

Οι αλγόριθμοι QRDQN και Rainbow εμφανίζονται στο γράφημα να μην έχουν την ίδια επιτυχία με τους προηγούμενους. Εντούτοις η εκπαίδευση τους είναι αρκετά σταθερή και το γράφημα παρουσιάζει ανοδική γραμμική πορεία. Συνεπώς στα βήματα εκπαίδευσης που εκτελέστηκαν πέραν του περιορισμένου γραφήματος, και οι δύο αλγόριθμοι ξεπερνούν την επίδοση του PPO. Η αργή σύγκλιση οφείλεται στην εξερεύνηση του περιβάλλοντος, στην οποία σε αντίθεση με Policy Gradient μεθόδους δίνεται μεγαλύτερη έμφαση.

Στο πολύπλοκο περιβάλλον του SpaceInvaders οι αλγόριθμοι FQF και IMPALA κρίνονται ανεπαρκείς για την επίλυση του. Ο FQF στο γράφημα εμφανίζεται να πετυχαίνει παρόμοιες επιδόσεις με αυτές του Rainbow και QRDQN, όμως στην μετέπειτα εκπαίδευση του εμφανίζεται καθοδική πορεία και οι ανταμοιβές φαίνονται να συγκλίνουν προς το 0. Ο IMPALA πετυχαίνει σταθερά ανταμοιβές 100 μονάδων, δίχως να βελτιώνεται κατά την εκπαίδευση του. Για τα αποτελέσματα αυτά πέραν της δυσκολίας του περιβάλλοντος, οφείλεται και η ανεπάρκεια από πλευράς υλικού. Συγκεκριμένα, ενώ το περιβάλλον συνιστούσε μεγάλη χωρητικότητα στον replay buffer, δεν ήταν δυνατή η υποστήριξη της χωρίς η εκπαίδευση να τερματίζεται απότομα λόγω έλλειψης μνήμης.

Συμπερασματικά, το περιβάλλον SpaceInvaders αποδεικνύεται ένα ιδιαίτερα απαιτητικό περιβάλλον, που απαιτεί πιο σύνθετες στρατηγικές μάθησης και εξερεύνησης του και για αυτό το λόγο Value-Based και προφανώς Model-Based προσεγγίσεις υπερτερούν των Policy Gradient. Εν αντιθέσει με το περιβάλλον Pong απαιτείται πολύ μεγαλύτερος χρόνος εκπαίδευσης για την επίτευξη αρκετά υψηλών ανταμοιβών και συνεπώς διακρίνονται μακροπρόθεσμα αλγόριθμοι που δίνουν μεγάλη έμφαση στην εξερεύνηση και συλλογή εμπειριών.

Κεφάλαιο 5

Συμπεράσματα και Μελλοντική Έρευνα

Η παρούσα πτυχιακή εργασία είχε στόχο τη μελέτη της ενισχυτικής μάθησης (Reinforcement Learning) μέσω της θεωρητικής διερεύνησης βασικών και προηγμένων αλγορίθμων καθώς και την πειραματική αξιολόγηση τους σε περιβάλλον εκπαίδευσης. Στο κεφάλαιο αυτό περιγράφονται συμπεράσματα που αφορούν τόσο τα πειράματα που διενεργήθηκαν, όσο και της γενικότερης διαδικασίας εκπόνησης της εργασίας. Επιπρόσθετα, προτείνονται επιμέρους θέματα για μελλοντική έρευνα στην περιοχή της ενισχυτικής μάθησης.

Η ενισχυτική μάθηση αποτελεί έναν από τους πιο δυναμικούς και πολλά υποσχόμενους τομείς της μηχανικής μάθησης. Αποτελεί ιδιαίτερα αποτελεσματική λύση στην ανάπτυξη αυτόνομων και ευφυών συστημάτων, στην βελτιστοποίηση στρατηγικών βιομηχανικών περιβαλλόντων και σε πολλές άλλες εφαρμογές. Πρόκειται για έναν τομέα που γνώρισε σχετικά πρόσφατα μεγάλη άνθηση και σε συνδυασμό με την πολυπλοκότητα που παρουσιάζει, κρίνεται αναγκαία η ύπαρξη εργαλείων που μπορούν να εισάγουν έναν χρήστη στην αξιοποίηση του.

Η διαρκής εξέλιξη εργαλείων και πλαισίων (frameworks) ανοιχτού κώδικα, όπως το DI-engine, καθιστά την ενισχυτική μάθηση περισσότερο προσιτή στην κοινότητα των ερευνητών και προγραμματιστών. Το DI-engine επέτρεψε την ευκολότερη υλοποίηση, παρέχοντας τους αλγορίθμους και τον κύκλο διεργασιών για την εκπαίδευση τους, ενώ παράλληλα έδωσε την δυνατότητα για τροποποιήσεις στο μεγάλο πλήθος υπερ-παραμέτρων και στο περιβάλλον εκπαίδευσης. Με την συνεχή ανάπτυξη, βελτίωση και τεκμηρίωση τέτοιων συστημάτων ενισχύεται η προσβασιμότητα της ενισχυτικής μάθησης και γίνεται δυνατή η αξιοποίηση της στην λύση περισσότερων καθημερινών προβλημάτων.

Μέσα από τα πειράματα που διεξήχθησαν έγινε αντιληπτή η αποδοτικότητα αλγορίθμων Policy Gradient σε απλά περιβάλλοντα, όπως το Pong, που δεν απαιτούν μακροπρόθεσμη στρατηγική με εξερεύνηση του περιβάλλοντος. Αντίθετα, αλγόριθμοι με προηγμένους μηχανισμούς εξερεύνησης και στοχαστικότητας επιτυγχάνουν καλύτερες επιδόσεις σε πιο σύνθετα περιβάλλοντα εκπαίδευσης, όπως το SpaceInvaders. Συνεπώς η επιλογή του κατάλληλου αλγορίθμου εξαρτάται από το επίπεδο δυσκολίας και τον τύπο του εκάστοτε προβλήματος, ενώ πολύ σημαντική κρίνεται και η επιλογή των κατάλληλων υπερ-παραμέτρων που ρυθμίζουν την λειτουργία του αλγορίθμου.

Ως μελλοντική έρευνα, προτείνονται διάφορες επεκτάσεις που θα μπορούσαν να γίνουν στους αλγορίθμους που δοκιμάστηκαν και στην διαφοροποίηση των περιβαλλόντων εκπαίδευσης. Αρχικά, θα παρουσίαζε ενδιαφέρον η επιλογή περισσότερων περιβαλλόντων διαβαθμισμένης δυσκολίας, που επιστρέφουν σπανιότερα ανταμοιβές, με στόχο την ανάδειξη των πλεονεκτημάτων περισσότερων αλγορίθμων. Σε ιδιαίτερα δύσκολα περιβάλλοντα θα μπορούσαν να επιστρατευτούν οι δυνατότητες περισσότερων Model-Based αλγορίθμων στον σχεδιασμό στρατηγικής και στην αποδοτική αξιοποίηση δειγμάτων.

Ορισμένοι αλγόριθμοι που δοκιμάστηκαν θα μπορούσαν να ενισχυθούν με αναδρομικά επίπεδα RNN αλλά και μηχανισμούς εξερεύνησης ICM, RND ή RE3, όπως παρουσιάστηκαν στο exploration module του 3^{ου} κεφαλαίου. Τέλος, ιδιαίτερο ενδιαφέρον έχει η ενσωμάτωση μοντέλων Transformer στην θέση των κλασικών αρχιτεκτονικών, καθώς υπερέχουν των RNN στην αποτύπωση μακροπρόθεσμων εξαρτήσεων, καθιστώντας τον αλγόριθμο πιο αποδοτικό σε απαιτητικά περιβάλλοντα εκπαίδευσης.

Αναφορές

- [1] Τεχνητή Νοημοσύνη, Ι. Βλαχάβας, Π.Κεφαλάς, Ν. Βασιλειάδης, Φ. Κόκκορας, Η.Σακελλαρίου.
- [2] Εισαγωγή στη Μηχανική Μάθηση, Γρηγόριος Τσουμάκας.
- [3] Y. Liu, J. Yang, L. Chen, T. Guo, and Y. Jiang, "Overview of Reinforcement Learning Based on Value and Policy", 2020.
- [4] Y. Li, "Deep Reinforcement Learning: An Overview", 2018.
- [5] Reinforcement Learning: An Introduction, Richard S. Sutton, Andrew Barto.
- [6] Steven Kapturowski, Georg Ostrovski, John Quan, Remi Munos, Will Dabney, "Recurrent Experience Replay in Distributed Reinforcement Learning", 2018.
- [7] Adrià Puigdomènech Badia, Bilal Piot, Steven Kapturowski, Pablo Sprechmann, Alex Vitvitskyi, Daniel Guo, Charles Blundell, "Agent57: Outperforming the Atari Human Benchmark", 2020.
- [8] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, Sergey Levine, "Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor", 2018.
- [9] Karl Cobbe, Jacob Hilton, Oleg Klimov, John Schulman, "Phasic Policy Gradient", 2018.
- [10] Md Masudur Rahman, Yexiang Xue, "ROBUST POLICY OPTIMIZATION IN DEEP REINFORCEMENT LEARNING", 2022.
- [11] Lasse Espeholt, Hubert Soyer, Remi Munos, Karen Simonyan, Volodymyr Mnih, Tom Ward, Yotam Doron, Vlad Firoiu, Tim Harley, Iain Dunning, Shane Legg, Koray Kavukcuoglu, "IMPALA: Scalable Distributed Deep-RL with Importance Weighted Actor-Learner Architectures", 2018.
- [12] Denis Yarats, Rob Fergus, Alessandro Lazaric, Lerrel Pinto, "Mastering Visual Continuous Control: Improved Data-Augmented Reinforcement Learning", 2021.
- [13] Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, Timothy Lillicrap, David Silver, "Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model", 2020.
- [14] ShengjieWang, Shaohuai Liu, Weirui Ye, Jiacheng You, Yang Gao, "EfficientZero V2: Mastering Discrete and Continuous Control with Limited Data", 2024.
- [15] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, Timothy Lillicrap, "Mastering Diverse Domains through World Models", 2024
- [16] Ramanan Sekar, Oleh Rybkin, Kostas Daniilidis, Pieter Abbeel, Danijar Hafner, Deepak Pathak, "Planning to Explore via Self-Supervised World Models", 2020.
- [17] Deepak Pathak, Pulkit Agrawal, Alexei A. Efros, Trevor Darrell, "Curiosity-driven Exploration by Self-supervised Prediction", 2017.
- [18] Yuri Burda, Harrison Edwards, Amos Storkey, Oleg Klimov, "EXPLORATION BY RANDOM NETWORK DISTILLATION", 2018.
- [19] Younggyo Seo, Lili Chen, Jinwoo Shin, Honglak Lee, Pieter Abbeel, Kimin Lee, "State Entropy Maximization with Random Encoders for Efficient Exploration", 2021.

- [20] OpenDILab <https://github.com/opensdilab/DI-engine>, 2021.
- [21] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, Soumith Chintala, "PyTorch: An Imperative Style, High-Performance Deep Learning Library", 2019.
- [22] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, Travis E. Oliphant, "Array Programming with NumPy", 2020.
- [23] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, Wojciech Zaremba, "OpenAI Gym", 2016.
- [24] Farama Foundation, <https://gymnasium.farama.org/v0.28.1/environments/atari/>, 2022.
- [25] Matteo Hessel, Joseph Modayil, Hado van Hasselt, Tom Schaul, Georg Ostrovski, Will Dabney, Dan Horgan, Bilal Piot, Mohammad Azar, David Silver, "Rainbow: Combining Improvements in Deep Reinforcement Learning", 2017.